

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل بیست و یکم: Link Analysis

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

21.1 The Web as a graph

21.1.1 Anchor text and the web graph

21.2 PageRank

21.2.1 Markov chains

21.2.2 The PageRank computation

21.2.3 Topic-specific PageRank

21.3 Hubs and Authorities

21.3.1 Choosing the subset of the Web

21.4 Chapter 21 Summary

21.5 End of Chapter Exercises

21.1 وب به عنوان یک گراف: قدرت نادیده گرفته لینکها

از ارجاعات علمی تا لینکهای وب 🔍

در دنیای آکادمیک، اهمیت یک مقاله علمی با تعداد دفعاتی که توسط مقالات دیگر به آن ارجاع داده می‌شود، سنجیده می‌شود. این ایده که «یک ارجاع، نوعی انتقال اعتبار است»، پایه و اساس یک حوزه به نام **Bibliometrics** است.

موتورهای جستجوی وب، به‌ویژه گوگل، این ایده را به دنیای وب آوردند. آنها فرض کردند که یک **لینک** از صفحه A به صفحه B، مانند یک ارجاع علمی، نوعی **تأیید اعتبار** برای صفحه B توسط سازنده صفحه A است. این ایده ساده اما قدرتمند، اساس الگوریتم‌های رتبه‌بندی مدرن را شکل داد.

📄 قدرت متن لنگر: وقتی وب‌سایت شما خودش را نمی‌شناسد

اولین شهودی که از تحلیل لینک به دست می‌آید، قدرت **متن لنگر** است - متنی که در یک لینک قابل کلیک قرار می‌گیرد. اغلب، این متن توصیف دقیق‌تری از صفحه مقصد ارائه می‌دهد تا خود صفحه.

بیایید یک مثال واقعی را بررسی کنیم: صفحه اصلی شرکت Airbnb در گذشته، کلمه «اجاره آپارتمان» را در متن خود نداشت! این صفحه بیشتر بر روی «تجربه منحصربه‌فرد» و «میزبان‌های محلی» تمرکز داشت. اما وبلاگ‌های مسافرتی و رسانه‌ها به این صفحه با متن‌های لنگری مانند «اجاره خانه برای سفر» یا «پلتفرم اجاره اقامتگاه» لینک داده‌اند.

نتیجه؟ وقتی کاربران عبارت «اجاره آپارتمان» را جستجو می‌کردند، صفحه Airbnb در نتایج بالا ظاهر می‌شد - نه به خاطر محتوای صفحه، بلکه به خاطر توصیفی که جامعه وب از آن ارائه کرده بود. این نشان می‌دهد که چگونه **جامعه نویسندگان وب** به‌طور ناخواسته نقش یک سیستم ایندکس‌دهی جمعی را ایفا می‌کنند.

🔍 لینک به‌عنوان رأی اعتماد: چگونه گوگل «Big Blue» را کشف کرد

وقتی در گوگل عبارت «Big Blue» را جستجو می‌کنید، اولین نتیجه صفحه اصلی IBM است. چرا؟ چون هزاران وب‌سایت و وبلاگ با این متن لنگر به صفحه IBM لینک داده‌اند. گوگل از طریق تحلیل میلیون‌ها لینک، فهمید که «Big Blue» یک نام مستعار برای IBM است.

این مثال نشان می‌دهد که چگونه لینک‌ها می‌توانند ارتباطات معنایی را که در متن صفحات وجود ندارد، آشکار کنند. موتورهای جستجو این متن‌های لنگر را با وزن‌دهی هوشمند (مشابه مفهوم IDF در ایندکس‌گذاری) به واژگان ایندکس صفحه مقصد اضافه می‌کنند و معمولاً به آنها اهمیت بالاتری نسبت به متن خود صفحه می‌دهند.

⚠️ محدودیت‌ها: چه زمانی یک لینک، رأی اعتماد نیست؟

همه لینک‌ها یکسان نیستند. موتورهای جستجو به سرعت فهمیدند که برخی لینک‌ها واقعاً رأی اعتماد نیستند. برای مثال:

- تقریباً تمام صفحات یک وب‌سایت دانشگاهی لینکی به صفحه «قوانین کپی‌رایت» یا «اطلاعات تماس با ما» دارند.
- آیا این یعنی صفحه «قوانین کپی‌رایت» مهم‌ترین صفحه آن وب‌سایت است؟ قطعاً خیر.

این نوع لینک‌ها که از یک **قالب مشترک** در تمام صفحات یک سایت ناشی می‌شوند، معمولاً توسط الگوریتم‌های تحلیل لینک نادیده گرفته یا وزن بسیار کمی به آنها اختصاص داده می‌شود، زیرا آنها نمایانگر یک تأیید واقعی و آگاهانه از محتوای صفحه مقصد نیستند.

🔄 اسپم متن لنگر: وقتی توصیف‌ها به دروغ تبدیل می‌شوند

قدرت متن لنگر همچنین زمینه‌ی سوءاستفاده را فراهم می‌کند. در گذشته، رقبا به صورت برنامه‌ریزی‌شده به صفحات یک شرکت با متن‌های لنگر منفی مانند «شرکت کلاهبرداری» لینک می‌دادند تا رتبه آن را در نتایج جستجو کاهش دهند.

این پدیده که به آن **اسپم متن لنگر** (Anchor Text Spam) گفته می‌شود، یکی از چالش‌های مهم موتورهای جستجو است. برای مقابله با آن، الگوریتم‌های پیچیده‌ای توسعه داده شده‌اند که الگوهای غیرطبیعی در توزیع متن‌های لنگر را شناسایی و وزن آنها را کاهش می‌دهند یا به طور کامل نادیده می‌گیرند.

PageRank 21.2: چگونه گوگل اهمیت را در وب اندازه‌گیری می‌کند 🌐

🌐 یک جهانگرد در دنیای وب: ایده‌ی پشت PageRank

تصور کنید یک جهانگرد دیجیتال در حال گشت‌وگذار در اینترنت است. او از صفحه‌ای شروع کرده و در هر گام، یکی از لینک‌های موجود در صفحه را به صورت تصادفی انتخاب می‌کند و به آن صفحه می‌رود.

با این منطق، صفحاتی که جهانگرد بیشتر به آنها سر می‌زند، باید مهم‌تر باشند. چرا؟ چون احتمالاً صفحات مهم‌تر، لینک‌های بیشتری از صفحات دیگر دریافت کرده‌اند و جهانگرد را به سمت خود جذب می‌کنند. این ایده‌ی ساده اما ناب، اساس الگوریتم PageRank است.

🚫 دو چالش بزرگ: بن‌بست‌ها و جزیره‌های جداافتاده

اما جهانگرد ما با دو مشکل اساسی روبرو است:

1. **بن‌بست‌ها (Sink Nodes):** صفحه‌ای بدون هیچ لینک خروجی. جهانگرد به این صفحه می‌رسد و در آن گیر می‌کند.
2. **جزیره‌های جداافتاده:** ممکن است جهانگرد در یک بخش کوچک از وب (مثلاً مجموعه‌ای از سایت‌های یک دانشگاه خاص) گیر کند و هرگز به بخش‌های دیگر وب نرسد.

🌟 راه‌حل جادویی: عملیات تله‌پورت (Teleport)

برای حل این مشکلات، به جهانگرد ما یک توانایی جادویی می‌دهیم: **تله‌پورت**. در هر گام، جهانگرد با احتمال کوچکی (معمولاً ۰.۱ یا ۱۰٪) از صفحه فعلی «ناپدید» شده و به صورت کاملاً تصادفی به هر صفحه‌ای در وب می‌رود.

این کار مانند این است که جهانگرد خسته از صفحه فعلی شده و یک آدرس جدید و تصادفی در نوار آدرس مرورگر خود تایپ کند. این مکانیزم ساده دو مشکل را همزمان حل می‌کند: هم از بن‌بست‌ها خارج می‌شود و هم به تمام بخش‌های وب دسترسی پیدا می‌کند.

🌐 PageRank در دنیای واقعی: داستان یک وبلاگ تکنولوژی

بیایید یک مثال واقعی را بررسی کنیم. دو وبلاگ تکنولوژی جدید، «تک‌بین» و «کدپارسی»، همزمان فعالیت خود را آغاز می‌کنند:

- **وبلاگ «تک‌بین»** موفق می‌شود از یک رسانه بزرگ معتبر مانند **وی‌جی‌بی‌آی** یک لینک دریافت کند.
- **وبلاگ «کدپارسی»** از ۵۰ وبلاگ کوچک‌تر اما تخصصی برنامه‌نویسی لینک می‌گیرد.

کدام وبلاگ احتمالاً رتبه بالاتری در گوگل خواهد داشت؟

الگوریتم PageRank به ما می‌گوید که لینک از **وی‌جی‌بی‌آی** (که خود PageRank بالایی دارد) بسیار ارزشمند است و به «تک‌بین» امتیاز قابل توجهی می‌دهد. اما در عین حال، مجموع امتیاز ۵۰ لینک از وبلاگ‌های کوچک‌تر نیز می‌تواند به همان اندازه یا حتی بیشتر باشد.

این مثال نشان می‌دهد که PageRank چگونه **اعتبار دیجیتال** را از طریق ساختار لینک‌های وب اندازه‌گیری می‌کند و به موتورهای جستجو کمک می‌کند تا صفحات معتبرتر را شناسایی کرده و در نتایج جستجو نمایش دهند.

In [2]: *# پیاده‌سازی الگوریتم PageRank*

```
import random

class PageRankSimulator:
    """
    با دو روش PageRank شبیه‌سازی الگوریتم
    PageRank پایهی اصلی محاسبه – (Iterative) روش تکراری.
    برای درک شهودی الگوریتم – Random Surfer شبیه‌سازی
    """

    def __init__(self, graph, alpha=0.1, tolerance=1e-6, max_iterations=100):
        """
        PageRank. مقداردهی اولیه شبیه‌ساز

        ورودی‌ها:
        - graph: دیکشنری نمایش‌دهنده گراف وب
          مثال: {"A": ["B", "C"], "B": ["C"], "C": ["A"]}
        - alpha: احتمال تله‌پورت (معمولاً 0.1)
        - tolerance: آستانه همگرایی برای روش تکراری
        - max_iterations: حداکثر تعداد تکرار برای جلوگیری از حلقه نامتناهی
        """

        self.graph = graph
        self.alpha = alpha
        self.tolerance = tolerance
        self.max_iterations = max_iterations

        # لیست تمام گره‌ها (صفحات)
        self.nodes = set(graph.keys())
        for neighbors in graph.values():
            self.nodes.update(neighbors)
        self.nodes = list(self.nodes)
        self.num_nodes = len(self.nodes)

        # تبدیل نام گره‌ها به اندیس برای سرعت بیشتر
        self.node_to_index = {node: i for i, node in enumerate(self.nodes)}

    def iterative_pagerank(self):
        """
        با روش تکراری (پایهی موتورهای جستجوی واقعی) PageRank محاسبه
        فرمول اصلی:
        PR(A) = (1 - α)/N + α * Σ(PR(Pi)/L(Pi))
        که در آن:
        - N = تعداد کل صفحات
        - Pi = لینک داده‌اند A صفحاتی که به
        - L(Pi) = تعداد لینک‌های خروجی صفحه Pi
        """

        N = self.num_nodes
        # مقداردهی اولیه: همه صفحات امتیاز یکسان دارند
        pagerank = [1.0 / N] * N
        new_pagerank = [0.0] * N

        for iteration in range(self.max_iterations):
            # برای همه صفحات (1-α)/N مرحله 1: محاسبه مقدار تله‌پورت
            teleport_value = (1.0 - self.alpha) / N
            new_pagerank = [teleport_value] * N

            # مرحله 2: توزیع امتیازات از طریق لینک‌ها
            for node in self.nodes:
                i = self.node_to_index[node]
                out_links = self.graph.get(node, [])
                if not out_links:
                    continue # صفحه بدون لینک خروجی، امتیازی منتقل نمی‌کند

                # هر صفحه، امتیاز خود را به‌طور یکنواخت بین لینک‌های خروجی تقسیم می‌کند
                share = (self.alpha * pagerank[i]) / len(out_links)
                for neighbor in out_links:
                    j = self.node_to_index[neighbor]
                    new_pagerank[j] += share

            # بررسی همگرایی
            diff = sum(abs(new_pagerank[i] - pagerank[i]) for i in range(N))
            if diff < self.tolerance:
                print(f"تکرار {iteration + 1} همگرایی پس از ✅")
                break

            pagerank = new_pagerank[:]

        # تبدیل نتیجه به دیکشنری خوانا
        result = {}
```

```
for i, node in enumerate(self.nodes):
    result[node] = pagerank[i]
return result

def simulate_random_surfer(self, steps=100000):
    """
    PageRank برای درک شهودی Random Surfer شبیهسازی مستقیم.
    این روش برای آموزش مناسب است، اما در عمل کارآمد نیست.
    """
    # شروع از یک صفحه تصادفی
    current = random.choice(self.nodes)
    visit_count = {node: 0 for node in self.nodes}

    for _ in range(steps):
        visit_count[current] += 1

        # تصمیم‌گیری: تله‌پورت یا پیمایش لینک؟
        if random.random() < self.alpha:
            # تله‌پورت: رفتن به صفحه‌ای تصادفی
            current = random.choice(self.nodes)
        else:
            # پیمایش لینک: انتخاب یک لینک خروجی به‌صورت تصادفی
            out_links = self.graph.get(current, [])
            if out_links:
                current = random.choice(out_links)
            else:
                # اگر لینک خروجی نداشت، تله‌پورت اجباری
                current = random.choice(self.nodes)

    # (تقریبی PageRank) تبدیل شمارش بازدیدها به احتمال
    total_visits = sum(visit_count.values())
    approx_pagerank = {node: count / total_visits for node, count in visit_count.items()}
    return approx_pagerank

# مثال کاربردی: گراف نمونه از شکل 21.1
if __name__ == "__main__":
    # گراف نمونه: A → {B, C, D}, B → {C}, C → {A}, D → {}
    sample_graph = {
        "A": ["B", "C", "D"],
        "B": ["C"],
        "C": ["A"],
        "D": [] # صفحه بدون لینک خروجی (Sink Node)
    }

    print("برای گراف نمونه PageRank مثال آموزشی")
    print("-" * 50)
    print("ساختار گراف:")
    for node, links in sample_graph.items():
        print(f" {node} → {links if links else '[] (بدون لینک خروجی)'}")

    # روش تکراری PageRank محاسبه
    pr_simulator = PageRankSimulator(sample_graph, alpha=0.1)
    iterative_result = pr_simulator.iterative_pagerank()

    print("\n📊 PageRank (روش تکراری):")
    sorted_pages = sorted(iterative_result.items(), key=lambda x: x[1], reverse=True)
    for rank, (page, score) in enumerate(sorted_pages, 1):
        print(f" {rank}. صفحه {page}: {score:.6f}")

    # (را بالا ببرید steps برای همگرایی، مقدار) Random Surfer تأیید با شبیه‌سازی
    print("\n🎲 Random Surfer (100,000 گام) شبیهسازی")
    surfer_result = pr_simulator.simulate_random_surfer(steps=100000)
    surfer_sorted = sorted(surfer_result.items(), key=lambda x: x[1], reverse=True)
    for rank, (page, score) in enumerate(surfer_sorted, 1):
        print(f" {rank}. صفحه {page}: {score:.6f}")

    print("\n💡 نکته:")
    print("- لینک دریافت می‌کند B و هم از A بالاترین امتیاز را دارد چون هم از 'C' صفحه -")
    print("- لینک دریافت می‌کند A امتیاز پایینی دارد چون لینک خروجی ندارد و فقط از 'D' صفحه -")
    print("- جلوگیری می‌کند 'D' از بن‌بست صفحه (تله‌پورت) α پارامتر -")
```

برای گراف نمونه PageRank مثال آموزشی:

ساختار گراف:

A → ['B', 'C', 'D']
B → ['C']
C → ['A']
D → [] (بدون لینک خروجی)
✅ همگرایی پس از 5 تکرار

📊 PageRank (روش تکراری) نتایج:

1. صفحه C: 0.256692
2. صفحه A: 0.250669
3. صفحه B: 0.233356
4. صفحه D: 0.233356

🎲 Random Surfer (100,000 گام) شبیهسازی:

1. صفحه A: 0.346840
2. صفحه C: 0.318120
3. صفحه D: 0.168400
4. صفحه B: 0.166640

💡 نکته:

- لینک دریافت می‌کند B و هم از A بالاترین امتیاز را دارد چون هم از 'C' صفحه -
- لینک دریافت می‌کند A امتیاز پایینی دارد چون لینک خروجی ندارد و فقط از 'D' صفحه -
- جلوگیری می‌کند 'D' از بن‌بست صفحه (تله‌پورت) α پارامتر -

21.2.1 زنجیره‌های مارکوف: سفر تصادفی در وب

🧠 زنجیره مارکوف: هر قدم فقط به جای فعلی بستگی دارد

تصور کنید در یک شهر جدید هستید و از یک توریست بپرسید: "اگر من الان در میدان اصلی هستم، کجا احتمالاً بعدی است؟" او به شما لیستی از مکان‌ها و احتمال رفتن به هر کدام را می‌دهد. سپس از میدان اصلی به یکی از آن مکان‌ها می‌روید و دوباره از یک توریست در آنجا می‌پرسید: "اگر من الان اینجا هستم، کجا بعدی می‌روم؟"

این یک **زنجیره مارکوف** است! در دنیای وب، زنجیره مارکوف یک مدل ریاضی برای توصیف حرکت یک کاربر بین صفحات وب است. ویژگی کلیدی این است که **آینده فقط به حالت فعلی بستگی دارد، نه به گذشته**. این یعنی اگر شما در صفحه A هستید، احتمال رفتن شما به صفحه B فقط به صفحه فعلی (A) بستگی دارد، نه به صفحاتی که قبلاً بازدید کرده‌اید. این ویژگی که به آن **خاصیت مارکوف** می‌گویند، به ما اجازه می‌دهد تا رفتار کاربران در وب را با یک مدل ساده اما قدرتمند توصیف کنیم.

🔍 ماتریس انتقال احتمال: نقشه احتمالات وب

در زنجیره مارکوف، از یک **ماتریس انتقال احتمال** برای نمایش احتمال حرکت از یک صفحه به صفحه دیگر استفاده می‌شود. این ماتریس به شکل $N \times N$ است که N تعداد صفحات وب است.

هر درایه P_{ij} در این ماتریس نشان‌دهنده احتمال حرکت از صفحه i به صفحه j است. ویژگی‌های این ماتریس عبارتند از:

- هر درایه P_{ij} در بازه $[0, 1]$ قرار دارد.
- مجموع درایه‌های هر سطر برابر با 1 است ($\sum_j P_{ij} = 1$).
- چنین ماتریسی یک **ماتریس تصادفی** (Stochastic Matrix) نامیده می‌شود.

برای درک بهتر، به مثال زیر توجه کنید:

فرض کنید سه سایت خبری A، B و C داریم: - سایت A به سایت‌های B و C لینک دارد. - سایت B فقط به سایت A لینک دارد. - سایت C فقط به سایت A لینک دارد.

ماتریس انتقال احتمال برای این شبکه به شکل زیر خواهد بود:

$$\begin{matrix} & A & B & C \\ A & [0 & 0.5 & 0.5] \\ B & [1 & 0 & 0] \\ C & [1 & 0 & 0] \end{matrix}$$

این ماتریس نشان می‌دهد که اگر در سایت A باشیم، 50% احتمال رفتن به B و 50% احتمال رفتن به C داریم. اما اگر در B یا C باشیم، با احتمال 100% به A بازمی‌گردیم.

🌐 ساخت ماتریس انتقال از گراف وب

برای ساخت ماتریس انتقال P از گراف وب، مراحل زیر را طی می‌کنیم:

- ابتدا **ماتریس مجاورت A** را می‌سازیم: اگر از صفحه i به صفحه j لینک باشد، $A_{ij} = 1$ ، در غیر این صورت $A_{ij} = 0$.
- اگر صفحه‌ای لینک خروجی نداشته باشد (صفحه تله)، تمام درایه‌های سطر آن برابر $N/1$ می‌شود (تله‌پورت اجباری).
- در سایر صفحات، هر لینک خروجی احتمال $1/(\text{تعداد لینک‌ها})$ دریافت می‌کند.
- کل ماتریس در $(\alpha - 1)$ ضرب می‌شود.
- مقدار α/N به هر درایه اضافه می‌شود تا تله‌پورت جهانی را مدل کند.

پارامتر α (معمولاً بین 0.1 تا 0.2) احتمال **تله‌پورت** یا پرش تصادفی کاربر به هر صفحه وب را نشان می‌دهد، حتی اگر لینکی به آن صفحه وجود نداشته باشد. این ویژگی مدل را واقعی‌تر می‌کند، زیرا کاربران واقعی همیشه از طریق لینک‌ها حرکت نمی‌کنند و ممکن است مستقیماً آدرس صفحه‌ای را در نوار آدرس تایپ کنند.

به عبارت دیگر، تله‌پورت به کاربر اجازه می‌دهد که در هر لحظه با احتمال α به هر صفحه‌ای در وب (حتی صفحاتی که به آن لینک ندارد) پرش کند. این ویژگی باعث می‌شود که زنجیره مارکوف **ارگودیک** شود و تضمین کند که حالت پایدار وجود دارد.

🌈 توزیع احتمال و بردار احتمال: کجا هستیم؟

در هر لحظه، موقعیت کاربر در وب را می‌توان با یک **بردار احتمال** نمایش داد. این بردار به ما می‌گوید که کاربر با چه احتمالی در هر صفحه قرار دارد.

اگر در زمان $t=0$ کاربر در صفحه i باشد، بردار احتمال اولیه به این صورت است: درایه i برابر 1 و بقیه درایه‌ها برابر 0. پس از یک گام ($t=1$)، توزیع احتمال جدید با ضرب بردار احتمال فعلی در ماتریس انتقال به دست می‌آید: $x_1 = \sim x_0 P$.

به طور کلی، پس از t گام، توزیع احتمال برابر است با $x_t = \sim x_0 P^t$.

این فرآیند را می‌توان به این صورت تصور کرد: در هر گام، بردار احتمال فعلی (که نشان می‌دهد کاربر با چه احتمالی در هر صفحه است) با ماتریس انتقال ضرب می‌شود تا بردار احتمال جدید به دست آید.

🎯 حالت پایدار و PageRank: مقصد نهایی سفر تصادفی

اگر زنجیره مارکوف **ارگودیک** باشد (یعنی هم **غیرقابل تجزیه** و هم **نادوره‌ای** باشد)، پس از تعداد زیادی گام، توزیع احتمال به یک **حالت پایدار** همگرا می‌شود. در این حالت، احتمال حضور در هر صفحه دیگر تغییر نمی‌کند.

این حالت پایدار را می‌توان با بردار π نمایش داد که معادله زیر را برآورده می‌کند:

$$\pi \sim = \pi P$$

این بردار پایدار، **بردار ویسته چپ اصلی** ماتریس P است که به بزرگترین مقدار ویژه (که برابر 1 است) تعلق دارد.

حالا نکته کلیدی: **PageRank هر صفحه برابر با احتمال حضور در آن صفحه در حالت پایدار است!** به عبارت دیگر، PageRank صفحه i برابر با $\pi(i)$ است.

این تعریف زیبا به ما می‌گوید که PageRank یک صفحه، احتمال حضور یک کاربر تصادفی در آن صفحه پس از گشت‌وگذار طولانی در وب است. صفحاتی که PageRank بالاتری دارند، صفحاتی هستند که کاربران تصادفی بیشتر به آن‌ها سر می‌زنند.

قضیه 21.1 این را به صورت رسمی بیان می‌کند: برای هر زنجیره مارکوف ارگودیک، یک بردار احتمال پایدار منحصربه‌فرد π وجود دارد که بردار ویسته چپ اصلی P است. اگر $\eta(i, t)$ تعداد بازدیدها از حالت i در t گام باشد، آنگاه:

$$\lim_{t \rightarrow \infty} \eta(i, t) / t = \pi(i)$$

که در آن $\pi(i)$ احتمال پایدار برای حالت i است و $\pi(i) > 0$ است.

🏠 مثال واقعی: PageRank در وبسایت‌های خبری

تصور کنید یک شبکه کوچک از وبسایت‌های خبری داریم:

- **خبرگزاری الف:** یک وبسایت خبری معتبر که به خبرگزاری ب و ج لینک داده است.
- **خبرگزاری ب:** یک وبسایت متوسط که فقط به خبرگزاری الف لینک داده است.
- **خبرگزاری ج:** یک وبسایت کوچک که به خبرگزاری الف و ب لینک داده است.
- **وبلاگ د:** یک وبلاگ شخصی که فقط به خبرگزاری الف لینک داده است.

با فرض $\alpha=0.15$ (احتمال تله‌پورت)، ماتریس انتقال برای این شبکه به شکل زیر خواهد بود:

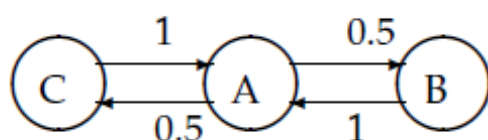
الف ب ج د الف [0.0375 0.4625 0.4625 0.0375] ب [0.0375 0.0375 0.0375 0.8875] ج [0.0375 0.0375 0.0375 0.8875] د [0.0375 0.0375 0.0375 0.8875]

پس از محاسبه حالت پایدار، PageRank این وبسایت‌ها به شکل زیر خواهد بود:

خبرگزاری الف: 0.387 خبرگزاری ب: 0.246 خبرگزاری ج: 0.215 وبلاگ د: 0.152

نتایج نشان می‌دهد که خبرگزاری الف بالاترین PageRank را دارد، زیرا هر سه وبسایت دیگر به آن لینک داده‌اند. این مثال ساده نشان می‌دهد که چگونه PageRank اهمیت یک صفحه را بر اساس تعداد و کیفیت لینک‌های ورودی به آن ارزیابی می‌کند.

جالب است بدانید که الگوریتم PageRank که توسط لری پیج و سرگی برین توسعه داده شد، اساس موفقیت اولیه گوگل بود و به گوگل کمک کرد تا نتایج جستجوی باکیفیت‌تری نسبت به رقبای خود ارائه دهد.



► **Figure 21.2** A simple Markov chain with three states; the numbers on the links indicate the transition probabilities.

شکل 21.2: یک زنجیره مارکوف ساده با سه حالت. اعداد روی لینک‌ها احتمال انتقال را نشان می‌دهند.

21.2.2 محاسبه PageRank: از تئوری تا عمل

🔗 چگونه PageRank محاسبه می‌شود؟

PageRank هر صفحه برابر است با مقدار ورودی متناظر در بردار ویژه چپ اصلی ماتریس انتقال احتمال P است. این بردار π در معادله زیر صدق می‌کند:

$$\pi P = \lambda \pi$$

که در آن λ یک مقدار ویژه است. برای توزیع ایستا، $\lambda = 1$ است، بنابراین:

$$\pi P = \pi$$

برای محاسبه این بردار، از روشی به نام **تکرار توان** (Power Iteration) استفاده می‌شود. این روش را می‌توان به این صورت تصور کرد:

تصور کنید یک کاربر اینترنت هستید که به صورت تصادفی روی لینک‌ها کلیک می‌کند. از صفحه‌ای شروع می‌کنید و سپس لینک‌های تصادفی را دنبال می‌کنید. گاهی هم (با احتمال α) به صورت تصادفی به هر صفحه‌ای در اینترنت می‌پرید (تله‌پورت). اگر این کار را برای مدت طولانی ادامه دهید، در نهایت متوجه می‌شوید که درصد زمانی که در هر صفحه می‌گذرانید به یک مقدار ثابت همگرا می‌شود. این درصد زمانی همان PageRank آن صفحه است!

از نظر ریاضی، اگر $x \sim$ توزیع اولیه احتمال روی حالت‌ها باشد، آنگاه توزیع در زمان t برابر است با $x \sim P^t$. با افزایش t ، این توزیع به یک **توزیع ایستا** همگرا می‌شود که مستقل از توزیع اولیه است (طبق قضیه 21.1).

مثال عملی: شبکه کوچک فروشگاه‌های آنلاین

فرض کنید سه فروشگاه آنلاین داریم: فروشگاه اصلی (A) و دو فروشگاه وابسته (B و C). فروشگاه اصلی به هر دو فروشگاه وابسته لینک دارد، و هر دو فروشگاه وابسته فقط به فروشگاه اصلی لینک دارند. با نرخ تله‌پورت $\alpha = 0.5$ ، ماتریس انتقال احتمال به صورت زیر است:

$$P = \begin{pmatrix} 1/6 & 2/3 & 1/6 \\ 5/12 & 1/6 & 5/12 \\ 1/6 & 2/3 & 1/6 \end{pmatrix}$$

اگر کاربر از فروشگاه اصلی (A) شروع کند (یعنی $x_0 = (1, 0, 0)$)، توزیع احتمال در هر گام به صورت زیر محاسبه می‌شود:

گام	$x_t \sim$ توزیع احتمال
0	(1, 0, 0)
1	(1/6, 2/3, 1/6)
2	(1/3, 1/3, 1/3)
3	(1/4, 1/2, 1/4)
4	(7/24, 5/12, 7/24)
∞	(5/18, 4/9, 5/18)

با گذشت زمان، توزیع به مقدار ایستای $\pi = (5/18, 4/9, 5/18)$ همگرا می‌شود. این یعنی کاربر در حالت پایدار حدود 28% زمان خود را در فروشگاه اصلی و 36% زمان را در هر یک از فروشگاه‌های وابسته می‌گذراند.

در این مثال ساده، می‌توان توزیع ایستا را مستقیماً با استفاده از تقارن گراف محاسبه کرد: حالت‌های 1 و 3 متقارن هستند (چون سطرهای اول و سوم ماتریس P یکسان‌اند). بنابراین، اگر احتمال ایستای هر یک از این دو حالت را p فرض کنیم، آنگاه توزیع ایستا به صورت $\pi = (p, 1 - 2p, p)$ خواهد بود. با جایگذاری در معادله $\pi = P \pi$ ، معادله ساده‌ای به دست می‌آید که راه‌حل آن $p = 5/18$ است.

مثال پیچیده‌تر: شبکه وبلاگ‌های فناوری

در مثال 21.1، یک گراف کوچک از وبلاگ‌های فناوری را در نظر بگیرید (شکل 21.4). این وبلاگ‌ها در مورد موضوعات مختلفی مانند خودرو، حیوانات و سرعت نوشته‌اند و به هم لینک داده‌اند. با نرخ تله‌پورت 0.14، ماتریس انتقال احتمال به صورت زیر است:

0.02	0.02	0.88	0.02	0.02	0.02	0.02
0.02	0.45	0.45	0.02	0.02	0.02	0.02
0.31	0.02	0.31	0.31	0.02	0.02	0.02
0.02	0.02	0.02	0.45	0.45	0.02	0.02
0.02	0.02	0.02	0.02	0.02	0.02	0.88
0.02	0.02	0.02	0.02	0.02	0.45	0.45
0.02	0.02	0.02	0.31	0.31	0.02	0.31

و بردار PageRank متناظر آن عبارت است از:

$$\sim x = (0.05, 0.04, 0.11, 0.25, 0.21, 0.04, 0.31)$$

در این شبکه، وبلاگ‌های q4، q3، q2 و q6 هر کدام حداقل دو لینک ورودی دارند. از میان این‌ها، q2 کمترین PageRank را دارد، چون پیمایش تصادفی تمایل دارد از بخش بالایی گراف خارج شود و تنها از طریق تله‌پورت بتواند به آن بازگردد. وبلاگ q7 بالاترین PageRank را دارد زیرا از بخش‌های مختلف گراف لینک‌های ورودی دریافت می‌کند.

🔍 PageRank در موتورهای جستجوی واقعی

مقادیر PageRank صفحات (و رتبه‌بندی ضمنی آن‌ها) **مستقل از هر پرس‌وجویی** است که کاربر ممکن است مطرح کند؛ بنابراین، PageRank یک معیار ایستا و مستقل از پرس‌وجو برای کیفیت ایستای هر صفحه وب است.

اما در موتورهای جستجوی واقعی مانند گوگل، PageRank فقط یکی از صدها عامل در رتبه‌بندی نتایج جستجو است. برای مثال، وقتی شما عبارت "خرید لپ‌تاپ" را جستجو می‌کنید، گوگل نه تنها PageRank صفحات را در نظر می‌گیرد، بلکه به عواملی مانند:

- مرتبط بودن محتوای صفحه با عبارت جستجو
- کیفیت و اعتبار صفحه
- تجربه کاربری صفحه (سرعت بارگذاری، طراحی ریسپانسیو و ...)
- موقعیت جغرافیایی کاربر
- تاریخچه جستجوی کاربر

نیز توجه می‌کند. در واقع، سهم نسبی PageRank در امتیاز کلی ممکن است توسط الگوریتم‌های یادگیری ماشین تعیین شود که با تحلیل میلیاردها جستجو، بهترین ترکیب از عوامل را برای هر نوع پرس‌وجو مشخص می‌کنند.

این رویکرد ترکیبی به گوگل اجازه می‌دهد تا نه تنها صفحات با اعتبار عمومی بالا را نمایش دهد، بلکه نتایجی را ارائه کند که دقیقاً به نیاز کاربر پاسخ می‌دهند.

In [4]: `# PageRank برای محاسبه Power Iteration پیاده‌سازی روش`

```
def power_iteration_pagerank(transition_matrix, initial_vector=None, tolerance=1e-6, max_iterations=100):
    """
    Power Iteration محاسبه بردار ویژه چپ اصلی (توزیع ایستا) با روش
    ورودی‌ها:
    - transition_matrix: P ماتریس انتقال احتمال
    - initial_vector: بردار اولیه توزیع احتمال (پیش‌فرض: توزیع یکنواخت)
    - tolerance: آستانه همگرایی
    - max_iterations: حداکثر تعداد تکرار

    خروجی:
    صفحات را نشان می‌دهد PageRank که ~π بردار ایستای -
```

```

"""
N = len(transition_matrix)

#مقداردهی اولیه: توزیع یکنواخت اگر بردار اولیه داده نشده باشد
if initial_vector is None:
    x = [1.0 / N] * N
else:
    x = initial_vector[:]

print("PageRank... برای محاسبه Power Iteration شروع روش")
print(f"{'>:6'} | {'بردار توزیع احتمال'}")
print("-" * 55)
print(f"{'0:<6'} | {[f'{v:.6f}' for v in x]}")

for iteration in range(1, max_iterations + 1):
    # محاسبه  $x_{new} = x * P$ 
    x_new = [0.0] * N
    for j in range(N): # برای هر ستون j
        for i in range(N): # برای هر سطر i
            x_new[j] += x[i] * transition_matrix[i][j]

    #  $x$  و  $x_{new}$  بررسی همگرایی: تفاوت بین
    diff = sum(abs(x_new[i] - x[i]) for i in range(N))
    print(f"iteration:<6 | {[f'{v:.6f}' for v in x_new]}")

    if diff < tolerance:
        print(f"\n✅ تفاوت = {diff:.2e} تکرار {iteration} همگرایی پس از")
        return x_new

    x = x_new[:]

print(f"\n⚠️ تکرار حاصل نشد {max_iterations} هشدار: همگرایی پس از")
return x

# مثال 1: ماتریس انتقال از شکل 21.3 (مثال کتاب)
if __name__ == "__main__":
    print("برای ماتریس انتقال شکل 21.3 PageRank مثال 1: محاسبه")
    print("=" * 60)

    # از فرمول (21.3)  $P$  ماتریس انتقال
    P_example = [
        [1/6, 2/3, 1/6], # سطر اول
        [5/12, 1/6, 5/12], # سطر دوم
        [1/6, 2/3, 1/6] # سطر سوم
    ]

    #  $x_0 = [1, 0, 0]$  بردار اولیه: شروع از حالت اول
    x0 = [1.0, 0.0, 0.0]

    result = power_iteration_pagerank(P_example, initial_vector=x0, tolerance=1e-8)

    print(f"\n{'توزیع ایستای نهایی':<10}: {result}")
    print(f"بررسی: {'result[0]:.6f'} ≈ 5/18 = {5/18:.6f}, {'result[1]:.6f'} ≈ 4/9 = {4/9:.6f}")

    print("\n" + "="*60)
    print("برای گراف شکل 21.4 PageRank مثال 2: محاسبه")
    print("="*60)

    # 21.4  $\alpha=0.14$  (با) ماتریس انتقال برای گراف 7 گره‌ای شکل
    P_fig21_4 = [
        [0.02, 0.02, 0.88, 0.02, 0.02, 0.02, 0.02],
        [0.02, 0.45, 0.45, 0.02, 0.02, 0.02, 0.02],
        [0.31, 0.02, 0.31, 0.31, 0.02, 0.02, 0.02],
        [0.02, 0.02, 0.02, 0.45, 0.45, 0.02, 0.02],
        [0.02, 0.02, 0.02, 0.02, 0.02, 0.02, 0.88],
        [0.02, 0.02, 0.02, 0.02, 0.02, 0.45, 0.45],
        [0.02, 0.02, 0.02, 0.31, 0.31, 0.02, 0.31]
    ]

    result2 = power_iteration_pagerank(P_fig21_4, tolerance=1e-8)
    print(f"\nبردار PageRank: {[f'{v:.6f}' for v in result2]}")
    print("بالاترین امتیاز را دارد، همان‌طور که در متن کتاب توضیح داده شده (ایندکس 6) d6 نکته: صفحه")

    print("\n💡 نکته:")
    print("را بدون جبر خطی پیچیده محاسبه کرد PageRank این پیاده‌سازی نشان می‌دهد که چگونه می‌توان -")
    print("در موتورهای جستجو است PageRank پایهی الگوریتم‌های واقعی محاسبه، Power Iteration روش -")
    print("بستگی دارد، نه به نقطه شروع  $\alpha$  توزیع ایستا صرفاً به ساختار گراف و مقدار -")

```

برای ماتریس انتقال شکل 21.3 PageRank مثال 1: محاسبه
=====

PageRank... برای محاسبه Power Iteration شروع روش
تکرار | بردار توزیع احتمال

0		['1.000000', '0.000000', '0.000000']
1		['0.166667', '0.666667', '0.166667']
2		['0.333333', '0.333333', '0.333333']
3		['0.250000', '0.500000', '0.250000']
4		['0.291667', '0.416667', '0.291667']
5		['0.270833', '0.458333', '0.270833']
6		['0.281250', '0.437500', '0.281250']
7		['0.276042', '0.447917', '0.276042']
8		['0.278646', '0.442708', '0.278646']
9		['0.277344', '0.445312', '0.277344']
10		['0.277995', '0.444010', '0.277995']
11		['0.277669', '0.444661', '0.277669']
12		['0.277832', '0.444336', '0.277832']
13		['0.277751', '0.444499', '0.277751']
14		['0.277791', '0.444417', '0.277791']
15		['0.277771', '0.444458', '0.277771']
16		['0.277781', '0.444438', '0.277781']
17		['0.277776', '0.444448', '0.277776']
18		['0.277779', '0.444443', '0.277779']
19		['0.277777', '0.444445', '0.277777']
20		['0.277778', '0.444444', '0.277778']
21		['0.277778', '0.444445', '0.277778']
22		['0.277778', '0.444444', '0.277778']
23		['0.277778', '0.444444', '0.277778']
24		['0.277778', '0.444444', '0.277778']
25		['0.277778', '0.444444', '0.277778']
26		['0.277778', '0.444444', '0.277778']
27		['0.277778', '0.444444', '0.277778']
28		['0.277778', '0.444444', '0.277778']

🟢 (9.93 -تفاوت =) همگرایی پس از 28 تکرار

توزیع ایستای نهایی: [0.27777777860562003, 0.4444444427887597, 0.27777777860562003]
بررسی: 0.444444 = 4/9 ≈ 0.444444, 0.277778 = 5/18 ≈ 0.277778

=====

برای گراف شکل 21.4 PageRank مثال 2: محاسبه
=====

PageRank... برای محاسبه Power Iteration شروع روش
تکرار | بردار توزیع احتمال

0		['0.142857', '0.142857', '0.142857', '0.142857', '0.142857', '0.142857', '0.142857']
1		['0.061429', '0.081429', '0.245714', '0.164286', '0.122857', '0.081429', '0.245714']
2		['0.091314', '0.055071', '0.179157', '0.233214', '0.161957', '0.055071', '0.231986']
3		['0.072111', '0.043836', '0.174322', '0.239669', '0.187713', '0.043836', '0.250395']
4		['0.070791', '0.039087', '0.151656', '0.246463', '0.195910', '0.039087', '0.273135']
5		['0.064303', '0.037130', '0.141991', '0.249491', '0.205511', '0.037130', '0.284822']
6		['0.061585', '0.036374', '0.132851', '0.251464', '0.210287', '0.036374', '0.295711']
7		['0.059020', '0.036134', '0.127623', '0.252906', '0.214379', '0.036134', '0.302737']
8		['0.057589', '0.036116', '0.123884', '0.254133', '0.217122', '0.036116', '0.308276']
9		['0.056591', '0.036195', '0.121648', '0.255268', '0.219342', '0.036195', '0.312319']
10		['0.056029', '0.036315', '0.120261', '0.256367', '0.221089', '0.036315', '0.315521']
11		['0.055714', '0.036453', '0.119514', '0.257452', '0.222577', '0.036453', '0.318091']
12		['0.055584', '0.036600', '0.119173', '0.258535', '0.223876', '0.036600', '0.320262']
13		['0.055573', '0.036751', '0.119113', '0.259619', '0.225059', '0.036751', '0.322160']
14		['0.055643', '0.036903', '0.119239', '0.260706', '0.226163', '0.036903', '0.323880']
15		['0.055768', '0.037057', '0.119490', '0.261797', '0.227218', '0.037057', '0.325483']
16		['0.055929', '0.037212', '0.119824', '0.262892', '0.228240', '0.037212', '0.327009']
17		['0.056115', '0.037368', '0.120216', '0.263992', '0.229242', '0.037368', '0.328486']
18		['0.056318', '0.037524', '0.120646', '0.265096', '0.230233', '0.037524', '0.329933']
19		['0.056533', '0.037681', '0.121102', '0.266205', '0.231217', '0.037681', '0.331362']
20		['0.056755', '0.037838', '0.121576', '0.267318', '0.232199', '0.037838', '0.332780']
21		['0.056983', '0.037997', '0.122063', '0.268436', '0.233179', '0.037997', '0.334194']
22		['0.057215', '0.038155', '0.122559', '0.269559', '0.234161', '0.038155', '0.335606']
23		['0.057450', '0.038315', '0.123062', '0.270686', '0.235144', '0.038315', '0.337019']
24		['0.057688', '0.038475', '0.123571', '0.271818', '0.236130', '0.038475', '0.338435']
25		['0.057927', '0.038636', '0.124083', '0.272955', '0.237120', '0.038636', '0.339854']
26		['0.058168', '0.038798', '0.124600', '0.274097', '0.238113', '0.038798', '0.341279']
27		['0.058411', '0.038960', '0.125119', '0.275243', '0.239110', '0.038960', '0.342708']
28		['0.058655', '0.039123', '0.125641', '0.276395', '0.240110', '0.039123', '0.344143']
29		['0.058900', '0.039287', '0.126166', '0.277551', '0.241115', '0.039287', '0.345583']
30		['0.059146', '0.039451', '0.126693', '0.278712', '0.242124', '0.039451', '0.347029']
31		['0.059393', '0.039616', '0.127222', '0.279877', '0.243136', '0.039616', '0.348481']
32		['0.059641', '0.039782', '0.127754', '0.281048', '0.244153', '0.039782', '0.349938']
33		['0.059891', '0.039948', '0.128288', '0.282223', '0.245175', '0.039948', '0.351402']
34		['0.060141', '0.040115', '0.128825', '0.283404', '0.246200', '0.040115', '0.352872']
35		['0.060393', '0.040283', '0.129364', '0.284589', '0.247230', '0.040283', '0.354348']
36		['0.060645', '0.040451', '0.129905', '0.285780', '0.248264', '0.040451', '0.355830']
37		['0.060899', '0.040621', '0.130448', '0.286975', '0.249303', '0.040621', '0.357319']
38		['0.061154', '0.040791', '0.130993', '0.288175', '0.250345', '0.040791', '0.358813']
39		['0.061409', '0.040961', '0.131541', '0.289380', '0.251392', '0.040961', '0.360314']
40		['0.061666', '0.041133', '0.132092', '0.290591', '0.252444', '0.041133', '0.361821']
41		['0.061924', '0.041305', '0.132644', '0.291806', '0.253500', '0.041305', '0.363334']
42		['0.062183', '0.041477', '0.133199', '0.293027', '0.254560', '0.041477', '0.364854']
43		['0.062443', '0.041651', '0.133756', '0.294252', '0.255625', '0.041651', '0.366380']
44		['0.062704', '0.041825', '0.134315', '0.295483', '0.256694', '0.041825', '0.367912']
45		['0.062967', '0.042000', '0.134877', '0.296719', '0.257768', '0.042000', '0.369451']
46		['0.063230', '0.042176', '0.135441', '0.297960', '0.258846', '0.042176', '0.370997']
47		['0.063494', '0.042352', '0.136008', '0.299206', '0.259928', '0.042352', '0.372548']
48		['0.063760', '0.042529', '0.136577', '0.300458', '0.261015', '0.042529', '0.374106']
49		['0.064027', '0.042707', '0.137148', '0.301714', '0.262107', '0.042707', '0.375671']
50		['0.064295', '0.042886', '0.137722', '0.302976', '0.263203', '0.042886', '0.377243']
51		['0.064563', '0.043065', '0.138298', '0.304244', '0.264304', '0.043065', '0.378820']
52		['0.064833', '0.043245', '0.138876', '0.305516', '0.265410', '0.043245', '0.380405']
53		['0.065105', '0.043426', '0.139457', '0.306794', '0.266520', '0.043426', '0.381996']
54		['0.065377', '0.043608', '0.140040', '0.308077', '0.267635', '0.043608', '0.383594']
55		['0.065650', '0.043790', '0.140626', '0.309366', '0.268754', '0.043790', '0.385198']
56		['0.065925', '0.043973', '0.141214', '0.310660', '0.269878', '0.043973', '0.386809']

57 | ['0.066201', '0.044157', '0.141805', '0.311959', '0.271007', '0.044157', '0.388427']
58 | ['0.066478', '0.044342', '0.142398', '0.313264', '0.272140', '0.044342', '0.390052']
59 | ['0.066756', '0.044527', '0.142993', '0.314574', '0.273279', '0.044527', '0.391683']
60 | ['0.067035', '0.044714', '0.143591', '0.315890', '0.274422', '0.044714', '0.393321']
61 | ['0.067315', '0.044901', '0.144192', '0.317211', '0.275570', '0.044901', '0.394966']
62 | ['0.067597', '0.045088', '0.144795', '0.318538', '0.276722', '0.045088', '0.396618']
63 | ['0.067880', '0.045277', '0.145401', '0.319870', '0.277880', '0.045277', '0.398277']
64 | ['0.068163', '0.045466', '0.146009', '0.321208', '0.279042', '0.045466', '0.399943']
65 | ['0.068449', '0.045656', '0.146620', '0.322551', '0.280209', '0.045656', '0.401616']
66 | ['0.068735', '0.045847', '0.147233', '0.323901', '0.281381', '0.045847', '0.403296']
67 | ['0.069022', '0.046039', '0.147849', '0.325255', '0.282558', '0.046039', '0.404983']
68 | ['0.069311', '0.046232', '0.148467', '0.326616', '0.283740', '0.046232', '0.406676']
69 | ['0.069601', '0.046425', '0.149088', '0.327982', '0.284926', '0.046425', '0.408377']
70 | ['0.069892', '0.046619', '0.149712', '0.329354', '0.286118', '0.046619', '0.410085']
71 | ['0.070184', '0.046814', '0.150338', '0.330731', '0.287315', '0.046814', '0.411801']
72 | ['0.070478', '0.047010', '0.150967', '0.332115', '0.288517', '0.047010', '0.413523']
73 | ['0.070773', '0.047207', '0.151598', '0.333504', '0.289723', '0.047207', '0.415253']
74 | ['0.071069', '0.047404', '0.152232', '0.334899', '0.290935', '0.047404', '0.416989']
75 | ['0.071366', '0.047602', '0.152869', '0.336299', '0.292152', '0.047602', '0.418734']
76 | ['0.071664', '0.047802', '0.153508', '0.337706', '0.293374', '0.047802', '0.420485']
77 | ['0.071964', '0.048001', '0.154150', '0.339118', '0.294601', '0.048001', '0.422244']
78 | ['0.072265', '0.048202', '0.154795', '0.340537', '0.295833', '0.048202', '0.424010']
79 | ['0.072567', '0.048404', '0.155442', '0.341961', '0.297071', '0.048404', '0.425783']
80 | ['0.072871', '0.048606', '0.156093', '0.343391', '0.298313', '0.048606', '0.427564']
81 | ['0.073176', '0.048810', '0.156745', '0.344828', '0.299561', '0.048810', '0.429352']
82 | ['0.073482', '0.049014', '0.157401', '0.346270', '0.300814', '0.049014', '0.431148']
83 | ['0.073789', '0.049219', '0.158059', '0.347718', '0.302072', '0.049219', '0.432951']
84 | ['0.074098', '0.049425', '0.158721', '0.349173', '0.303335', '0.049425', '0.434762']
85 | ['0.074408', '0.049631', '0.159384', '0.350633', '0.304604', '0.049631', '0.436581']
86 | ['0.074719', '0.049839', '0.160051', '0.352100', '0.305878', '0.049839', '0.438407']
87 | ['0.075031', '0.050047', '0.160720', '0.353572', '0.307157', '0.050047', '0.440240']
88 | ['0.075345', '0.050257', '0.161393', '0.355051', '0.308442', '0.050257', '0.442082']
89 | ['0.075660', '0.050467', '0.162068', '0.356536', '0.309732', '0.050467', '0.443931']
90 | ['0.075977', '0.050678', '0.162746', '0.358027', '0.311028', '0.050678', '0.445788']
91 | ['0.076295', '0.050890', '0.163426', '0.359525', '0.312329', '0.050890', '0.447652']
92 | ['0.076614', '0.051103', '0.164110', '0.361029', '0.313635', '0.051103', '0.449525']
93 | ['0.076934', '0.051317', '0.164796', '0.362539', '0.314947', '0.051317', '0.451405']
94 | ['0.077256', '0.051531', '0.165486', '0.364055', '0.316264', '0.051531', '0.453293']
95 | ['0.077579', '0.051747', '0.166178', '0.365578', '0.317587', '0.051747', '0.455189']
96 | ['0.077904', '0.051963', '0.166873', '0.367107', '0.318915', '0.051963', '0.457093']
97 | ['0.078229', '0.052181', '0.167571', '0.368642', '0.320249', '0.052181', '0.459004']
98 | ['0.078557', '0.052399', '0.168272', '0.370184', '0.321589', '0.052399', '0.460924']
99 | ['0.078885', '0.052618', '0.168975', '0.371732', '0.322934', '0.052618', '0.462852']
100 | ['0.079215', '0.052838', '0.169682', '0.373287', '0.324284', '0.052838', '0.464788']

⚠ هشدار: همگرایی پس از 100 تکرار حاصل نشد.

نهایی: ['0.079215', '0.052838', '0.169682', '0.373287', '0.324284', '0.052838', '0.464788'] PageRank بردار
بالاترین امتیاز را دارد، همان‌طور که در متن کتاب توضیح داده شده (ایندکس 6) d6 نکته: صفحه

🔦 نکته:

- را بدون جبر خطی پیچیده محاسبه کرد PageRank این پیادسازى نشان می‌دهد که چگونه می‌توان
- در موتورهای جستجو است PageRank پایهی الگوریتم‌های واقعی محاسبه، Power Iteration روش
- بستگی دارد، نه به نقطه شروع α توزیع ایستا صرفاً به ساختار گراف و مقدار

🗨 21.2.3 PageRank شخصی‌سازی‌شده: وقتی گوگل شما را می‌شناسد

🎯 فراتر از PageRank عمومی: دنیای PageRank موضوعی

تا اینجا، PageRank را به‌عنوان معیاری برای اهمیت عمومی یک صفحه وب شناختیم. اما یک سؤال مهم: آیا اهمیت همه صفحات برای همه کاربران یکسان است؟ یک ورزشکار حرفه‌ای به دنبال نتایج زنده فوتبال است و یک محقق سیاسی به دنبال تحلیل‌های روز. آیا باید هر دو نتایج یکسانی دریافت کنند؟

اینجاست که ایده **PageRank موضوعی** (Topic-specific PageRank) وارد می‌شود. در این مدل، ما فرض می‌کنیم کاربران به موضوعات خاصی علاقه دارند. برای مثال، یک کاربر علاقه‌مند به ورزش، ترجیح می‌دهد صفحات مرتبط با ورزش در نتایج جستجویش رتبه بالاتری داشته باشند.

چگونه این کار را انجام می‌دهیم؟ با تغییر قوانین بازی تله‌پورت! به‌جای اینکه Random Surfer ما به یک صفحه وب کاملاً تصادفی پرش کند، او را به یک صفحه وب **تصادفی اما مرتبط با موضوع مورد علاقه** پرش می‌دهیم.

برای مثال، اگر موضوع مورد نظر "ورزش" باشد، ما مجموعه‌ای از صفحات ورزشی معتبر (مثلاً از طریق دایرکتوری‌هایی مانند DMOZ یا با استفاده از دانش خود موتور جستجو) را شناسایی می‌کنیم. سپس، هر بار که سورفینگ ما تصمیم به تله‌پورت می‌گیرد، او را به یکی از این صفحات ورزشی می‌فرستیم.

از آنجایی که وب‌سایت‌های ورزشی تمایل دارند به یکدیگر لینک دهند (مثلاً ESPN به سایت رسمی یک تیم)، این سورفینگ ورزشی علاقه‌مند به احتمال زیاد در اکوسیستم ورزش باقی می‌ماند. در نتیجه، **توزیع ایستای جدید** که به دست می‌آید (آن را

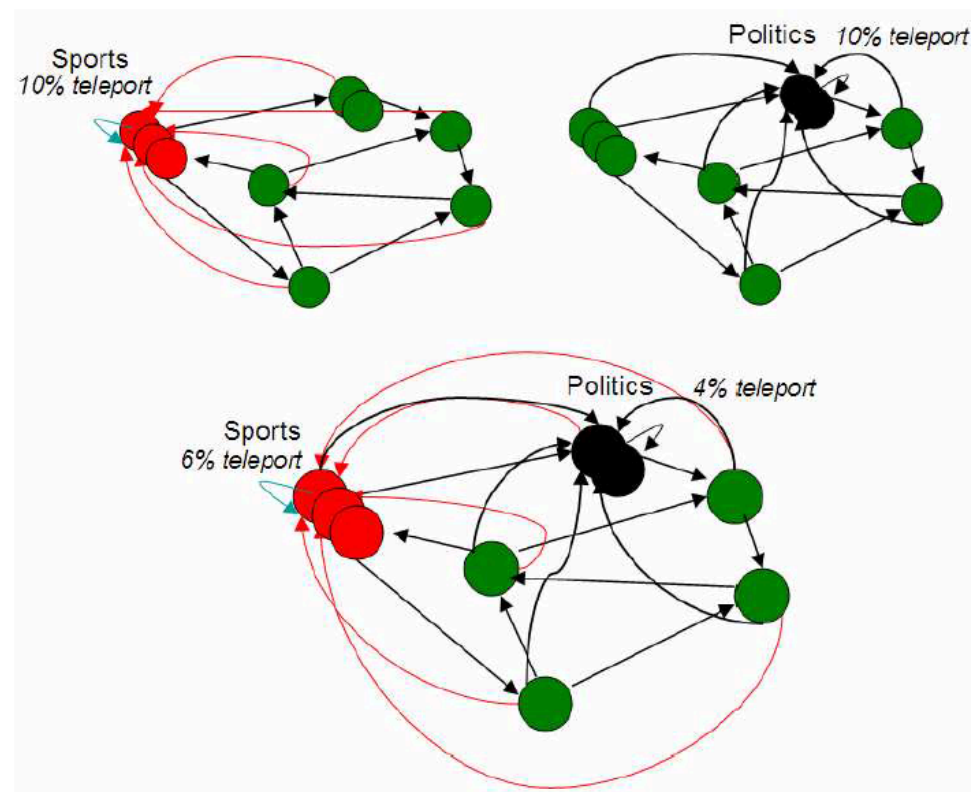
👤 PageRank شخصی‌سازی‌شده: ترکیب علایق

اما افراد معمولاً فقط به یک موضوع علاقه ندارند. یک کاربر ممکن است هم به ورزش و هم به سیاست علاقه داشته باشد. اینجاست که ایده **PageRank شخصی‌سازی‌شده** (Personalized PageRank) به کمک ما می‌آید.

فرض کنید کاربری به نام سارا وجود دارد که علایق او ترکیبی از ورزش (۶۰٪) و سیاست (۴۰٪) است. چگونه می‌توانیم PageRank را برای او شخصی‌سازی کنیم؟

روند سورفینگ سارا به این صورت خواهد بود:

- هر بار که سارا تصمیم به تله‌پورت می‌گیرد، ابتدا سیستم تصمیم می‌گیرد که او را به دنیای ورزش بفرستد یا به دنیای سیاست. این تصمیم با احتمال ۶۰٪ به نفع ورزش و با احتمال ۴۰٪ به نفع سیاست گرفته می‌شود.
 - سپس، اگر قرار بود به دنیای ورزش برود، او را به یک صفحه تصادفی از میان مجموعه صفحات ورزشی می‌فرستد. اگر هم قرار بود به دنیای سیاست برود، او را به یک صفحه تصادفی از میان مجموعه صفحات سیاسی می‌فرستد.
- این مدل منجر به یک زنجیره مارکوف جدید با توزیع ایستای منحصربه‌فردی می‌شود که دقیقاً متناسب با علایق سارا است.



► Figure 21.5 Topic-specific PageRank. In this example we consider a user whose interests are 60% sports and 40% politics. If the teleportation probability is 10%, this user is modeled as teleporting 6% to sports pages and 4% to politics pages.

شکل 21.5: نمایش PageRank موضوعی. در این مثال، کاربری با علایق ترکیبی (۶۰٪ ورزش و ۴۰٪ سیاست) در نظر گرفته شده است. اگر احتمال تله‌پورت ۱۰٪ باشد، این کاربر به این صورت مدل می‌شود که ۶٪ مواقع به صفحات ورزشی و ۴٪ مواقع به صفحات سیاسی تله‌پورت می‌کند.

⚡ معجزه خطی: محاسبه کارآمد برای میلیون‌ها کاربر

در نگاه اول، ممکن است این ایده بسیار پرهزینه به نظر برسد. آیا باید برای هر کاربر با هر ترکیبی از علایق، یک محاسبه جداگانه انجام دهیم؟ این کار غیرممکن به نظر می‌رسد!

اما اینجاست که جادوی جبر خطی به کمک ما می‌آید. نیازی نیست برای هر کاربر یک PageRank جداگانه محاسبه کنیم. به جای آن، فقط کافی است **بردارهای PageRank موضوعی پایه** را یک‌بار برای همیشه محاسبه کنیم (یعنی یک PageRank برای ورزش، یک PageRank برای سیاست، و غیره).

سپس، PageRank شخصی‌سازی‌شده برای هر کاربر به‌سادگی یک ترکیب خطی از این بردارهای پایه است. برای سارا (۶۰٪

ورزش و ۴۰٪ سیاست)، PageRank شخصی‌سازی‌شده او به این صورت محاسبه می‌شود:

$$\pi \sim \pi_{ورزش} + 0.4 \pi_{سارا} = 0.6 \pi$$

این ویژگی، استفاده از PageRank شخصی‌سازی‌شده را حتی برای میلیاردها کاربر با پروفایل‌های مختلف ممکن می‌سازد. این

روش به موتورهای جستجو اجازه می‌دهد تا نتایجی را ارائه دهند که دقیقاً متناسب با علایق فردی هر کاربر است.

In [5]: *# موضوعی (Topic-specific) PageRank پیاده‌سازی*

```
def topic_specific_pagerank(graph, topic_nodes, alpha=0.1, tolerance=1e-6, max_iterations=100):
    """
    (topic_nodes) موضوعی برای یک زیرمجموعه از صفحات PageRank محاسبه

    ورودی‌ها:
    - graph: دیکشنری گراف وب (صفحه -> لیست لینک‌های خروجی)
    - topic_nodes: مجموعه‌ای از صفحات مرتبط با یک موضوع خاص
    - alpha: احتمال تله‌پورت (معمولاً 0.1)
    - tolerance, max_iterations: پارامترهای همگرایی

    خروجی:
    غیرصفر است (topic_nodes صفحات قابل‌دسترسی از) Y که فقط برای صفحات در مجموعه PageRank دیکشنری -
    """

    # رسید topic_nodes صفحاتی که می‌توان به آن‌ها از Y پیدا کردن تمام گره‌های مرتبط با موضوع
    all_nodes = set(graph.keys())
    for neighbors in graph.values():
        all_nodes.update(neighbors)

    # نگاشت گره‌ها به اندیس
    nodes = list(all_nodes)
    node_to_index = {node: i for i, node in enumerate(nodes)}
    N = len(nodes)

    # PageRank مقداردهی اولیه
    pagerank = [1.0 / N] * N

    # توزیع تله‌پورت: فقط به گره‌های موضوعی
    teleport_dist = [0.0] * N
    topic_count = 0
    for i, node in enumerate(nodes):
        if node in topic_nodes:
            teleport_dist[i] = 1.0
            topic_count += 1

    # اگر هیچ گره موضوعی وجود نداشت، تله‌پورت یکنواخت باشد
    if topic_count == 0:
        teleport_dist = [1.0 / N] * N
    else:
        teleport_dist = [p / topic_count for p in teleport_dist] # توزیع یکنواخت روی موضوع

    # روش تکراری
    for iteration in range(max_iterations):
        new_pagerank = [0.0] * N

        # توزیع موضوعی * (1 - alpha): بخش تله‌پورت
        for i in range(N):
            new_pagerank[i] += (1.0 - alpha) * teleport_dist[i]

        # بخش انتشار از طریق لینک‌ها: alpha * Σ(PR(Pi)/L(Pi))
        for node in nodes:
            i = node_to_index[node]
            if pagerank[i] == 0:
                continue
            out_links = graph.get(node, [])
            if not out_links:
                continue
            share = (alpha * pagerank[i]) / len(out_links)
            for neighbor in out_links:
                if neighbor in node_to_index:
                    j = node_to_index[neighbor]
                    new_pagerank[j] += share

        # بررسی همگرایی
        diff = sum(abs(new_pagerank[i] - pagerank[i]) for i in range(N))
        if diff < tolerance:
            break
        pagerank = new_pagerank[:]

    # ساخت دیکشنری نتیجه
    result = {}
    for i, node in enumerate(nodes):
        result[node] = pagerank[i]
    return result

def personalized_pagerank(sports_pagerank, politics_pagerank, sports_weight=0.6):
    """
    موضوعی PageRank شخصی‌سازی‌شده با ترکیب خطی دو PageRank ساخت

    all_nodes = set(sports_pagerank.keys()) | set(politics_pagerank.keys())
```

```
personalized = {}
for node in all_nodes:
    sp = sports_pagerank.get(node, 0.0)
    pp = politics_pagerank.get(node, 0.0)
    personalized[node] = sports_weight * sp + (1 - sports_weight) * pp
return personalized

# مثال کاربردی: گراف نمونه با دو حوزه موضوعی
if __name__ == "__main__":
    # (سیاسی) p0, p1, p2 و (ورزشی) s0, s1, s2 گراف نمونه: گره‌های
    sample_graph = {
        # (بُنگ‌های درون‌موضوعی زیاده) صفحات ورزشی
        "s0": ["s1", "s2", "p0"],
        "s1": ["s0", "s2"],
        "s2": ["s0", "s1"],
        # (بُنگ‌های درون‌موضوعی زیاده) صفحات سیاسی
        "p0": ["p1", "p2", "s0"],
        "p1": ["p0", "p2"],
        "p2": ["p0", "p1"],
        # صفحه عمومی
        "general": ["s0", "p0"]
    }

    # تعریف مجموعه‌های موضوعی
    sports_nodes = {"s0", "s1", "s2"}
    politics_nodes = {"p0", "p1", "p2"}

    print("موضوعی و شخصی‌سازی‌شده PageRank مثال آموزشی")
    print("=" * 55)

    # موضوعی برای ورزش PageRank محاسبه
    sports_pr = topic_specific_pagerank(
        sample_graph,
        sports_nodes,
        alpha=0.1
    )

    # موضوعی برای سیاست PageRank محاسبه
    politics_pr = topic_specific_pagerank(
        sample_graph,
        politics_nodes,
        alpha=0.1
    )

    # شخصی‌سازی‌شده (60٪ ورزش، 40٪ سیاست) PageRank ساخت
    personalized_pr = personalized_pagerank(sports_pr, politics_pr, sports_weight=0.6)

    # نمایش نتایج
    print("\n🏆 PageRank (ورزش) - ۱۰ گره برتر")
    sorted_sports = sorted(sports_pr.items(), key=lambda x: x[1], reverse=True)[:5]
    for node, score in sorted_sports:
        if score > 0:
            print(f"    {node}: {score:.6f}")

    print("\n🏆 PageRank (سیاست) - ۱۰ گره برتر")
    sorted_politics = sorted(politics_pr.items(), key=lambda x: x[1], reverse=True)[:5]
    for node, score in sorted_politics:
        if score > 0:
            print(f"    {node}: {score:.6f}")

    print("\n🎯 PageRank (60٪ ورزش + 40٪ سیاست) - ۱۰ گره برتر")
    sorted_personal = sorted(personalized_pr.items(), key=lambda x: x[1], reverse=True)[:5]
    for node, score in sorted_personal:
        print(f"    {node}: {score:.6f}")

    print("\n💡 نکته:")
    print("- موضوعی، امتیاز بیشتری به صفحات درون یک حوزه اختصاص می‌دهد PageRank")
    print("- !شخصی‌سازی‌شده با ترکیب خطی ساخته می‌شود - نیازی به محاسبه مجدد نیست PageRank")
    print("- این روش، امکان سفارشی‌سازی رتبه‌بندی برای میلیون‌ها کاربر را فراهم می‌کند")
```

موضوعی و شخصی‌سازی‌شده PageRank مثال آموزشی
=====

🏆 موضوعی (ورزش) - ۱۰ گره برتر PageRank:

s0:	0.333119
s1:	0.327478
s2:	0.327478
p0:	0.011143
p2:	0.000391

🏆 موضوعی (سیاست) - ۱۰ گره برتر PageRank:

p0:	0.333119
p2:	0.327478
p1:	0.327478
s0:	0.011143
s1:	0.000391

🎯 شخصی‌سازی‌شده (60٪ ورزش + 40٪ سیاست) - ۱۰ گره برتر PageRank:

s0:	0.204329
s1:	0.196643
s2:	0.196643
p0:	0.139934
p1:	0.131226

💡 نکته:

- موضوعی، امتیاز بیشتری به صفحات درون یک حوزه اختصاص می‌دهد PageRank
- !شخصی‌سازی‌شده با ترکیب خطی ساخته می‌شود - نیازی به محاسبه مجدد نیست PageRank
- این روش، امکان سفارشی‌سازی رتبه‌بندی برای میلیون‌ها کاربر را فراهم می‌کند

21.3 هاب‌ها و مقام‌ها: کشف گنجینه‌های اطلاعاتی در وب 📖

تصور کنید به تازگی به بیماری "میگرن" مبتلا شده‌اید و می‌خواهید اطلاعات معتبر و قابل اعتماد درباره آن پیدا کنید. اگر عبارت "میگرن" را در یک موتور جستجو وارد کنید، با هزاران نتیجه مواجه می‌شوید. کدام صفحات معتبر هستند و کدام صرفاً ادعا می‌کنند که متخصص هستند؟

در اینجا، دو نوع صفحه وب وجود دارد که برای شما مفید خواهند بود:

1. **صفحات مقام (Authority):** این صفحات منابع اصلی و معتبر اطلاعات هستند. در مورد میگرن، این صفحات

شامل وبسایت‌های انجمن‌های پزشکی معتبر، مقالات علمی در مجلات پزشکی، یا صفحات وبسایت‌های سازمان‌های بهداشتی معتبر است.

2. **صفحات هاب (Hub):** این صفحات خودشان حاوی اطلاعات اصلی نیستند، بلکه مجموعه‌ای از لینک‌ها به منابع

معتبر را گردآوری کرده‌اند. مثلاً یک وبلاگ که توسط یک بیمار میگرنی اداره می‌شود و بهترین منابع اطلاعاتی درباره میگرن را معرفی کرده است.

الگوریتم **HITS** (Hyperlink-Induced Topic Search) دقیقاً برای همین منظور طراحی شده است: برای هر پرس‌وجو، دو امتیاز جداگانه به هر صفحه اختصاص می‌دهد:

- **امتیاز هاب (Hub Score):** نشان‌دهنده توانایی صفحه در اشاره به صفحات معتبر است.
- **امتیاز مقام (Authority Score):** نشان‌دهنده اعتبار صفحه به‌عنوان منبع اطلاعات است.

🔗 رابطه متقابل هاب‌ها و مقام‌ها

ایده کلیدی پشت الگوریتم HITS این است که هاب‌ها و مقام‌ها به‌صورت متقابل یکدیگر را تقویت می‌کنند:

- یک **هاب خوب** صفحه‌ای است که به مقام‌های خوب لینک می‌دهد.
- یک **مقام خوب** صفحه‌ای است که توسط هاب‌های خوب به آن لینک داده شده است.

این رابطه متقابل را می‌توان با معادلات زیر نشان داد:

$$h(v) \leftarrow \sum_{y \rightarrow v} a(y)$$

$$a(v) \leftarrow \sum_{y \rightarrow v} h(y)$$

که در آن: $h(v)$ امتیاز هاب صفحه v است، $a(v)$ امتیاز مقام صفحه v است. - مجموع اول برابر است با مجموع امتیازات مقام صفحاتی که v به آن‌ها لینک داده است. - مجموع دوم برابر است با مجموع امتیازات هاب صفحاتی که به v لینک داده‌اند.

این معادلات به‌صورت ماتریسی بازنویسی می‌شوند:

$$\sim h \leftarrow A \sim a$$

$$\sim a \leftarrow A^T \sim h$$

که در آن A ماتریس مجاورت گراف وب است. با جایگزینی مکرر، به معادلات ویژه‌مقداری می‌رسیم:

$$\tilde{h} = (1/\lambda_h) A A^T \tilde{h}$$

$$\tilde{a} = (1/\lambda_a) A^T A \tilde{a}$$

بنابراین، بردارهای نهایی هاب و مقام، **بردارهای ویژه اصلی** ماتریس‌های $A A^T$ و $A^T A$ هستند. در عمل، این بردارها با روش‌های تکراری (مثل Power Iteration) محاسبه می‌شوند.

🎯 مثال کاربردی: جستجوی اطلاعات در مورد هوش مصنوعی

فرض کنید می‌خواهید اطلاعات معتبر در مورد "یادگیری عمیق" پیدا کنید. در این حالت:

- **مقام‌ها** ممکن است شامل صفحات وب‌سایت‌های دانشگاهی معتبر (مثل استنفورد، ام‌آی‌تی)، مقالات بنیادی در این زمینه، یا مستندات رسمی فریمورک‌های معتبر یادگیری عمیق (مثل TensorFlow یا PyTorch) باشند.
- **هاب‌ها** ممکن است شامل وبلاگ‌های تخصصی که بهترین منابع یادگیری عمیق را معرفی کرده‌اند، یا صفحه‌هایی که مجموعه‌ای از مقالات مهم و آموزش‌های باکیفیت را گردآوری کرده‌اند، باشند.

الگوریتم HITS با تحلیل لینک‌ها بین این صفحات، می‌تواند به شما کمک کند تا هم منابع اصلی و معتبر (مقام‌ها) و هم راهنماهای مفید برای پیدا کردن این منابع (هاب‌ها) را شناسایی کنید.

📊 مراحل اجرای الگوریتم HITS

الگوریتم HITS به‌صورت زیر عمل می‌کند:

1. برای پرس‌وجوی داده‌شده، یک زیرمجموعه از صفحات وب مرتبط را جمع‌آوری می‌کنیم (این فرآیند در بخش بعدی توضیح داده می‌شود).
2. برای هر صفحه در این زیرمجموعه، امتیاز هاب و مقام اولیه را 1 در نظر می‌گیریم.
3. به‌صورت تکراری، امتیازات هاب و مقام را با استفاده از معادلات به‌روزرسانی می‌کنیم تا زمانی که مقادیر همگرا شوند.
4. در نهایت، صفحات را بر اساس امتیازات هاب و مقام رتبه‌بندی کرده و دو لیست جداگانه ارائه می‌دهیم.

نکته کلیدی این است که HITS فقط ساختار لینک‌ها را تحلیل می‌کند و به محتوای متن صفحات توجهی ندارد (مگر در مرحله جمع‌آوری زیرمجموعه اولیه).

🐾 مثال عددی: تحلیل یک شبکه کوچک

در مثال 21.2 برای پرس‌وجوی "jaguar" (که هم می‌تواند به حیوان و هم به خودرو اشاره کند)، الگوریتم HITS صفحات زیر را به‌عنوان مقام‌ها و هاب‌های اصلی شناسایی می‌کند:

- صفحه q3 به‌عنوان **مقام اصلی** شناخته می‌شود، زیرا دو هاب (q2 و q6) از طریق لینک‌هایی با متن "jaguar" به آن اشاره کرده‌اند.
- صفحات q2 و q6 به‌عنوان **هاب‌های اصلی** شناخته می‌شوند، زیرا به مقام‌های معتبر لینک داده‌اند.

این مثال نشان می‌دهد که چگونه HITS می‌تواند با تحلیل ساختار لینک‌ها، صفحات معتبر را از صفحات کم‌اهمیت‌تر تمیز دهد، حتی وقتی کلمات کلیدی به تنهایی کافی نیستند.

In [6]: *# HITS (Hubs and Authorities) پیاده‌سازی الگوریتم*

```
def hits_algorithm(graph, max_iterations=100, tolerance=1e-6):
    """
    محاسبه امتیازهای هاب و مقام برای گراف داده‌شده.

    ورودی‌ها:
    - graph: دیکشنری نمایش‌دهنده گراف جهت‌دار
      مثال: {"A": ["B", "C"], "B": ["C"], "C": ["A"]}
    - max_iterations: حداکثر تعداد تکرار
    - tolerance: آستانه همگرایی

    خروجی:
    - دیکشنری شامل دو کلید -
      'hubs': دیکشنری امتیاز هاب‌ها
      'authorities': دیکشنری امتیاز مقام‌ها
    """
    # جمع‌آوری تمام گره‌ها
    all_nodes = set(graph.keys())
    for neighbors in graph.values():
        all_nodes.update(neighbors)
    nodes = sorted(all_nodes) # قابل‌پیش‌بینی

    # مقداردهی اولیه: همه امتیازها برابر 1
    hubs = {node: 1.0 for node in nodes}
    authorities = {node: 1.0 for node in nodes}

    for iteration in range(max_iterations):
        # مرحله 1: به‌روزرسانی امتیاز مقام‌ها
        new_authorities = {}
        for node in nodes:
            auth_score = 0.0
            # جمع امتیاز هاب‌هایی که به این گره لینک داده‌اند
            for source, targets in graph.items():
                if node in targets:
                    auth_score += hubs.get(source, 0.0)
            new_authorities[node] = auth_score

        # مرحله 2: به‌روزرسانی امتیاز هاب‌ها
        new_hubs = {}
        for node in nodes:
            hub_score = 0.0
            # جمع امتیاز مقام‌هایی که این گره به آنها لینک داده
            for target in graph.get(node, []):
                hub_score += new_authorities.get(target, 0.0)
            new_hubs[node] = hub_score

        # نرمال‌سازی برای جلوگیری از انفجار عددی
        # HITS رایج‌ترین روش در L2 نرمال‌سازی
        def normalize(score_dict):
            total_sq = sum(v**2 for v in score_dict.values())
            if total_sq == 0:
                return {k: 0.0 for k in score_dict}
            norm = total_sq ** 0.5
            return {k: v / norm for k, v in score_dict.items()}

        new_authorities = normalize(new_authorities)
        new_hubs = normalize(new_hubs)

        # بررسی همگرایی
        auth_diff = sum(abs(new_authorities[n] - authorities[n]) for n in nodes)
        hub_diff = sum(abs(new_hubs[n] - hubs[n]) for n in nodes)

        if auth_diff < tolerance and hub_diff < tolerance:
            print(f"✅ تکرار {iteration + 1} همگرایی پس از")
            break

        authorities = new_authorities
        hubs = new_hubs

    return {"hubs": hubs, "authorities": authorities}

# مثال کاربردی: گراف شکل 21.4 (ساده‌شده برای آموزش)
if __name__ == "__main__":
    # d0 تا d6 گراف نمونه مبتنی بر شکل 21.4 با گره‌های
    sample_graph = {
        "d0": ["d2", "d1"],
        "d1": ["d2"],
        "d2": ["d0", "d3"],
        "d3": ["d4", "d6"],
        "d4": ["d6"],
        "d5": ["d6"],
        "d6": ["d3"]
    }

    print("برای گراف شکل 21.4 HITS مثال آموزشی الگوریتم")
    print("=" * 50)

    # اجرای الگوریتم HITS
    results = hits_algorithm(sample_graph)
    hubs = results["hubs"]
    authorities = results["authorities"]
```

```
# نمایش ۳ گره برتر در هر دسته
print("\n 🏆 ۳ گره برتر از نظر امتیاز مقام ۳ (Authorities):")
top_authorities = sorted(authorities.items(), key=lambda x: x[1], reverse=True)[:3]
for rank, (node, score) in enumerate(top_authorities, 1):
    print(f" {rank}. {node}: {score:.6f}")

print("\n 🌐 ۳ گره برتر از نظر امتیاز هاب ۳ (Hubs):")
top_hubs = sorted(hubs.items(), key=lambda x: x[1], reverse=True)[:3]
for rank, (node, score) in enumerate(top_hubs, 1):
    print(f" {rank}. {node}: {score:.6f}")

print("\n 💡 نکته:")
print("- دو نوع صفحه را شناسایی می‌کند HITS الگوریتم -")
print("مقام‌ها: صفحاتی که توسط هاب‌های متعددی لینک شده‌اند •")
print("هاب‌ها: صفحاتی که به مقام‌های متعددی لینک داده‌اند •")
print("- وابسته به پرس‌وجو است و روی یک زیرگراف موضوعی اجرا می‌شود HITS، PageRank، برخلاف -")
print("- جلوی رشد نامحدود امتیازها را می‌گیرد و همگرایی را تضمین می‌کند L2 نرمال‌سازی -")
```

برای گراف شکل HITS 21.4 مثال آموزشی الگوریتم

=====

✓ همگرایی پس از 52 تکرار

🏆 ۳ گره برتر از نظر امتیاز مقام ۳ (Authorities):

1. d6: 0.923880
2. d4: 0.382683
3. d2: 0.000001

🌐 ۳ گره برتر از نظر امتیاز هاب ۳ (Hubs):

1. d3: 0.707107
2. d4: 0.500000
3. d5: 0.500000

💡 نکته:

- دو نوع صفحه را شناسایی می‌کند HITS الگوریتم -
- مقام‌ها: صفحاتی که توسط هاب‌های متعددی لینک شده‌اند
- هاب‌ها: صفحاتی که به مقام‌های متعددی لینک داده‌اند
- وابسته به پرس‌وجو است و روی یک زیرگراف موضوعی اجرا می‌شود HITS، PageRank، برخلاف
- جلوی رشد نامحدود امتیازها را می‌گیرد و همگرایی را تضمین می‌کند L2 نرمال‌سازی -

21.3.1 انتخاب زیرمجموعه وب: هنر پیدا کردن گنجینه‌های پنهان 📖

🔍 چالش اصلی: صفحات معتبر بدون کلمات کلیدی

یک چالش بزرگ در الگوریتم HITS این است که بسیاری از صفحات معتبر ممکن است کلمه کلیدی پرس‌وجو را در محتوای خود نداشته باشند. این پدیده به خصوص برای شرکت‌های بزرگ و برندهای معتبر رایج است.

برای مثال، وب‌سایت شرکت اپل منبع معتبری برای اطلاعات درباره آیفون است، اما ممکن است صفحه اصلی آن کلمه "آیفون" را نداشته باشد و فقط از برند "Apple" استفاده کند. یا وب‌سایت سازمان بهداشت جهانی (WHO) منبع معتبری برای اطلاعات درباره کرونا است، اما ممکن است در صفحه اصلی خود از کلمه "کرونا" استفاده نکند و عبارت "COVID-19" را به کار ببرد.

این چالش باعث می‌شود که اگر فقط صفحاتی را که شامل کلمه کلیدی پرس‌وجو هستند در نظر بگیریم، ممکن است منابع معتبر و اصلی را از دست بدهیم.

🚀 راه‌حل: از مجموعه ریشه تا مجموعه پایه

برای حل این مشکل، از یک فرآیند دو مرحله‌ای برای ساخت مجموعه پایه استفاده می‌کنیم:

1. **مجموعه ریشه (Root Set):** تمام صفحاتی که کلمه کلیدی پرس‌وجو را در متن خود دارند. برای مثال، اگر پرس‌وجوی ما "هوش مصنوعی در پزشکی" باشد، مجموعه ریشه شامل تمام صفحاتی است که این عبارت را در متن خود دارند.
2. **مجموعه پایه (Base Set):** شامل مجموعه ریشه به‌علاوه تمام صفحاتی که یا به صفحات ریشه لینک داده‌اند یا توسط صفحات ریشه لینک شده‌اند.

برای درک بهتر، این مثال را در نظر بگیرید: فرض کنید در حال جستجوی اطلاعات درباره "سرطان سینه" هستیم. مجموعه ریشه شامل صفحاتی است که مستقیماً از این عبارت استفاده می‌کنند. اما وب‌سایت انجمن سرطان آمریکا، که یک منبع بسیار معتبر است، ممکن است در صفحه اصلی خود از عبارت "سرطان سینه" استفاده نکند و فقط از عبارت "Breast Cancer" استفاده کند.

با این حال، اگر یک وبلاگ پزشکی (که در مجموعه ریشه قرار دارد) به وبسایت انجمن سرطان آمریکا لینک داده باشد، این وبسایت معتبر وارد مجموعه پایه می‌شود و در نتیجه در تحلیل HITS شرکت داده خواهد شد.

💡 مزایای کلیدی این رویکرد

این روش گسترش مجموعه ریشه به مجموعه پایه سه مزیت کلیدی دارد:

- **شناسایی مقام‌های پنهان:** صفحات معتبری که کلمه کلیدی را ندارند، از طریق لینک‌های ورودی از هاب‌های مرتبط شناسایی می‌شوند.
- **گسترش شبکه مقام‌ها:** اگر یک هاب معتبر در مجموعه ریشه باشد، تمام مقام‌هایی که به آن‌ها لینک داده، در مجموعه پایه گنجانده می‌شوند.
- **شناسایی هاب‌های مرتبط:** اگر یک مقام معتبر در مجموعه ریشه باشد، تمام هاب‌هایی که به آن لینک داده‌اند، وارد مجموعه پایه می‌شوند.

به طور خلاصه، این گسترش به ما کمک می‌کند تا یک شبکه غنی از هاب‌ها و مقام‌های مرتبط با موضوع مورد نظر ایجاد کنیم، حتی اگر برخی از این صفحات مستقیماً کلمه کلیدی پرس‌وجو را در متن خود نداشته باشند.

🌐 پدیده شگفت‌انگیز: بازیابی بین‌زبانی

یکی از جالب‌ترین نتایج الگوریتم HITS این است که می‌تواند صفحات را به زبان‌های مختلف بازیابی کند، حتی اگر پرس‌وجو به یک زبان خاص باشد. این پدیده که **بازیابی بین‌زبانی** (Cross-Language Retrieval) نامیده می‌شود، کاملاً از طریق تحلیل لینک (نه ترجمه زبانی) حاصل می‌شود.

برای مثال، اگر شما عبارت "japan elementary schools" را به انگلیسی جستجو کنید، ممکن است صفحات ژاپنی (به زبان ژاپنی) در نتایج ظاهر شوند. چطور این اتفاق می‌افتد؟

تصور کنید یک وبلاگ انگلیسی‌زبان درباره سیستم آموزشی ژاپن وجود دارد که به وبسایت‌های مدارس ژاپنی (به زبان ژاپنی) لینک داده است. این وبلاگ انگلیسی در مجموعه ریشه قرار می‌گیرد (چون حاوی کلمات کلیدی انگلیسی است)، اما وبسایت‌های ژاپنی که به آن‌ها لینک داده شده، در مجموعه پایه قرار می‌گیرند. سپس الگوریتم HITS این وبسایت‌های ژاپنی را به‌عنوان مقام‌های معتبر شناسایی می‌کند، حتی اگر به زبان انگلیسی نباشند.

Hubs

- schools
- LINK Page-13
- "ú-{|Šw□Z
- □a%□□Šw□Zfz□[f□fy□[fW
- 100 Schools Home Pages (English)
- K-12 from Japan 10/...net and Education)
- http://www...iglobe.ne.jp/~IKESAN
- ,l,f,j□□Šw□Z,U"N,P'g•Œê
- □ÒŠ—'—§□ÒŠ—Œ□□Šw□Z
- Koulutus ja oppilaitokset
- TOYODA HOMEPAGE
- Education
- Cay's Homepage(Japanese)
- -y'□□Šw□Z,l,fz□[f□fy□[fW
- UNIVERSITY
- %□—□□Šw□Z DRAGON97-TOP
- □Â%□□Šw□Z,T"N,P'g,fz□[f□fy□[fW
- ¶µ°é¼ÄÄ© ¥á¥E¥á¼ ¥á¥E¥á¼

Authorities

- The American School in Japan
- The Link Page
- %□□è□□s—§"ā"□□Šw□Zfz□[f□fy□[fW
- Kids' Space
- "À□é□□s—§"À□é□¼•□□Šw□Z
- «{□é□"ç'āŠw□□'©□□Šw□Z
- KEIMEI GAKUEN Home Page (Japanese)
- Shiranuma Home Page
- fuzoku-es.fukui-u.ac.jp
- welcome to Miasa E&J school
- □_□□□Œ§□E%□□□□s—§'†□□□¼□□Šw□Z,l,fy
- http://www...p/~m_maru/index.html
- fukui haruyama-es HomePage
- Torisu primary school
- goo
- Yakumo Elementary,Hokkaido,Japan
- FUZOKU Home Page
- Kamishibun Elementary School...

► Figure 21.6 A sample run of HITS on the query japan elementary schools.

شکل 21.6: نتایج اجرای HITS برای پرس‌وجوی "japan elementary schools". این شکل بالاترین هاب‌ها و مقام‌ها را نشان می‌دهد. هر ردیف عنوان صفحه HTML مربوطه را نمایش می‌دهد. توجه کنید که بسیاری از نتایج به زبان ژاپنی هستند، با وجود اینکه پرس‌وجو به انگلیسی بوده است.

🔧 نکات اجرایی و بهینه‌سازی

در پیاده‌سازی عملی الگوریتم HITS، چند نکته مهم برای بهینه‌سازی عملکرد وجود دارد:

- **اندازه مجموعه ریشه:** نیازی به شامل کردن همه صفحات مرتبط با پرس‌وجو نیست. معمولاً حدود 200 صفحه برای مجموعه ریشه کافی است تا نتایج باکیفیتی تولید شود.
 - **تعداد تکرارها:** برای محاسبه امتیازهای هاب و مقام، نیازی به دقت بالا نیست. معمولاً 5 تا 10 تکرار از معادلات HITS برای یافتن رتبه‌بندی نسبی کافی است.
 - **بهینه‌سازی محاسبات:** به دلیل پراکنده‌بودن گراف وب (هر صفحه به طور متوسط به حدود 10 صفحه دیگر لینک می‌دهد)، به‌روزرسانی‌ها به‌صورت جمع‌های افزایشی (نه ضرب ماتریسی) انجام می‌شود تا محاسبات کارآمدتر شوند.
- این بهینه‌سازی‌ها باعث می‌شوند که الگوریتم HITS بتواند به سرعت و با کارایی بالا نتایج باکیفیتی تولید کند، حتی برای مجموعه‌های بزرگ از صفحات وب.

🔗 جمع‌بندی فصل ۲۱: تحلیل پیوندها

چگونه می‌توان اهمیت و اعتبار یک صفحه وب را بر اساس ساختار پیوندهای آن در وب سنجید؟ فصل ۲۱ به ما نشان می‌دهد که چگونه تحلیل گراف وب (شبکه‌ای از صفحات و پیوندها) منجر به توسعه الگوریتم‌های رتبه‌بندی انقلابی مانند PageRank و HITS شده است. این الگوریتم‌ها با تفسیر هر پیوند به عنوان یک «رأی اعتماد»، توانستند کیفیت صفحات را به شیوه‌ای فراتر از تطبیق متن ساده ارزیابی کنند و به موتورهای جستجو کمک کنند تا نتایج مرتبط‌تر و معتبرتری را به کاربران ارائه دهند.

- با ****مفهوم متن لنگر (Anchor Text)**** آشنا شدیم. این متن، همان متن قابل کلیک است که در یک پیوند hyperlink قرار دارد و اغلب توصیف دقیقی از صفحه مقصد ارائه می‌دهد. این ویژگی به ویژه برای ایندکس‌گذاری صفحاتی که

خودشان به درستی توصیف نشده‌اند (مانند صفحه اصلی IBM که ممکن است کلمه "computer" را نداشته باشد) بسیار مفید است.

- الگوریتم ****PageRank**** را بررسی کردیم. این الگوریتم، که توسط گوگل توسعه یافته، هر صفحه را بر اساس تعداد و کیفیت پیوندهایی که به آن داده شده است، امتیازی بین ۰ و ۱ اختصاص می‌دهد. این مدل بر اساس مفهوم «کاربر تصادفی» (Random Surfer) عمل می‌کند که در وب به صورت تصادفی پیوندها را دنبال می‌کند یا به صورت تصادفی به صفحه‌ای دیگر «تلیپورت» می‌کند. PageRank، احتمال حضور کاربر در یک صفحه در حالت پایدار را نشان می‌دهد و یک معیار کیفیت ایستا و مستقل از پرس‌وجو است.
- با ****PageRank**** موضوع-محور (Topic-Specific PageRank) آشنا شدیم. در این رویکرد، به جای تلیپورت کردن کاربر به یک صفحه کاملاً تصادفی، او به صفحات مرتبط با یک موضوع خاص (مانند ورزش یا سیاست) تلیپورت می‌کند. این کار باعث می‌شود صفحات مرتبط با آن موضوع، امتیاز PageRank بالاتری کسب کنند. این ایده اساس ****PageRank**** شخصی‌سازی‌شده (Personalized PageRank) را نیز فراهم می‌کند که می‌تواند بر اساس علایق ترکیبی یک کاربر، امتیازهای PageRank مختلف را با هم ترکیب کند.
- الگوریتم ****HITS (Hyperlink-Induced Topic Search)**** را مطالعه کردیم. این الگوریتم برای هر صفحه دو امتیاز محاسبه می‌کند: ****امتیاز هاب (Hub Score)**** و ****امتیاز اتوریته (Authority Score)****. صفحات ****اتوریته**** منابع معتبر و اصلی در یک موضوع هستند (مانند صفحه یک مؤسسه تحقیقاتی معتبر). صفحات ****هاب**** صفحاتی هستند که به منابع معتبر زیادی لینک می‌دهند (مانند یک فهرست دست‌نویس شده از منابع مفید). این الگوریتم به صورت تکراری این دو امتیاز را محاسبه می‌کند تا بهترین هاب‌ها و اتوریته‌ها را برای یک پرس‌وجوی مشخص پیدا کند.

بیا یاد این مفاهیم را با چند مثال واقعی و کاربردی زنده کنیم:

🔗 **قدرت متن لنگر:** ممکن است صفحه اصلی شرکت IBM (ibm.com) کلمه "computer" را در متن خود نداشته باشد. اما هزاران وب‌سایت دیگر در هنگام لینک دادن به IBM، از کلمه "computer" در متن لنگر خود استفاده می‌کنند. موتورهای جستجو از این سیگنال استفاده می‌کنند و به صفحه IBM امتیاز بالایی برای جستجوی "computer" می‌دهند. به همین دلیل، جستجوی "big blue" (لقب معروف IBM) نیز معمولاً صفحه اصلی این شرکت را به عنوان نتیجه برتر نشان می‌دهد.

🏆 **PageRank به مثابه رأی اعتماد:** تصور کنید هر پیوند به یک صفحه، یک رأی اعتماد به آن صفحه است. اما همه رأی‌ها برابر نیستند. یک پیوند از یک وب‌سایت بسیار معتبر و محبوب (مانند یک خبرگزاری بزرگ) بسیار ارزشمندتر از یک پیوند از یک وبلاگ ناشناس است. PageRank دقیقاً همین منطق را پیاده‌سازی می‌کند؛ صفحاتی که از منابع معتبرتری لینک دریافت کرده‌اند، رتبه بالاتری خواهند داشت، زیرا کاربر تصادفی احتمال بیشتری دارد که از طریق آن پیوندهای معتبر به این صفحات برسد.

🎯 **HITS: پیدا کردن مراجع و راهنماها:** اگر شما عبارت "لوکیمی" را جستجو کنید، الگوریتم HITS تلاش می‌کند دو نوع صفحه را پیدا کند: (۱) صفحات ****اتوریته**** مانند وب‌سایت موسسه ملی سرطان که منبع اصلی و معتبر اطلاعات در این زمینه است. (۲) صفحات ****هاب**** مانند وبلاگ یک پزشک یا صفحه‌ای که یک بیمار برای جمع‌آوری منابع مفید درباره لوکیمی ساخته است و به منابع معتبر متعددی (از جمله موسسه ملی سرطان) لینک داده است. HITS با تحلیل این پیوندها، بهترین منابع (اتوریته‌ها) و بهترین راهنماها (هاب‌ها) را شناسایی می‌کند.

🦋 **PageRank شخصی‌سازی‌شده:** یک کاربر ممکن است ۶۰٪ به ورزش و ۴۰٪ به سیاست علاقه داشته باشد. موتور جستجو می‌تواند با ترکیب PageRank موضوع-محور ورزش (که در آن تلیپورت به صفحات ورزشی است) و PageRank موضوع-محور سیاست، یک رتبه‌بندی شخصی‌سازی‌شده برای این کاربر ایجاد کند. در این رتبه‌بندی، صفحات مرتبط با ورزش و سیاست بر اساس ترکیب علایق او، امتیاز بالاتری کسب می‌کنند و نتایج جستجوی شخصی‌تری به او نمایش داده می‌شود.

این فصل نشان داد که تحلیل پیوندها با تبدیل وب به یک گراف بزرگ و استفاده از هوش جمعی سازندگان صفحات وب (از طریق پیوندهایشان)، انقلابی در فناوری جستجو ایجاد کرد. الگوریتم‌هایی مانند PageRank و HITS با فراتر رفتن از تطبیق ساده متن، توانستند اعتبار و کیفیت صفحات را بسنجند و این یکی از دلایل اصلی موفقیت موتورهای جستجوی مدرن در ارائه نتایج باکیفیت و مرتبط است.

End of Chapter Exercises

مسئله 21.1: مفهوم گراف وب (The Web as a Graph Concept)

با در نظر گرفتن وب به عنوان یک گراف جهت‌دار، گره‌ها و یال‌ها نماینده چه مفاهیمی هستند؟ چرا فرض می‌کنیم این گراف به شدت پراکنده (Sparse) است؟

مسئله 21.2: اهمیت متن لینک (Importance of Anchor Text)

چرا در موتورهای جستجو، کلماتی که در Anchor text قرار دارند گاهی ارزش بیشتری از محتوای درون خود صفحه مقصد (Target Page) دارند؟

مسئله 21.3: متغیرهای پنهان در گراف (Hidden Variables in Graph)

توضیح دهید چگونه بررسی لینک‌های ورودی (In-links) به یک صفحه می‌تواند اطلاعاتی درباره کیفیت آن صفحه بدهد، حتی اگر محتوای متنی صفحه بررسی نشود.

مسئله 21.4: زنجیره مارکوف در وب (Markov Chains in the Web)

مفهوم «موج‌سوار تصادفی» (Random Surfer) در زنجیره مارکوف چیست؟ چه رابطه‌ای بین این مفهوم و احتمال حضور در یک صفحه وب وجود دارد؟

مسئله 21.5: ماتریس گذار (Transition Matrix)

فرض کنید یک گراف وب دارای N صفحه است و از صفحه A به 3 صفحه دیگر لینک خروجی وجود دارد. مقادیر سطر مربوط به صفحه A در ماتریس گذار (احتمالات) چگونه پر می‌شود؟

مسئله 21.6: گره‌های بن‌بست (Dead Ends)

گره بن‌بست یا Dead End در گراف وب به چه معناست و حضور آن در الگوریتم PageRank چه مشکل ریاضیاتی برای ماتریس احتمالات ایجاد می‌کند؟

مسئله 21.7: تله‌های عنکبوتی (Spider Traps)

تله عنکبوتی چیست؟ تفاوت ساختاری آن با گره بن‌بست را توضیح دهید و بگویید چرا موج‌سوار تصادفی در آن گیر می‌افتد.

مسئله 21.8: عامل میرایی (Damping Factor)

نقش پارامتر **Damping Factor** که معمولاً با α یا d نمایش داده می‌شود، در فرمول **PageRank** چیست؟ اگر مقدار آن 1 باشد، چه اتفاقی می‌افتد؟

مسئله 21.9: رتبه صفحه موضوعی (Topic-Specific PageRank)

تفاوت اصلی فرمول **PageRank** کلاسیک با **Topic-Specific PageRank** در کدام بخش از معادله پرش تصادفی (Teleportation) نهفته است؟

مسئله 21.10: هاب و اتوریته (Hubs and Authorities)

در الگوریتم **HITS**، مفاهیم **Hub** و **Authority** را تعریف کنید. یک صفحه خوب **Hub** چه ویژگی‌هایی دارد؟

مسئله 21.11: مجموعه ریشه و پایه (Root Set and Base Set)

در الگوریتم **HITS**، تفاوت بین **Root Set** (که از موتور جستجوی متنی به دست می‌آید) و **Base Set** چیست و **Base Set** چگونه ساخته می‌شود؟

مسئله 21.12: همگرایی در الگوریتم (Algorithm Convergence)

منظور از همگرایی در محاسبه الگوریتم‌های **PageRank** و **HITS** چیست و به صورت تجربی معمولاً بعد از چند تکرار (Iteration) این اتفاق رخ می‌دهد؟

مسئله 21.13: سناریوی تغییرات متن لینک (Anchor Text Variations Scenario)

یک وبسایت فروشگاهی برای صفحه «گوشی هوشمند» خود، لینک‌هایی با عنوان «خرید موبایل»، «گوشی ارزان» و «کلیک کنید» دریافت کرده است. به عنوان مهندس جستجو، چگونه سناریویی طراحی می‌کنید که وزن کلمات عمومی مثل «کلیک کنید» در **Anchor Text** کاهش یابد؟

مسئله 21.14: محاسبه حالت پایدار در گراف کوچک (Steady-State Calculation Scenario)

یک گراف وب شامل 3 صفحه A ، B و C است. A به B و C لینک داده، B فقط به C و C فقط به A لینک داده است. با در نظر گرفتن $d = 0.85$ ، سیستم معادلات خطی برای محاسبه **PageRank** این 3 صفحه را بنویسید.

مسئله 21.15: سناریوی تغییر عامل میرایی (Damping Factor Variation Scenario)

فرض کنید در یک موتور جستجوی سازمانی (اینترانت)، طول مسیر وب‌گردی کاربران بسیار کوتاه‌تر از وب جهانی است. آیا در این سناریو، شما پارامتر میرایی (d) را نسبت به مقدار استاندارد 0.85 کاهش می‌دهید یا افزایش؟ دلیل خود را با تحلیل رفتار **Random Surfer** بیان کنید.

مسئله 21.16: تحلیل ماتریس تله‌پورت (Teleportation Matrix Analysis)

نشان دهید که افزودن بردار تله‌پورت به ماتریس گذار، باعث می‌شود گراف وب به یک گراف کاملاً متصل (Fully Connected) تبدیل شود. این کار چگونه مشکل ارگودیک نبودن (Non-ergodicity) زنجیره مارکوف را حل می‌کند؟

مسئله 21.17: سناریوی شخصی‌سازی اخبار (News Personalization Scenario)

شما در حال طراحی موتور جستجو برای یک پورتال خبری هستید. کاربر X همیشه اخبار ورزشی می‌خواند. چگونه از Topic-Specific PageRank استفاده می‌کنید تا در جستجوی کلمه «عقاب» (که هم پرنده است و هم نام یک تیم/ورزشکار)، نتایج به سمت ورزش گرایش پیدا کند؟

مسئله 21.18: طراحی بردار شخصی‌سازی (Designing Personalization Vector)

در مسئله قبل، بردار پرش تصادفی v را چگونه مقداردهی می‌کنید اگر سایت شما دارای 1000 صفحه باشد که 200 تای آن تگ «ورزشی» دارند؟

مسئله 21.19: سناریوی محاسبه یک تکرار هیتس (Single Iteration HITS Scenario)

در یک گراف، صفحه X به صفحات Y و Z لینک داده است. صفحه W نیز به Z لینک دارد. فرض کنید در ابتدا تمام مقادیر Hub و Authority برابر 1 هستند. بعد از یک مرحله به‌روزرسانی (بدون نرمال‌سازی)، مقادیر Hub برای X و مقادیر Authority برای Z چقدر خواهد بود؟

مسئله 21.20: مشکل انحراف موضوعی در هیتس (Topic Drift Problem Scenario)

فرض کنید کوئری کاربر «آموزش جاوا» است. در مرحله تشکیل Base Set، یکی از صفحات ریشه به سایت yahoo.com لینک داده است. چرا این موضوع باعث می‌شود yahoo.com در الگوریتم HITS اتوریتی (Authority) بالایی بگیرد با وجود اینکه ربطی به جاوا ندارد؟ این سناریو به چه نامی شناخته می‌شود؟

مسئله 21.21: مقایسه رفتار رتبه صفحه و هیتس (PageRank vs HITS Behavior Scenario)

شما قصد دارید برای یک موتور جستجوی پزشکی که روزانه هزاران مقاله جدید در آن منتشر می‌شود، سیستم رتبه‌دهی بسازید. با توجه به اینکه مقالات جدید لینک ورودی (In-link) زیادی ندارند، استفاده از الگوریتم HITS در زمان اجرای کوئری منطقی‌تر است یا PageRank آفلاین؟ تحلیل کنید.

مسئله 21.22: مهندسی معکوس مزرعه لینک (Reverse Engineering a Link Farm) ★

یک سئوکار کلاه سیاه 10000 صفحه بی‌محتوا ایجاد کرده که همگی به صورت حلقوی به یکدیگر لینک داده‌اند و همگی یک لینک خروجی به صفحه هدف (Target) دارند. با فرمول‌های ریاضی اثبات کنید که چگونه این ساختار باعث انباشت PageRank در صفحه هدف می‌شود. راهکار سیستمی شما برای شناسایی این گراف‌های متراکم مصنوعی چیست؟

مسئله 21.23: بمب‌گذاری گوگل و متن لینک (Google Bombing Scenario) ★

گروهی از هکرها هزاران لینک با عنوان «بدترین شرکت دنیا» به سایت شرکت رقیب ایجاد کرده‌اند. از نظر ساختار گراف وب (21.1.1)، این کوئری چگونه موتور جستجو را فریب می‌دهد؟ الگوریتمی طراحی کنید که با تحلیل توزیع کلمات در Anchor Text ها، این نوع حملات (Google Bombing) را خنثی کند.

مسئله 21.24: مدل‌سازی ریاضی تله عنکبوتی پیچیده (Mathematical Modeling of a Spider Trap) ★

فرض کنید گرافی شامل دو بخش است: بخش سالم وب (با N صفحه) و یک تله عنکبوتی (با M صفحه). تله به این شکل است که هیچ لینکی به بیرون ندارد. معادله توزیع حالت پایدار بردار رتبه π را بر حسب ماتریس گذار P و بردار تله‌پورت e بنویسید. ثابت کنید که با گذشت زمان، اگر $\alpha \rightarrow 1$ میل کند، تمام وزن سیستم به سمت تله عنکبوتی مکیده می‌شود.

مسئله 21.25: مقیاس‌پذیری و گراف‌های عظیم (Scalability in Massive Graphs) ★

گراف وب دارای 100 میلیارد گره است. ماتریس گذار در حافظه هیچ سروری جا نمی‌شود. الگوریتم PageRank را با پارادایم MapReduce بشکنید. توابع Map و Reduce را برای یک مرحله تکرار Power Iteration بنویسید و توضیح دهید چگونه مشکل گره‌های بن‌بست (Dead ends) را در فاز Reduce مدیریت می‌کنید.

مسئله 21.26: نقض ارگودیسیت در زنجیره مارکوف (Ergodicity Violation Scenario) ★

نشان دهید اگر گراف وب شامل یک گراف دوگانه (Bipartite) کاملاً مجزا از بقیه وب باشد که صفحات لایه اول فقط به لایه دوم و برعکس لینک می‌دهند، ماتریس گذار بدون بردار تله‌پورت متناوب (Periodic) خواهد بود. این تناوب چه بلایی بر سر همگرایی متد Power Iteration می‌آورد؟

مسئله 21.27: بهینه‌سازی بردار پرش تصادفی (Optimizing Random Jump Vector) ★

به جای پرش تصادفی یکنواخت در PageRank، می‌خواهیم احتمال پرش به هر صفحه با کیفیت محتوای آن (TrustScore) متناسب باشد. فرمول جدید PageRank را با استفاده از بردار TrustScore (که جمع درایه‌هایش 1 است) بازنویسی کنید و استدلال کنید که این روش چگونه با اسپم وب (Web Spam) مبارزه می‌کند.

مسئله 21.28: رتبه صفحه موضوعی در لحظه (Real-Time Topic-Specific PR) ★

شما 100 موضوع مختلف دارید و می‌خواهید Topic-Specific PageRank را پیاده‌سازی کنید. محاسبه 100 بردار PageRank برای میلیاردها صفحه به صورت آفلاین هزینه بالایی دارد. چگونه با محاسبه بردارهای پایه (Basis Vectors) به صورت آفلاین، می‌توانید رتبه صفحه برای یک کاربر با ترکیب علایق (مثلاً 60% ورزشی، 40% تکنولوژی) را به صورت در لحظه و خطی (Linear Combination) محاسبه کنید؟ اثبات ریاضی خطی بودن این ترکیب را بنویسید.

مسئله 21.29: تحلیل مقادیر ویژه در هیتس (Eigenvalue Analysis in HITS) ★

در الگوریتم HITS، بردارهای Hub (h) و Authority (a) با ماتریس مجاورت A به صورت $a = A^T h$ و $h = A a$ آپدیت می‌شوند. با جایگذاری، نشان دهید که بردار Authority در واقع بردار ویژه (Eigenvector) متناظر با بزرگترین مقدار ویژه برای کدام ماتریس متقارن است؟ این موضوع چه بینشی درباره رفتار گراف‌کاوی HITS به ما می‌دهد؟

مسئله 21.30: سناریوی فیلترینگ مجموعه پایه (Base Set Filtering Scenario) ★

در **HITS**، اگر کوثری کلمه «اپل» باشد، **Base Set** شامل سایت‌های فروش میوه و سایت‌های تکنولوژی می‌شود. این موضوع باعث ایجاد دو **Community** مجزا در گراف می‌شود. چگونه با تحلیل مقادیر ویژه دوم و سوم ($A^T A$) ماتریس **Second and Third Eigenvectors** می‌توانیم این دو جامعه را از هم تفکیک کنیم تا به کاربر اجازه انتخاب دسته‌بندی بدهیم؟

مسئله 21.31: حمله سیل در گراف وب (Sybil Attack in Web Graph) ★

در یک شبکه وب غیرمتمرکز، یک مهاجم هزاران نود جعلی (**Sybil**) می‌سازد تا رتبه یک مقاله خاص را بالا ببرد. مقایسه کنید که الگوریتم‌های **HITS** و **PageRank** هر کدام چقدر در برابر این حمله آسیب‌پذیر هستند؟ کدام یک سریع‌تر تحت تاثیر تعداد لینک‌های ورودی جعلی قرار می‌گیرد؟

مسئله 21.32: رتبه صفحه برای محتوای پویا (PageRank for Dynamic Content) ★

در شبکه‌های اجتماعی (مثل توییتر که ساختار گراف فالوورها دارد)، محتوا هر ثانیه تولید می‌شود. اعمال **PageRank** کلاسیک روی این گراف با تغییرات مداوم غیرممکن است. یک الگوریتم تقریبی (**Approximation Algorithm**) پیشنهاد دهید که با استفاده از گراف محلی (**Local Graph Neighborhood**) آپدیت‌های در لحظه برای رتبه صفحات/کاربران جدید ایجاد کند.

مسئله 21.33: وزن‌دهی به یال‌ها بر اساس رفتار کاربر (Edge Weighting via User Behavior) ★

در فرمول استاندارد، موج‌سوار تصادفی از هر لینک با احتمال مساوی $1/N_{out}$ عبور می‌کند. حال فرض کنید دیتای کلیک کاربران (**Click-through Log**) را در اختیار دارید. مدل مارکوف را به گونه‌ای بازنویسی کنید که احتمالات گذار (**Transition Probabilities**) تابعی از نرخ کلیک (**CTR**) لینک‌ها باشند. این کار چه چالشی در زمینه توزیع احتمالات (**Stochastic Matrix**) ایجاد می‌کند و راه‌حل شما چیست؟

مسئله 21.34: هیتس در گراف‌های دوگانه ناقص (HITS in Imperfect Bipartite Graphs) ★

گراف وب هرگز یک گراف دوگانه (**Bipartite**) کامل نیست و صفحات می‌توانند همزمان **Hub** و **Authority** باشند. فرمول‌بندی **HITS** را گسترش دهید تا در آن، اتوریتی خود یک صفحه به طور مستقیم روی هاب بودن خودش تاثیر بگذارد (یک حلقه بازخورد داخلی). معادلات جدید به‌روزرسانی را بنویسید.

مسئله 21.35: تقلب در اتوریتی با استفاده از هاب‌های جعلی (Authority Manipulation via Fake Hubs) ★

من یک سایت مرجع (**Authority**) دارم و می‌خواهم رتبه آن را در **HITS** بالا ببرم. اگر ۱۰۰ سایت وبلاگ جعلی بسازم (**Hub**) که به سایت من و سایت‌های بسیار معتبر مثل ویکی‌پدیا و بی‌بی‌سی لینک بدهند، چه اتفاقی برای امتیاز هاب وبلاگ‌های من و به تبع آن امتیاز اتوریتی سایت هدفم می‌افتد؟ این باگ را چگونه در سیستم **Patch** می‌کنید؟

مسئله 21.36: طراحی سیستم ترکیبی بازیابی اطلاعات (Designing a Hybrid IR System) ★

معماری یک سیستم جستجوی وب را طراحی کنید که در آن:

1. ارزیابی متن لینک‌ها (Anchor Text) با مدل TF-IDF ترکیب می‌شود.

2. PageRank به صورت آفلاین برای تعیین کیفیت عمومی سایت‌ها محاسبه می‌شود.

3. HITS به صورت در لحظه روی Top 1000 نتایج اولیه اجرا می‌شود.

تابع نهایی رتبه‌بندی (Scoring Function) خود را به صورت فرموله بنویسید و پارامترهایی که برای کالیبره کردن این سه

بخش (متن، PageRank، HITS) استفاده می‌کنید را شرح دهید.

پاسخنامه مسئله 21.1: مفهوم گراف وب (The Web as a Graph Concept)

مقدمه و هدف مسئله

در مباحث بازیابی اطلاعات (Information Retrieval) و کاوش وب (Web Mining)، برای تحلیل ساختار شبکه جهانی وب و اجرای الگوریتم‌هایی نظیر رتبه‌بندی صفحه (PageRank)، نیازمند یک مدل ریاضی هستیم. بهترین ابزار برای این مدل‌سازی، «نظریه گراف» است. هدف این مسئله، درک نحوه نگاشت ساختار وب به یک گراف ریاضی و تحلیل ویژگی بنیادی آن یعنی «پراکندگی» (Sparsity) است.

گام 1) تعریف گراف وب و اجزای آن

از منظر ریاضی، شبکه جهانی وب را می‌توان به صورت یک گراف جهت‌دار (Directed Graph) به شکل $G = (V, E)$ مدل‌سازی کرد. در این مدل‌سازی، نگاشت مفاهیم به شرح زیر است:

1. گره‌ها یا رأس‌ها (V - Nodes/Vertices):

مفهوم در وب: هر گره در این گراف، دقیقاً نماینده یک «صفحه وب» (Web Page) یا یک سند (مانند فایل PDF، تصویر و غیره) است.

شناسه (ID): هر گره در این گراف با یک آدرس منحصر به فرد یا همان URL شناسایی می‌شود.

2. یال‌ها یا پیوندها (E - Edges/Links):

مفهوم در وب: هر یال، نماینده یک «ابریوند» (Hyperlink) است که یک صفحه وب را به صفحه دیگری متصل می‌کند.

اهمیت جهت‌دار بودن: یال‌ها در گراف وب «جهت‌دار» (Directed) هستند. اگر صفحه A دارای یک لینک به صفحه B باشد، یک یال جهت‌دار از A به B رسم می‌شود ($A \rightarrow B$). این جهت‌دار بودن به این دلیل است که وجود لینک از A به B، لزوماً به معنای وجود لینک متقابل از B به A نیست.

گام 2) درک مفهوم گراف پراکنده (Sparse Graph)

در نظریه گراف، گراف‌ها بر اساس تعداد یال‌هایشان به دو دسته کلی «متراکم» (Dense) و «پراکنده» (Sparse) تقسیم می‌شوند. گراف پراکنده گرافی است که در آن تعداد یال‌های موجود، بسیار کمتر از حداکثر تعداد یال‌های ممکن بین گره‌ها باشد.

گام 3) چرا گراف وب به شدت پراکنده (Sparse) است؟

برای اثبات این موضوع که چرا فرض می‌کنیم گراف وب به شدت پراکنده است، مسئله را با یک استدلال منطقی و ریاضی بررسی می‌کنیم:

1. بررسی حداکثر یال‌های ممکن:

فرض کنید وب دارای **N** صفحه (گره) است. اگر قرار بود هر صفحه وب به تمامی صفحات دیگر موجود در اینترنت لینک بدهد (تشکیل یک گراف کامل جهت‌دار)، تعداد کل پیوندهای ممکن در این گراف برابر بود با:

$$|E_{max}| = N \times (N - 1) \approx N^2$$

در دنیای واقعی، تعداد صفحات وب (**N**) در مقیاس میلیاردها (و حتی تریلیون‌ها) صفحه است. بنابراین، **N²** عددی فوق‌العاده نجومی و غیرقابل تصور خواهد بود.

2. بررسی تعداد یال‌های واقعی (درجه خروجی):

در عمل، یک صفحه وب معمولی (مثل یک مقاله خبری، یک پست وبلاگ یا یک صفحه محصول) به طور میانگین تنها شامل تعداد محدودی لینک خروجی است (مثلاً بین **10** تا **100** لینک). به این میانگین لینک‌های خروجی هر صفحه، میانگین درجه خروجی (**Average Out-degree**) می‌گویند که آن را با **k** نشان می‌دهیم. **k** در مقایسه با **N** یک عدد ثابت و بسیار کوچک است (**k** << **N**).

بنابراین، تعداد کل یال‌های واقعی (لینک‌های موجود در کل اینترنت) تقریباً برابر است با:

$$|E_{actual}| \approx k \times N$$

3. نتیجه‌گیری ریاضی و منطقی:

با مقایسه تعداد یال‌های واقعی و حداکثر یال‌های ممکن، به وضوح می‌بینیم که:

$$k \times N \ll N^2 \implies |E_{actual}| \ll |E_{max}|$$

تعداد لینک‌های موجود در وب به شکل چشمگیری کمتر از ظرفیت کل لینک‌های ممکن است. به همین دلیل، گراف وب یک گراف به شدت پراکنده (**Highly Sparse Graph**) در نظر گرفته می‌شود.

نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

چرا پراکنده بودن گراف وب مهم است؟

درک این ویژگی برای پیاده‌سازی موتورهای جستجو حیاتی است. اگر گراف وب متراکم بود، برای ذخیره آن در حافظه کامپیوتر باید از «ماتریس مجاورت» (**Adjacency Matrix**) استفاده می‌کردیم که به فضایی از مرتبه **O(N²)** نیاز داشت (که برای میلیاردها صفحه وب به دلیل کمبود حافظه **RAM** کاملاً غیرممکن است). اما به دلیل پراکندگی شدید این گراف، ما ساختار وب را با استفاده از «لیست مجاورت» (**Adjacency List**) ذخیره می‌کنیم که تنها به فضایی از مرتبه **O(N)** نیاز دارد و پردازش گراف وب را روی سرورها امکان‌پذیر می‌سازد.

ویژگی ساختاری	ماتریس مجاورت (Adjacency Matrix)	لیست مجاورت (Adjacency List)
پیچیدگی فضایی (Space Complexity)	$O(N^2)$	$O(N + E) \approx O(N)$
میزان اتلاف حافظه برای گراف وب	بسیار بالا (ذخیره میلیاردها صفر برای یال‌های ناموجود)	بسیار بهینه (صرفاً لینک‌های واقعی ذخیره می‌شوند)
کارایی در اینترنت واقعی	غیرممکن (Out of Memory)	استاندارد صنعتی موتورهای جستجو

پاسخنامه مسئله 21.2: اهمیت متن لینک (Importance of Anchor Text)

مقدمه و هدف مسئله

در معماری موتورهای جستجو (مانند گوگل)، استخراج معنا و ارتباط یک صفحه وب با عبارات جستجو شده، تنها به محتوای درون خود آن صفحه محدود نمی‌شود. موتورهای جستجو از ساختار پیوندهای وب به عنوان یک منبع غنی از اطلاعات استفاده می‌کنند. هدف این مسئله، بررسی این موضوع است که چرا «متن پیوند» (**Anchor Text**) به عنوان یک متادیتای خارجی، گاهی سیگنال قوی‌تر و معتبرتری نسبت به محتوای داخلی خود صفحه برای رتبه‌بندی (**Ranking**) ارائه می‌دهد.

گام 1) تعریف متن لینک (Anchor Text)

متن لینک یا **Anchor Text**، همان عبارت یا کلمه قابل کلیک است که کاربران با فشردن آن به یک صفحه دیگر هدایت می‌شوند. در کدهای **HTML**، این متن بین تگ‌های پیوند قرار می‌گیرد؛ به عنوان مثال در قطعه کد زیر، عبارت «**مبانی بازیابی اطلاعات**» همان **Anchor Text** است:

مبانی بازیابی اطلاعات

گام 2) چالش اعتماد به محتوای نویسنده (Self-Description)

برای درک ارزش **Anchor Text**، ابتدا باید ضعف تکیه صرف بر محتوای خود صفحه را بشناسیم. اگر یک موتور جستجو تنها به کلمات داخل یک صفحه وب اعتماد کند، با پدیده‌ای به نام «**هرزنگاری کلمات کلیدی**» (**Keyword Stuffing**) مواجه می‌شود. طراحان وب‌سایت‌ها می‌توانند برای فریب موتور جستجو، کلمات کلیدی پر جستجو را به دفعات در صفحه خود تکرار کنند، حتی اگر آن کلمات ارتباطی با محتوای واقعی صفحه نداشته باشند. در واقع، محتوای صفحه نشان‌دهنده چیزی است که نویسنده ادعا می‌کند، که لزوماً بی‌طرفانه یا دقیق نیست.

گام 3) دلایل برتری و ارزش بالای Anchor Text

موتورهای جستجو به 3 دلیل کلیدی، ارزش فوق‌العاده‌ای برای متن لینک قائل هستند:

1. ارزیابی بی‌طرفانه و شخص ثالث (Objective Third-Party Endorsement):

متن لینک توسط فردی نوشته می‌شود که به یک صفحه ارجاع داده است، نه سازنده آن صفحه. وقتی صفحه **A** به صفحه **B** لینک می‌دهد، **Anchor Text** در واقع نظر نویسنده صفحه **A** در مورد محتوای صفحه **B** است. موتورهای جستجو این را به عنوان یک «رأی اعتماد» و توصیفی بی‌طرفانه از سوی جامعه وب در نظر می‌گیرند.

2. خلاصه‌سازی دقیق و طبیعی (Concise Summarization):

کاربران به طور طبیعی برای لینک دادن، از موجزترین و دقیق‌ترین کلمات استفاده می‌کنند تا به خواننده بگویند با کلیک روی لینک با چه چیزی مواجه می‌شود. این عبارات کوتاه، چکیده‌ای بسیار غنی و باکیفیت از موضوع اصلی صفحه مقصد هستند که استخراج معنا را برای موتور جستجو بسیار آسان‌تر از پردازش کل متن یک مقاله طولانی می‌کنند.

3. نمایه‌سازی محتوای غیرمتنی (Indexing Non-Textual Content):

بسیاری از صفحات وب فاقد متن قابل خواندن توسط ماشین هستند (مانند صفحاتی که صرفاً حاوی یک تصویر، ویدیو، فایل PDF یا برنامه‌های فلش قدیمی هستند). در این حالت، موتور جستجو نمی‌تواند محتوای صفحه را بخواند. در اینجا **Anchor Text** به عنوان یک برچسب (**Tag**) عمل کرده و به موتور جستجو می‌گوید که این فایل غیرمتنی درباره چه موضوعی است.

گام 4) نتیجه‌گیری مکانیزم موتور جستجو و مدل‌سازی ریاضی

ترکیب این دلایل باعث می‌شود که موتورهای جستجو بتوانند یک صفحه را برای یک کلمه کلیدی خاص در رتبه‌های بالا نمایش دهند، حتی اگر آن کلمه کلیدی یک بار هم در متن خود آن صفحه نوشته نشده باشد. موتور جستجو نتیجه می‌گیرد: «اگر صدها وب‌سایت مختلف، با عبارت "بهترین کتاب ساختمان داده" به یک صفحه خاص لینک داده‌اند، پس آن صفحه قطعاً درباره همین موضوع است، فارغ از اینکه نویسنده‌اش چه ادعایی دارد.»

در مدل‌های پیشرفته بازبازی اطلاعات، برای دخالت دادن این مفهوم در رتبه‌بندی، امتیاز نهایی یک سند (**d**) برای یک پرسمان (**q**) به صورت ترکیب خطی از وزن محتوای داخلی و وزن متن لینک‌ها محاسبه می‌شود:

$$Score(q, d) = \alpha \cdot ContentScore(q, d) + \beta \cdot AnchorScore(q, d)$$

در این رابطه، وزن β به دلیل اعتبار بالای متن لینک، اغلب تأثیر بسزایی در تعیین جایگاه نهایی سند در نتایج جستجو ایفا می‌کند.

مقایسه تطبیقی سیگنال‌های رتبه‌بندی

ویژگی مورد ارزیابی	محتوای صفحه (Page Content)	متن لینک (Anchor Text)
منبع توصیف و تولید	ادعای شخصی نویسنده صفحه (درونی)	تأییدیه بی‌طرفانه دیگران (بیرونی)
آسیب‌پذیری در برابر اسپم	بالا (امکان Keyword Stuffing)	پایین‌تر (نیازمند کنترل سایت‌های دیگر)
فایل‌های غیرمتنی	ناتوان در پردازش (مثل تصاویر و ویدیوها)	بسیار کارآمد (عملکرد مشابه برچسب‌گذاری)

نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

آسیب‌پذیری موتورهای جستجو (Google Bombing):

ارزش بالای **Anchor Text** در الگوریتم‌های اولیه، منجر به پیدایش یک تکنیک هک رتبه‌بندی به نام «بمب گوگلی» شد. در این روش، گروهی از افراد با ایجاد هزاران لینک با یک **Anchor Text** خاص (غالباً تمسخرآمیز) به یک سایت مشخص، موتور جستجو را فریب می‌دادند تا آن سایت را برای آن کلمه کلیدی در رتبه اول نمایش دهد. موتورهای جستجوی امروزی با استفاده از تحلیل

معنایی و بررسی اعتبار سایت‌های مبدأ، این مشکل را تا حد زیادی برطرف کرده‌اند، اما **Anchor Text** همچنان یکی از قوی‌ترین سیگنال‌های رتبه‌بندی در وب به شمار می‌رود.

پاسخنامه مسئله 21.3: متغیرهای پنهان در گراف (Hidden Variables in Graph)

مقدمه و هدف مسئله

در تحلیل شبکه وب، اطلاعات تنها در کلمات و متون صفحات (محتوای صریح) خلاصه نمی‌شوند. بخش عظیمی از دانش در «ساختار توپولوژیک» (**Topological Structure**) گراف وب نهفته است. هدف این مسئله، درک نحوه استخراج ویژگی‌های کیفی و اعتباری یک صفحه (به عنوان متغیرهای پنهان) است؛ آن هم صرفاً با تحلیل یال‌های ورودی (**In-links**) به آن گره در گراف، بدون نیاز به پردازش حتی 1 کلمه از محتوای درون صفحه.

گام 1) پیوند به عنوان «رأی اعتماد» (Link as a Vote of Confidence)

در ساختار وب، قرار دادن یک لینک خروجی از صفحه **A** به صفحه **B** نیازمند تصمیم‌گیری و صرف زمان توسط نویسنده صفحه **A** است. معمولاً هیچ‌کس به یک صفحه بی‌کیفیت یا نامرتب لینک نمی‌دهد. بنابراین، از منظر موتور جستجو، هر پیوند ورودی (**In-link**) به یک صفحه، معادل یک «رأی اعتماد» (**Endorsement**) یا توصیه از طرف جامعه وب است.

گام 2) مفهوم «متغیرهای پنهان» (Hidden Variables)

متغیرهایی مانند «کیفیت» (**Quality**)، «اعتبار» (**Authority**) و «قابل اعتماد بودن» (**Trustworthiness**)، ویژگی‌هایی نیستند که در کدهای **HTML** یک صفحه به صورت صریح نوشته شده باشند (نویسنده نمی‌تواند با یک تگ **HTML** به موتور جستجو بگوید "صفحه من باکیفیت است"). به این ویژگی‌ها، متغیرهای پنهان می‌گویند. گراف وب به ما اجازه می‌دهد این متغیرهای پنهان را از طریق رفتار جمعی (**Collective Behavior**) سایر گره‌ها کشف کنیم. اگر صفحه‌ای دارای تعداد زیادی یال ورودی باشد، نشان‌دهنده این متغیر پنهان است که آن صفحه احتمالاً دارای محتوای باکیفیت و مرجع است.

گام 3) توسعه بازگشتی کیفیت (Recursive Nature of Authority)

بررسی پیوندهای ورودی تنها به «تعداد» (**Quantity**) محدود نمی‌شود، بلکه «کیفیت» (**Quality**) لینک‌دهنده‌ها نیز یک متغیر پنهان بسیار حیاتی است. در گراف وب، همه آرا (لینک‌ها) ارزش برابری ندارند. منطق ریاضی حاکم بر این شبکه‌ها به این شکل است:

اگر یک سایت بسیار معتبر و شناخته‌شده (مانند وب‌سایت یک دانشگاه معتبر یا یک خبرگزاری بین‌المللی) به صفحه شما لینک بدهد، اعتبار بسیار بالایی به صفحه شما منتقل می‌شود.

در مقابل، صدها لینک ورودی از وبلاگ‌های ناشناخته، تأثیر به مراتب کمتری خواهند داشت.

بنابراین، کیفیت یک صفحه به کیفیت صفحاتی بستگی دارد که به آن لینک داده‌اند؛ و کیفیت آن صفحات نیز به نوبه خود به لینک‌های ورودی خودشان بستگی دارد. این یک ماهیت «بازگشتی» (**Recursive**) ایجاد می‌کند که به دقت ارزش پنهان گره‌ها را محاسبه می‌نماید. این منطق به صورت ریاضی در موتورهای جستجو با معادله زیر مدل‌سازی می‌شود:

$$PR(u) = \frac{1-d}{N} + d \sum_{v \in B_u} \frac{PR(v)}{L(v)}$$

نمودار شماتیک جریان اعتبار و متغیر پنهان در گراف وب



گام 4) استقلال ارزیابی از محتوای متنی (Content-Agnostic Evaluation)

اما چرا این روش بدون خواندن محتوای صفحه کار می‌کند؟ زمانی که ما گراف لینک‌ها را تحلیل می‌کنیم، در واقع ارزیابی کیفیت را به خرد جمعی کل کاربران اینترنت «برون‌سپاری» (Crowdsourcing) کرده‌ایم. موتور جستجو نیازی ندارد بداند متن صفحه به چه زبانی نوشته شده، آیا شامل کلمات کلیدی هست یا خیر، و یا اینکه اصلاً متن است یا یک گالری تصویر. موتور جستجو به سادگی استدلال می‌کند: "من نمی‌دانم داخل این گره چیست، اما از آنجا که گره‌های معتبر زیادی در گراف به آن اشاره (یال ورودی) دارند، پس قطعاً این گره حاوی محتوای ارزشمندی است."

نکته آموزشی

تولد الگوریتم پیج‌رنک (PageRank) و هیتس (HITS):

این منطق که «اعتبار یک صفحه ناشی از اعتبار لینک‌دهندگان به آن است»، دقیقاً هسته مرکزی و ایده انقلابی الگوریتم PageRank بود که توسط لری پیج و سرگی برین (بنیان‌گذاران گوگل) معرفی شد و موتورهای جستجو را برای همیشه متحول کرد. همچنین در الگوریتم HITS (طراحی شده توسط جان کلاینبِرگ)، این متغیرهای پنهان به 2 مفهوم مجزا تقسیم شدند:

1. Hubs (مراکز توزیع): صفحاتی که لینک‌های خروجی خوبی به منابع معتبر دارند.

2. Authorities (مراجع): صفحاتی که لینک‌های ورودی خوبی از Hub‌های معتبر دریافت کرده‌اند.

درک این دینامیک، پایه و اساس تحلیل شبکه‌های اجتماعی (Social Network Analysis) و وب‌کاوی را تشکیل می‌دهد.

پاسخنامه مسئله 21.4: زنجیره مارکوف در وب (Markov Chains in the Web)

مقدمه و هدف مسئله

یکی از چالش‌های اصلی موتورهای جستجو در مقیاس وب، محاسبه میزان اهمیت صفحات مستقل از محتوای متنی آن‌هاست. الگوریتم پیج‌رنک (**PageRank**) برای حل این مشکل، وب را به عنوان یک «موج‌سوار تصادفی» مدل‌سازی کرد. هدف این مسئله، درک ارتباط ریاضی بین گشت‌وگذار تصادفی در گراف وب و رسیدن به یک احتمال ثابت (جهت رتبه‌بندی صفحات) است.

گام 1) مفهوم «موج‌سوار تصادفی» (The Random Surfer Model)

موج‌سوار تصادفی یک مدل انتزاعی از کاربری است که در اینترنت وب‌گردی می‌کند. تصور کنید کاربری در یک صفحه وب قرار دارد. این کاربر محتوای صفحه را نمی‌خواند، بلکه در هر مرحله، به طور کاملاً تصادفی روی یکی از لینک‌های خروجی آن صفحه کلیک می‌کند تا به صفحه بعدی برود.

اگر صفحه i دارای k لینک خروجی (**Out-degree**) باشد، احتمال اینکه موج‌سوار روی هر یک از آن لینک‌ها کلیک کند و به صفحه j برود، دقیقاً با توزیع یکنواخت برابر با تابع زیر است:

$$P(i \rightarrow j) = \frac{1}{k}$$

گام 2) نگاشت مدل به «زنجیره مارکوف» (Markov Chain)

زنجیره مارکوف یک سیستم ریاضی است که انتقال از یک «حالت» (**State**) به حالت دیگر را بر اساس قوانین احتمالات مدل‌سازی می‌کند. مهم‌ترین ویژگی آن «خاصیت بی‌حافظگی» (**Memoryless Property**) است؛ یعنی وضعیت بعدی سیستم، تنها به وضعیت فعلی بستگی دارد و مستقل از مسیری است که در گذشته طی شده است.

حالت‌ها (States): در گراف وب، هر صفحه اینترنتی یک «حالت» مجزا در زنجیره مارکوف است.

انتقال (Transition): کلیک کردن موج‌سوار روی یک لینک، نشان‌دهنده انتقال از یک حالت به حالت دیگر است. این

مقادیر در یک ماتریس به نام ماتریس انتقال (P) ذخیره می‌شوند که در آن درایه P_{ij} نشان‌دهنده احتمال رفتن از صفحه i به صفحه j است.

گام 3) رابطه موج‌سوار تصادفی و «احتمال حضور» (توزیع ایستا)

وقتی موج‌سوار تصادفی کلیک‌های خود را از یک صفحه به صفحه دیگر تا بی‌نهایت ادامه دهد، چه اتفاقی می‌افتد؟ بر اساس قضایای زنجیره مارکوف، پس از تعداد زیادی تکرار، سیستم به یک نقطه تعادل می‌رسد که به آن «توزیع ایستا» (**Stationary Distribution**) یا حالت پایدار می‌گویند.

رابطه با احتمال حضور: توزیع ایستا برداری مانند π است که در معادله دیفرانسیلی زیر صدق می‌کند. درایه π_i ام این بردار، نشان‌دهنده احتمال این است که اگر در یک زمان تصادفی در آینده سیستم را متوقف کنیم، موج‌سوار در صفحه i حضور داشته باشد.

$$\pi = \pi P$$

تفسیر در بازیابی اطلاعات: در موتورهای جستجو، این «احتمال حضور»، دقیقاً همان امتیاز **PageRank** آن صفحه است. هر چه احتمال حضور موج‌سوار در یک صفحه بیشتر باشد، آن صفحه در ساختار وب مهم‌تر و مرکزی‌تر است.

گام 4) رفع چالش‌های گراف وب با «جهش تصادفی» (Teleportation)

برای اینکه زنجیره مارکوف در گراف وب دارای یک توزیع ایستای یکتای ریاضی باشد، گراف باید «ارگودیک» (**Ergodic**) باشد. در گراف واقعی وب، صفحاتی وجود دارند که هیچ لینک خروجی ندارند (گره‌های بن‌بست - **Dead Ends**) یا حلقه‌های بسته‌ای تشکیل داده‌اند (تله‌های عنکبوتی - **Spider Traps**) که موج‌سوار برای همیشه در آن‌ها گیر می‌افتد. برای حل این مشکل، مدل موج‌سوار با یک «عامل جهش یا میرایی» (**Teleportation/Damping Factor**) که معمولاً مقدار آن $\alpha = 0.85$ در نظر گرفته می‌شود، اصلاح شد:

موج‌سوار با احتمال 85% رفتار عادی دارد و روی یکی از لینک‌های داخل صفحه کلیک می‌کند.

با احتمال 15%، موج‌سوار از کلیک کردن خسته می‌شود، آدرس یک صفحه کاملاً تصادفی در کل اینترنت را در مرورگر تایپ کرده و به آنجا «جهش» (**Teleport**) می‌کند. این کار تضمین می‌کند که موج‌سوار در هیچ بن‌بستی گرفتار نمی‌شود.

فرمول نهایی و کامل پیچ‌رنک که این جهش تصادفی را برای یک صفحه خاص مانند p_i مدل‌سازی می‌کند به شرح زیر است (جایی که N تعداد کل صفحات وب است):

$$PR(p_i) = \frac{1 - \alpha}{N} + \alpha \sum_{p_j \in M(p_i)} \frac{PR(p_j)}{L(p_j)}$$

نمودار شماتیک درخت تصمیم‌گیری در رفتار موج‌سوار تصادفی



نکته آموزشی

الگوریتم **PageRank** با تبدیل کردن اینترنت به یک زنجیره مارکوف عظیم (با میلیاردها حالت)، موفق شد ساختار بی‌نظم وب را بر اساس ریاضیات احتمالات مرتب کند. در این الگو، «اهمیت یک صفحه» پاسخی ریاضی به این سوال ساده است:

پاسخنامه مسئله 21.5: ماتریس گذار (Transition Matrix)

بخش 1: مقدمه و هدف مسئله

ماتریس گذار (Transition Matrix) که غالباً با حرف **P** نشان داده می‌شود، هسته مرکزی محاسبات در مدل «زنجیره مارکوف» است. این ماتریس، ساختار گراف جهت‌دار وب را به یک مدل ریاضی مبتنی بر احتمالات تبدیل می‌کند. هدف این مسئله، درک نحوه محاسبه احتمالات انتقال برای یک گره خاص و نحوه قرارگیری این مقادیر در درایه‌های (عناصر) ماتریس است.

بخش 2: درک ابعاد و ساختار ماتریس گذار

فرض کنید گراف وب دارای **N** صفحه (گره) است. بنابراین، ماتریس گذار **P** یک ماتریس مربعی با ابعاد $N \times N$ خواهد بود. ویژگی‌های این ساختار به شرح زیر است:

سطرها (Rows): هر سطر در این ماتریس، نشان‌دهنده گره «مبدأ» است. سطر مربوط به صفحه **A**، توزیع احتمالات خروج از صفحه **A** را به سمت سایر صفحات نشان می‌دهد.

ستون‌ها (Columns): هر ستون در این ماتریس، نشان‌دهنده گره «مقصد» است.

درایه‌ها (Elements): درایه P_{ij} (تقاطع سطر i و ستون j)، بیانگر احتمال این است که کاربر از صفحه i مستقیماً به صفحه j حرکت کند.

بخش 3: محاسبه احتمالات در مدل پایه (بدون جهش)

بر اساس مدل «موج‌سوار تصادفی» (Random Surfer)، کاربر موجود در صفحه **A** روی یکی از لینک‌های خروجی آن با احتمال برابر کلیک می‌کند. در صورت مسئله ذکر شده که از صفحه **A** به **3** صفحه دیگر لینک خروجی وجود دارد. به بیان ریاضی، «درجه خروجی» (Out-degree) گره **A** برابر با **3** است. بنابراین، احتمال اینکه کاربر روی هر یک از این **3** لینک کلیک کند برابر است با:

$$P(A \rightarrow j) = \frac{1}{\text{Out-degree}(A)} = \frac{1}{3}$$

بخش 4: نحوه پر شدن سطر مربوط به صفحه A

اکنون باید این احتمالات را در سطر **A** از ماتریس $N \times N$ قرار دهیم. مقادیر این سطر عملاً به **2** دسته تقسیم می‌شوند:

1. درایه‌های متناظر با 3 صفحه مقصد (دارای لینک):

در ستون‌های مربوط به این **3** صفحه، مقدار $\frac{1}{3}$ قرار می‌گیرد. این نشان می‌دهد که در صورت حضور در صفحه **A**، احتمال رفتن به هر یک از این سه صفحه **33.3%** است.

2. درایه‌های متناظر با سایر صفحات (بدون لینک):

از آنجا که هیچ لینک مستقیمی از صفحه **A** به $N - 3$ صفحه دیگر گراف (از جمله خود صفحه **A**) وجود ندارد، احتمال

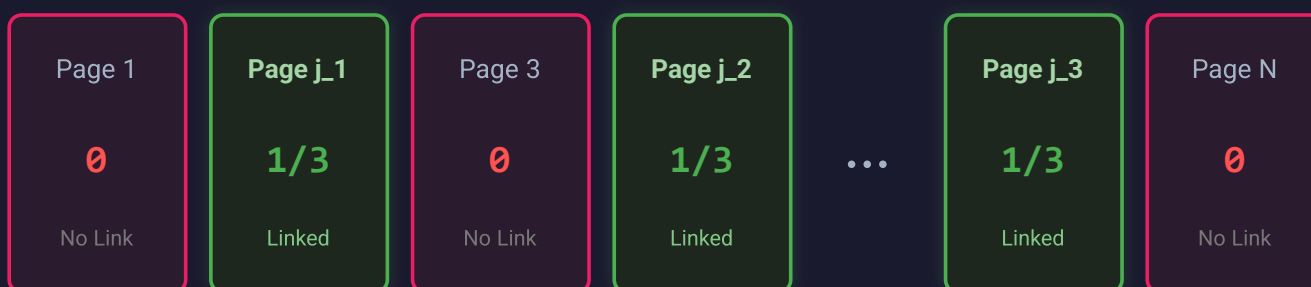
رفتن به آن‌ها در مدل پایه برابر با 0 است. بنابراین، در تمام ستون‌های دیگر این سطر، مقدار 0 قرار می‌گیرد.

اگر فرض کنیم صفحات مقصد به ترتیب j_1, j_2 و j_3 هستند، مقادیر سطر A به صورت تابع چندضابطه‌ای زیر تعریف می‌شود:

$$P_{A,j} = \begin{cases} \frac{1}{3} & \text{if } j \in \{j_1, j_2, j_3\} \\ 0 & \text{otherwise} \end{cases}$$

بخش 5: نمودار شماتیک درایه‌های سطر A در ماتریس گذار

توزیع احتمالات خروجی در سطر A (Row A Array)



مجموع کل درایه‌های سطر A به طور دقیق برابر با 1 (یا 100%) است.

بخش 6: بررسی ویژگی تصادفی بودن سطر (Stochastic Property)

یکی از قوانین بنیادین ماتریس‌های گذار در زنجیره مارکوف این است که جمع احتمالات هر سطر (برای گره‌هایی که بن‌بست نیستند) باید دقیقاً برابر با 1 باشد؛ ویژگی‌ای که در ریاضیات به آن ماتریس تصادفی سطرها (**Row-Stochastic Matrix**) می‌گویند.

در مورد سطر A، مجموع احتمالات به شرح زیر است:

$$\sum_{j=1}^N P_{Aj} = \underbrace{\frac{1}{3} + \frac{1}{3} + \frac{1}{3}}_{3 \text{ Links}} + \underbrace{0 + \dots + 0}_{N-3 \text{ No Links}} = 1$$

این اثبات نشان می‌دهد که کاربر با احتمال 100% بالاخره از صفحه A خارج شده و به یکی از آن 3 صفحه خروجی می‌رود.

بخش 7: جمع‌بندی

محاسباتی که در این مسئله انجام دادیم، معماری بنیادین ماتریس گذار پایه (**Base Transition Matrix**) را تشریح می‌کند. با این حال، باید در نظر داشت که وجود درایه‌های صفر در این ماتریس (احتمال صفر برای رفتن به صفحاتی که لینک مستقیم ندارند)، در گراف‌های عظیم وب باعث بروز پدیده‌هایی نظیر گره‌های بن‌بست (**Dead Ends**) و تله‌های عنکبوتی (**Spider Traps**) می‌گردد که همگرایی مدل مارکوف را با شکست مواجه می‌سازد.

برای حل این چالش حیاتی در الگوریتم‌های موتور جستجو (نظیر **PageRank**)، مهندسان مفهومی به نام **عامل جهش** (**Teleportation / Damping Factor**) را به ماتریس اضافه می‌کنند. با اعمال این پارامتر (که معمولاً با $\alpha = 0.85$ تنظیم می‌شود)، ماتریس پایه به یک ساختار کاملاً متراکم به نام **ماتریس گوگل** (**Google Matrix - G**) ارتقا می‌یابد.

در این ساختار اصلاح‌شده، فرمول درایه‌ها به صورت $G_{ij} = \alpha P_{ij} + (1 - \alpha) \frac{1}{N}$ بازنویسی می‌گردد. نتیجه‌ی این تغییر پارادایم آن است که هیچ درایه‌ای در گراف وب **صفر مطلق نخواهد بود**؛ یعنی مدل ریاضی همیشه یک احتمال حداقلی پرش تصادفی (**Random Jump**) را حتی برای صفحاتی که هیچ پیوند مستقیمی با یکدیگر ندارند، لحاظ می‌کند تا پایداری و همگرایی سیستم رتبه‌بندی به طور کامل تضمین گردد.

پاسخنامه مسئله 21.6: گره‌های بن‌بست (Dead Ends)

مقدمه و هدف مسئله

یکی از چالش‌های بنیادین در تبدیل شبکه اینترنت به یک مدل ریاضی (زنجیره مارکوف)، وجود صفحاتی است که رفتار استاندارد «موج‌سوار تصادفی» را مختل می‌کنند. در این مسئله، به تعریف «گره‌های بن‌بست» (**Dead Ends**) می‌پردازیم و پیامدهای مخرب ریاضیاتی آن‌ها را بر روی ماتریس گذار (**Transition Matrix**) و الگوریتم **PageRank** بررسی می‌کنیم.

گام 1) تعریف گره بن‌بست در گراف وب

در گراف جهت‌دار وب، گره بن‌بست به صفحه‌ای گفته می‌شود که دارای هیچ لینک خروجی (**Out-link**) نیست. به عبارت دیگر، درجه خروجی (**Out-degree**) این گره دقیقاً برابر با 0 است.

از دیدگاه مدل «موج‌سوار تصادفی»، اگر کاربر در وب‌گردی خود به چنین صفحه‌ای برسد، هیچ مسیری (لینکی) برای خروج از آن صفحه و رفتن به صفحات دیگر ندارد و عملاً در آنجا متوقف می‌شود. در دنیای واقعی وب، فایل‌های **PDF**، تصاویر، یا صفحاتی که طراح سایت فراموش کرده در آن‌ها منوی ناوبری (**Navigation**) قرار دهد، نمونه‌های بارزی از گره‌های بن‌بست هستند.

گام 2) مشکل ریاضیاتی در ماتریس احتمالات (ماتریس گذار)

همان‌طور که در مسئله قبلی آموختیم، ماتریس گذار (P) باید دارای یک ویژگی بنیادین باشد: **ماتریس تصادفی سطرها** (**Row-Stochastic Matrix**). این یعنی جمع احتمالات در هر سطر از ماتریس باید دقیقاً برابر با 1 باشد:

$$\sum_{j=1}^N P_{ij} = 1$$

حضور گره بن‌بست این قانون ریاضی را نقض می‌کند:

فرض کنید گره **D** یک بن‌بست است.

چون هیچ لینک خروجی از **D** وجود ندارد، احتمال رفتن از **D** به هر گره دیگری در گراف (حتی خودش) برابر با 0 است.

بنابراین، تمام درایه‌های سطر مربوط به گره **D** در ماتریس گذار برابر با 0 خواهند بود.

نتیجه: جمع مقادیر سطر **D** برابر با **0** می‌شود و ماتریس دیگر **Row-Stochastic** نخواهد بود:

$$\sum_{j=1}^N P_{Dj} = 0$$

گام 3) پیامد بن‌بست‌ها در الگوریتم PageRank (نشت احتمال)

الگوریتم **PageRank** امتیاز صفحات را بر اساس «توزیع احتمال» در یک زنجیره مارکوف محاسبه می‌کند و قانون اصلی احتمالات این است که جمع کل احتمالات در سیستم باید همواره 100% (یا 1) باشد. وقتی ماتریس گذار دارای سطرهای کاملاً صفر (گره‌های بن‌بست) باشد، پدیده‌ای مخرب به نام «نشت احتمال» (**Probability Leakage**) رخ می‌دهد. روند این تخریب به این شکل است:

1. موج‌سواران به مرور زمان وارد گره بن‌بست می‌شوند و مقداری از «احتمال» سیستم را با خود به آنجا می‌برند.

2. چون هیچ راه خروجی وجود ندارد، در گام بعدی محاسبات (ضرب ماتریسی بردار توزیع در ماتریس گذار)، این احتمال در **0** ضرب شده و از بین می‌رود.

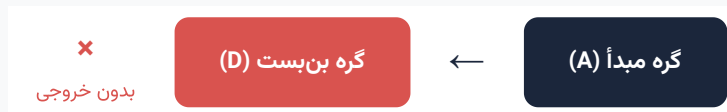
3. با تکرار الگوریتم (**Iteration**)، در هر مرحله مقداری از مجموع کل احتمالات سیستم کسر می‌شود.

4. در نهایت (در بینهایت)، مجموع امتیازات تمام صفحات گراف به **0** میل می‌کند و الگوریتم عملاً هیچ خروجی معناداری تولید نخواهد کرد. این فاجعه ریاضی با حد زیر نشان داده می‌شود:

$$\lim_{t \rightarrow \infty} \sum_{i=1}^N PR_i(t) = 0$$

نمودار شماتیک: تأثیر گره بن‌بست و راهکار ترمیم ماتریس

مدل‌سازی گره بن‌بست در گراف



وضعیت سطر مربوط به گره بن‌بست (سطر D) در ماتریس گذار

قبل از ترمیم (نشت احتمال)	0	0	0	...	0	Sum = 0
بعد از ترمیم (مالیات‌گیری)	1/N	1/N	1/N	...	1/N	Sum = 1

نکته آموزشی

راه‌حل موتورهای جستجو برای مشکل بن‌بست‌ها:

برای حل این نقص ریاضی، الگوریتم **PageRank** از ترفندی به نام «تغییر مسیر تصادفی» (**Random Teleportation**) یا مالیات‌گیری (**Taxation**) استفاده می‌کند. در این روش، فرض می‌شود اگر موج‌سوار به یک گره بن‌بست برسد، متوقف

نمی‌شود؛ بلکه آدرس یک سایت کاملاً جدید را در مرورگر تایپ کرده و با احتمال مساوی ($\frac{1}{N}$) به یکی از صفحات دیگر در کل وب پرش (**Teleport**) می‌کند.

به بیان ریاضی، برای رفع نشت احتمال، ما سطرهای صفر ماتریس را با مقادیر یکنواخت $\frac{1}{N}$ پر می‌کنیم تا جمع سطر مجدداً 1 شود و ماتریس خاصیت تصادفی (**Stochastic**) خود را بازیابد.

پاسخنامه مسئله 21.7: تله‌های عنکبوتی (Spider Traps)

بخش 1: مقدمه و هدف مسئله

علاوه بر گره‌های بن‌بست (**Dead Ends**)، نوع دیگری از ناهنجاری ساختاری در گراف وب وجود دارد که الگوریتم‌های مبتنی بر گشت‌وگذار تصادفی (مانند **PageRank**) را دچار اختلال جدی می‌کند. این ناهنجاری «تله عنکبوتی» نام دارد. هدف این مسئله، شناخت دقیق این ساختار، مقایسه آن با بن‌بست‌ها و تحلیل رفتار ریاضیاتی آن (مکانیسم جذب احتمالات) در یک زنجیره مارکوف است.

بخش 2: تعریف تله عنکبوتی (Spider Trap)

تله عنکبوتی در گراف جهت‌دار وب به مجموعه‌ای از گره‌ها (یا حتی یک گره منفرد) گفته می‌شود که دارای لینک‌های خروجی هستند، اما هیچ‌یک از این لینک‌ها به گره‌های خارج از آن مجموعه اشاره نمی‌کنند. به عبارت ساده‌تر، اگر یک گره (یا گروهی از گره‌ها) فقط به خودش یا منحصرأً به اعضای گروه خودش لینک داده باشد و هیچ پلی به بقیه اینترنت نداشته باشد، یک تله عنکبوتی تشکیل داده است. این ساختارها می‌توانند به صورت عامدانه (توسط اسپمرها برای فریب موتورهای جستجو و تورم **PageRank**) یا به صورت تصادفی (خطای برنامه‌نویسی و معماری سایت) ایجاد شوند.

بخش 3: تفاوت ساختاری و ریاضیاتی با گره بن‌بست

برای درک عمیق‌تر این ناهنجاری، باید مرز ریاضیاتی بین بن‌بست و تله را در ماتریس گذار (P) مشخص کنیم:

گره بن‌بست (Dead End): درجه خروجی (**Out-degree**) آن دقیقاً برابر با 0 است. هیچ لینکی در آن وجود ندارد. در ماتریس گذار، سطر مربوط به این گره کاملاً صفر می‌شود و خاصیت تصادفی سطرها از بین می‌رود.

$$\text{Out-degree} = 0 \implies \sum P_{ij} = 0$$

تله عنکبوتی (Spider Trap): درجه خروجی آن بزرگتر از 0 است. لینک خروجی وجود دارد، اما مقصد تمام این لینک‌ها درون خود تله است (مثلاً گره فقط به خودش لینک داده است: **Self-loop**). در ماتریس گذار، سطر مربوط به این گره‌ها صفر نمی‌شود، بلکه جمع احتمالات آن دقیقاً برابر با 1 است. در نتیجه، ماتریس خاصیت تصادفی (**Stochastic**) بودن خود را به لحاظ تئوری کاملاً حفظ می‌کند.

$$\text{Out-degree} > 0 \implies \sum P_{ij} = 1$$

بخش 4: چرا موج‌سوار تصادفی در تله گیر می‌افتد؟ (Absorbing State)

بر اساس مدل «موج‌سوار تصادفی» (**Random Surfer**)، کاربر با کلیک روی لینک‌ها از صفحه‌ای به صفحه دیگر می‌رود. وقتی موج‌سوار وارد یک تله عنکبوتی می‌شود، چون تمام لینک‌های موجود در آن صفحات به درون خود همان ساختار اشاره دارند، احتمال (**Probability**) خروج از این مجموعه دقیقاً برابر با 0 است.

از منظر ریاضیات زنجیره مارکوف، این مجموعه به یک «حالت جاذب» (**Absorbing State**) تبدیل می‌شود:

1. در هر تکرار (**Iteration**) از الگوریتم **PageRank**، مقداری از توزیع احتمال کل شبکه وارد این تله می‌شود.

2. چون احتمال خروج صفر است، هیچ احتمالی از تله به بقیه گراف برنمی‌گردد.

3. با گذشت زمان (تکرارهای متوالی الگوریتم)، تله عنکبوتی مانند یک سیاه‌چاله تمام احتمال موجود در گراف وب را می‌بلعد و به خود اختصاص می‌دهد.

4. نتیجه نهایی این است که امتیاز **PageRank** گره‌های داخل تله به عدد 1 (یا 100%) میل می‌کند و امتیاز تمامی صفحات دیگر اینترنت به 0 می‌رسد! این فاجعه در قالب حد ریاضی زیر نمایش داده می‌شود:

$$\lim_{t \rightarrow \infty} PR(\text{Trap Node}) = 1, \quad \lim_{t \rightarrow \infty} PR(\text{Other Nodes}) = 0$$

بخش 5: نمودار شماتیک مکانیزم جذب احتمال در تله

مکانیزم بلعیده شدن احتمال در یک تله ساده (Self-loop Trap)



ورود بی‌بازگشت احتمال شبکه



نتیجه ماتریسی: سطر B در ماتریس گذار برابر است با $P_{BB} = 1$ و بقیه درایه‌های آن سطر تماماً صفر هستند.

تله‌های عنکبوتی، حفره‌های سیاه ریاضیاتی در گراف وب هستند که الگوریتم‌های رتبه‌بندی خطی را دچار فروپاشی می‌سازند. با پیشروی در گام‌های زمانی گشت تصادفی (**Markov Iterations**)، توزیع احتمال (امتیاز اعتباری صفحات) از سایر گره‌های شبکه تخلیه شده و به صورت مجانبی در گره تله انباشت می‌گردد، تا جایی که امتیاز کل سیستم در یک بن‌بست محلی بلوکه می‌شود.

جالب توجه است که راه‌حل مهندسی در موتورهای جستجوی مدرن برای خنثی‌سازی تله‌های عنکبوتی، دقیقاً مشابه راهکار مقابله با گره‌های بن‌بست است: **جهش تصادفی (Random Teleportation)**. با معرفی ضریب میرایی (**Damping Factor**) که در الگوریتم **PageRank** معمولاً با پارامتر $\alpha = 0.85$ کالیبره می‌شود، موتور جستجو اینگونه مدل‌سازی می‌کند که کاربر وب با احتمال **15%** از کلیک کردن روی لینک‌های تکراری درون یک تله خسته شده و به صورت مستقیم آدرس یک سایت جدید را در نوار مرورگر وارد می‌نماید.

این پرش تصادفی مداوم، یک راه فرار پایدار و ریاضیاتی برای موج‌سوار ایجاد کرده و از انباشت **100** درصدی احتمالات در تله‌های عنکبوتی جلوگیری به عمل می‌آورد. معادله تصحیح‌شده‌ی ماتریس گذار (ماتریس گوگل) که این مکانیزم خروج را فرموله می‌کند، نمایانگر این مداخله‌ی استراتژیک است:

$$G_{ij} = \alpha P_{ij} + (1 - \alpha) \frac{1}{N}$$

این افزونه، تضمین می‌نماید که سیستم حتی در مخرب‌ترین توپولوژی‌های شبکه‌ای نیز همگرایی قطعی خود را حفظ خواهد کرد.

پاسخنامه مسئله 21.8: عامل میرایی (Damping Factor)

مقدمه و هدف مسئله

همان‌طور که در مسائل قبلی بررسی کردیم، ساختار واقعی وب پر از گره‌های بن‌بست (**Dead Ends**) و تله‌های عنکبوتی (**Spider Traps**) است که باعث فروپاشی مدل پایه مارکوف می‌شوند. لری پیچ و سرگی برین برای حل این مشکل، پارامتری به نام **عامل میرایی (Damping Factor)** را معرفی کردند که با نماد α یا d نشان داده می‌شود. هدف این مسئله، درک جایگاه ریاضی این پارامتر در فرمول **PageRank** و بررسی سناریوی حدی $d = 1$ است.

گام 1) جایگاه Damping Factor در فرمول PageRank

فرمول کلاسیک **PageRank** برای یک صفحه مانند **A** به شکل زیر تعریف و تکمیل می‌شود:

$$PR(A) = \frac{1 - d}{N} + d \sum_{T_i \in M(A)} \frac{PR(T_i)}{C(T_i)}$$

PR(A): امتیاز **PageRank** صفحه **A**.

T_i: صفحاتی که به **A** لینک داده‌اند (پیوندهای ورودی).

C(T_i): تعداد کل لینک‌های خروجی صفحه **T_i** (درجه خروجی).

N: تعداد کل صفحات موجود در شبکه اینترنت.

d: عامل میرایی (Damping Factor)، که عددی بین 0 و 1 است (معمولاً **d = 0.85** تنظیم می‌شود).

گام 2) نقش مفهومی Damping Factor (رفتار موج‌سوار)

این پارامتر رفتار موج‌سوار تصادفی را به 2 بخش تقسیم می‌کند:

1. رفتار کلیک‌محور (با احتمال d):

بخش دوم فرمول که با $d \sum$ شروع می‌شود، نشان‌دهنده رفتار طبیعی کاربر است. یعنی کاربر با احتمال d (مثلاً 85%) تصمیم می‌گیرد روی یکی از لینک‌های موجود در صفحه‌ای که در آن قرار دارد کلیک کند و مسیر گراف را ادامه دهد.

2. رفتار پرش یا جهش تصادفی (با احتمال 1 - d):

بخش اول فرمول یعنی کسر $\frac{1-d}{N}$ نشان‌دهنده «جهش تصادفی» (Teleportation) است. کاربر با احتمال $1 - d$ (مثلاً 15%) از کلیک کردن روی لینک‌ها خسته می‌شود، گراف فعلی را رها کرده و آدرس یک صفحه کاملاً تصادفی در کل اینترنت را در مرورگر تایپ می‌کند. این ویژگی دقیقاً همان چیزی است که موج‌سوار را از گیر افتادن در بن‌بست‌ها و تله‌های عنکبوتی نجات می‌دهد.

نمودار شماتیک: معماری انشعاب احتمال توسط Damping Factor



گام 3) بررسی سناریوی d = 1 (حذف عامل میرایی)

حال فرض کنید در سیستم تنظیم کنیم **d = 1**. از نظر ریاضی و سیستمی چه اتفاقی می‌افتد؟

1. صفر شدن احتمال جهش:

اگر **d = 1** باشد، عبارت $1 - d$ برابر با صفر خواهد شد. این یعنی احتمال **Teleportation** کاملاً از بین می‌رود و الگوریتم فاقد مکانیزم فرار می‌شود.

2. فروپاشی فرمول به مدل پایه:

فرمول **PageRank** به حالت ساده شده زیر تقلیل می یابد. این همان مدل پایه مارکوف است که پیش تر دیدیم:

$$PR(A) = \sum \frac{PR(T_i)}{C(T_i)}$$

3. پیامدهای مخرب در گراف وب:

اثر بن بست ها: چون هیچ جهش تصادفی وجود ندارد، موج سواری که به یک گره بن بست می رسد، متوقف می شود. به مرور زمان، «نشت احتمال» رخ داده و مجموع امتیازات تمام صفحات سیستم به صفر میل می کند.

اثر تله های عنکبوتی: اگر تله ای در شبکه وجود داشته باشد، موج سوار هرگز نمی تواند با تایپ آدرس جدید از آن فرار کند. تمام امتیاز **PageRank** شبکه مکیده شده و در گره های تله انباشته می شود.

عدم همگرایی ریاضی: از منظر جبر خطی، گراف دیگر «ارگودیک» (**Ergodic**) نخواهد بود. قضیه پرون-فروبنیوس (**Perron-Frobenius**) نقض شده و سیستم نمی تواند به یک توزیع ایستای (**Stationary Distribution**) یکتا و معتبر برای رتبه بندی صفحات برسد.

نکته آموزشی

چرا عدد 0.85؟

انتخاب عدد **d = 0.85** توسط گوگل تصادفی نبود. این عدد یک نقطه تعادل (**Trade-off**) مهندسی بسیار دقیق است:

اگر d خیلی کوچک باشد (مثلاً 0.5): اثر لینک ها و ساختار واقعی وب نادیده گرفته شده و رتبه بندی بیشتر بر اساس پرش های تصادفی (شانس) انجام می شود.

اگر d خیلی بزرگ باشد (مثلاً 0.99): الگوریتم برای همگرا شدن به تکرارهای (**Iterations**) بسیار زیادی نیاز دارد که هزینه پردازشی سرورها را به شدت بالا می برد و سیستم در برابر تله های عنکبوتی آسیب پذیرتر می شود.

مقدار 0.85 بهترین تعادل بین «دقت رتبه بندی» و «سرعت محاسبات ریاضی» را فراهم می کند.

پاسخنامه مسئله 21.9: رتبه صفحه موضوعی (Topic-Specific PageRank)

مقدمه و هدف مسئله

الگوریتم پیچرنک (**PageRank**) در حالت کلاسیک خود، یک ارزیابی «عمومی» و سراسری از اهمیت صفحات وب ارائه می دهد. با این حال، در کاربردهای واقعی، موتورهای جستجو غالباً نیاز دارند نتایج را بر اساس یک موضوع خاص (مثلاً پزشکی، ورزشی) یا حتی ترجیحات یک کاربر مشخص، شخصی سازی کنند.

این شخصی سازی از طریق الگوریتمی به نام **Topic-Specific PageRank** (که گاهی **Personalized PageRank** نیز نامیده می شود) انجام می گیرد. هدف این مسئله، درک تفاوت ریاضیاتی این دو روش در بخش «پرش تصادفی» (**Teleportation**) از فرمول است.

گام 1) مرور پرش تصادفی در PageRank کلاسیک (توزیع یکنواخت)

برای درک تفاوت، ابتدا باید به رفتار موجسوار در مدل کلاسیک نگاه کنیم. در **PageRank** استاندارد، زمانی که موجسوار از کلیک کردن روی لینکها خسته می‌شود، آدرس یک صفحه جدید را تایپ کرده و به آن پرش (جهش) می‌کند.

بردار جهش (Teleportation Vector): در این حالت، موجسوار با احتمال کاملاً برابر به هر یک از N صفحه موجود در کل وب می‌پرد.

معادله ریاضی: بخش مربوط به پرش تصادفی در فرمول کلاسیک به صورت $\frac{1-d}{N}$ نوشته می‌شود. در اینجا بردار جهش دارای توزیع یکنواخت (**Uniform Distribution**) است؛ یعنی برای هر صفحه i مقدار آن برابر است با $\frac{1}{N}$.

گام 2) معرفی مجموعه موضوعی (S)

در رویکرد **Topic-Specific**، ما ابتدا یک مجموعه از صفحات مرجع و مورد اعتماد (**Trusted/Seed Pages**) که دقیقاً مرتبط با موضوع هدف ما هستند را انتخاب می‌کنیم. این مجموعه را S (**Topic Set**) می‌نامیم. به عنوان مثال، اگر موضوع جستجو «ورزش» باشد، مجموعه S می‌تواند شامل سایت‌های **ESPN**، سایت رسمی فیفا و چند خبرگزاری معتبر ورزشی دیگر باشد.

گام 3) تغییر در معادله پرش تصادفی (توزیع سوگیرانه)

تفاوت اصلی و بنیادین دقیقاً در همین نقطه رخ می‌دهد: سوگیری در پرش تصادفی (**Teleportation Bias**). در الگوریتم موضوعی، موجسوار دیگر به صورت تصادفی به هر جایی در اینترنت نمی‌پرد؛ بلکه زمانی که تصمیم به جهش می‌گیرد، فقط و فقط به یکی از صفحات موجود در مجموعه مرجع (S) پرش می‌کند. احتمال پرش به صفحاتی که خارج از مجموعه S هستند دقیقاً برابر با 0 است.

معادله ریاضی جدید: با استفاده از بردار شخصی‌سازی (v_A)، فرمول اصلی به شکل زیر اصلاح و تکمیل می‌شود:

$$PR(A) = (1 - d) \cdot v_A + d \sum_{T_i \in M(A)} \frac{PR(T_i)}{C(T_i)}$$

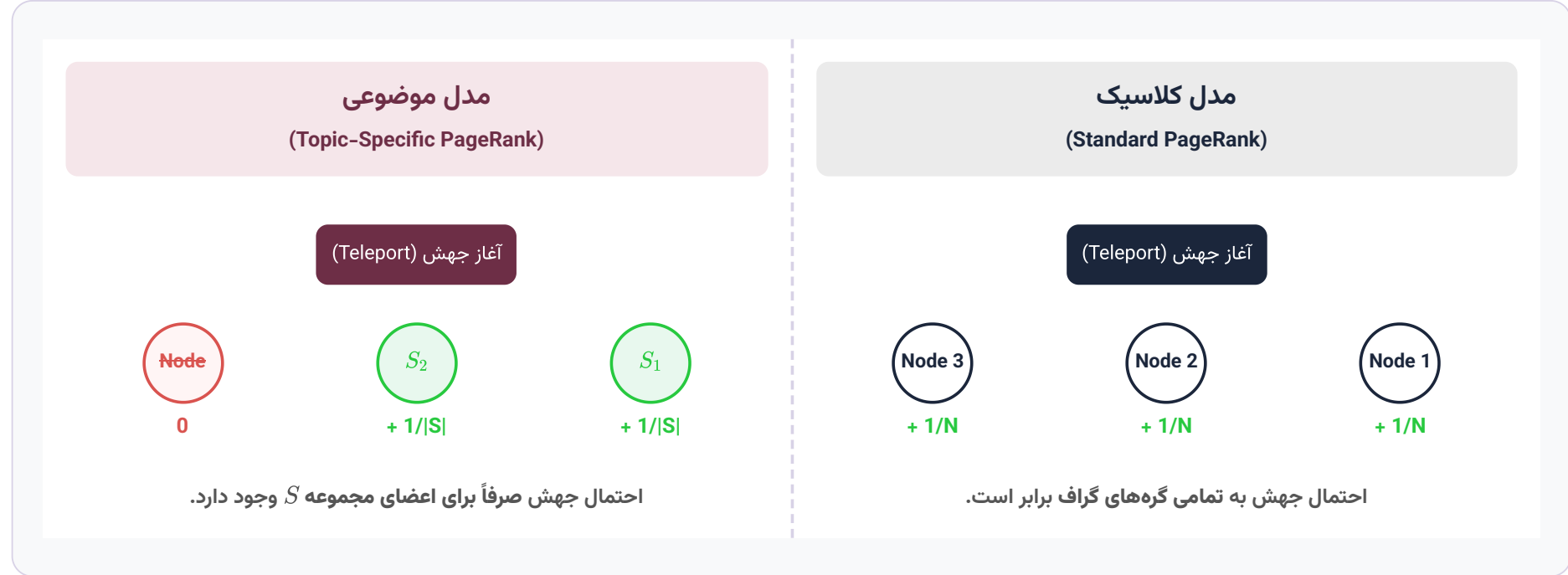
در این معادله، v_A (مقدار بردار شخصی‌سازی برای صفحه A) به صورت یک تابع چندضابطه‌ای تعریف می‌شود:

$$v_A = \begin{cases} \frac{1}{|S|} & \text{if } A \in S \\ 0 & \text{if } A \notin S \end{cases}$$

اگر صفحه A عضو مجموعه موضوعی S باشد ($A \in S$): مقدار $v_A = \frac{1}{|S|}$ است (که در آن $|S|$ تعداد صفحات داخل مجموعه S است).

اگر صفحه A عضو مجموعه S نباشد ($A \notin S$): مقدار $v_A = 0$ خواهد بود.

نمودار شماتیک: مقایسه بردار جهش در دو مدل رتبه‌بندی



گام 4) نتیجه ساختاری این تغییر

این تغییر ریاضی به ظاهر ساده، اثر موجی (**Ripple Effect**) بسیار قدرتمندی در کل گراف وب ایجاد می‌کند. از آنجا که موج‌سوار پس از هر بار پرش، سفر خود را دوباره از سایت‌های معتبر ورزشی (مجموعه S) آغاز می‌کند، صفحاتی که با لینک‌های کوتاه و مستقیم به این سایت‌ها متصل هستند، بیشترین ترافیک (احتمال) را دریافت می‌کنند. در نتیجه، رتبه (**PageRank**) صفحاتی که در همسایگی گرافیکی مجموعه S قرار دارند به شدت افزایش می‌یابد و صفحات نامرتبط به آن موضوع، افت رتبه شدیدی پیدا می‌کنند.

نکته آموزشی

تولد الگوریتم TrustRank:

تفاوت ریاضیاتی که در این مسئله آموختیم، پایه و اساس یکی از مهم‌ترین الگوریتم‌های ضد اسپم گوگل به نام **TrustRank** است. در **TrustRank**، مجموعه S شامل سایت‌هایی است که توسط انسان بررسی شده و 100% امن و معتبر تشخیص داده شده‌اند (مثل سایت‌های دانشگاهی یا دولتی).

الگوریتم با استفاده از فرمول **Topic-Specific PageRank**، رتبه اعتماد را محاسبه می‌کند؛ بنابراین هرچه یک سایت با لینک‌های کمتری به هسته سایت‌های مورد اعتماد متصل باشد، رتبه اعتماد کمتری دریافت کرده و احتمال اسپم بودن آن بیشتر ارزیابی می‌شود.

پاسخنامه مسئله 21.10: هاب و اتوریتی (Hubs and Authorities)

مقدمه و هدف مسئله

الگوریتم **HITS (Hyperlink-Induced Topic Search)** که توسط جان کلاینبِرگ (**Jon Kleinberg**) توسعه یافت، رویکرد متفاوتی نسبت به **PageRank** برای تحلیل گراف وب ارائه می‌دهد. این الگوریتم به جای اختصاص دادن یک امتیاز واحد به هر صفحه، 2 نقش (یا امتیاز) مجزا برای هر گره تعریف می‌کند. هدف این مسئله، شناخت این دو مفهوم کلیدی و درک رابطه ریاضیاتی و منطقی بین آن‌هاست.

گام 1) تعریف Authority (مرجع)

در الگوریتم **HITS**، یک صفحه **Authority** (مرجع یا منبع)، صفحه‌ای است که محتوای بسیار ارزشمند، دقیق و معتبری درباره یک موضوع خاص ارائه می‌دهد.

ویژگی بارز گرافیکی: این صفحات معمولاً دارای تعداد زیادی لینک ورودی (In-links) هستند.

مثال: در موضوع «برنامه‌نویسی»، وب‌سایت رسمی پایتون (**python.org**) یا **StackOverflow** صفحات **Authority** محسوب می‌شوند، زیرا هزاران سایت دیگر برای ارائه اطلاعات موثق به آن‌ها ارجاع داده‌اند.

گام 2) تعریف Hub (مرکز توزیع / راهنما)

یک صفحه **Hub**، صفحه‌ای است که لزوماً خودش محتوای اصلی یا دانش عمیقی تولید نمی‌کند، بلکه به عنوان یک **فهرست راهنما** یا **دایرکتوری باکیفیت** عمل کرده و کاربران را به سمت بهترین منابع (**Authorities**) هدایت می‌کند.

ویژگی بارز گرافیکی: این صفحات دارای تعداد زیادی لینک خروجی (Out-links) باکیفیت هستند.

مثال: یک پست وبلاگ با عنوان «معرفی 10 سایت برتر برای یادگیری پایتون» که شامل لینک‌هایی به منابع معتبر است، یک صفحه **Hub** به شمار می‌رود.

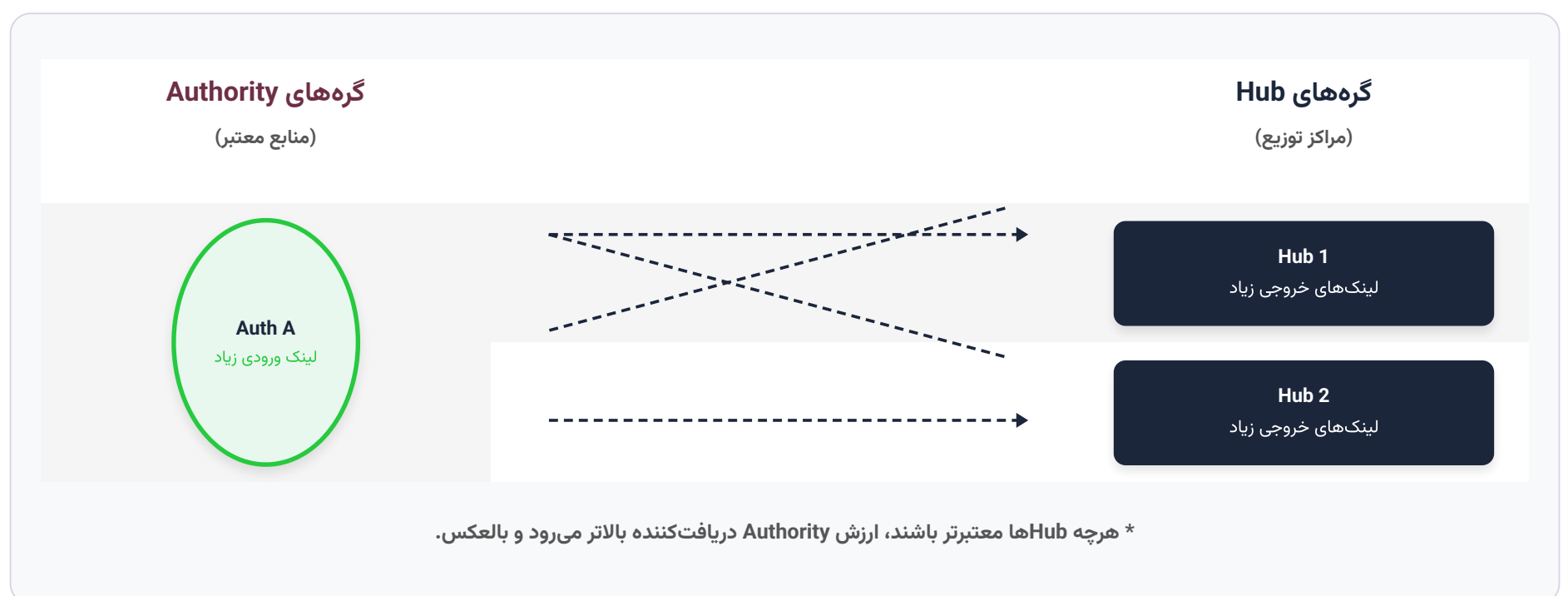
گام 3) ویژگی‌های یک صفحه Hub «خوب»

بر اساس منطق الگوریتم **HITS**، کیفیت یک **Hub** صرفاً به تعداد لینک‌های خروجی آن بستگی ندارد (در غیر این صورت هر صفحه اسپمی با هزاران لینک می‌توانست یک **Hub** عالی باشد). یک صفحه **Hub** خوب دارای ویژگی‌های زیر است:

1. **ارجاع به Authorities باکیفیت:** یک **Hub** خوب به صفحاتی لینک می‌دهد که خودشان **Authority**‌های بسیار قدرتمندی باشند.

2. **تمرکز موضوعی (Topical Focus):** لینک‌های خروجی آن باید انسجام موضوعی داشته باشند و کاربر را به منابع پراکنده و نامرتب هدایت نکنند.

نمودار شماتیک: ساختار دوقطبی Hub و Authority



گام 4) رابطه تقویت متقابل (Mutually Reinforcing Relationship)

هسته مرکزی ریاضیات در الگوریتم **HITS**، رابطه‌ای چرخشی و بازگشتی است که بین این دو مفهوم وجود دارد:

یک **Authority** خوب، صفحه‌ای است که توسط **Hub**‌های خوب زیادی به آن لینک داده شده باشد.

یک **Hub خوب**، صفحه‌ای است که به **Authority‌های خوب** زیادی لینک داده باشد.

این رابطه باعث می‌شود که امتیازات طی یک فرآیند تکراری (**Iterative**) محاسبه شوند. فرمول‌های به‌روزرسانی در هر مرحله به شکل زیر تکمیل و مدل‌سازی می‌شوند:

امتیاز Authority گره p : مجموع امتیاز **Hub** تمام گره‌هایی (q) که به p لینک داده‌اند.

$$A(p) = \sum_{q \rightarrow p} H(q)$$

امتیاز Hub گره p : مجموع امتیاز **Authority** تمام گره‌هایی (q) که p به آن‌ها لینک داده است.

$$H(p) = \sum_{p \rightarrow q} A(q)$$

(توجه: از منظر جبر خطی، اگر M ماتریس مجاورت باشد، این رابطه به صورت بردارهای $\vec{h} = M\vec{a}$ و $\vec{a} = M^T\vec{h}$ نوشته می‌شود. پس از هر مرحله محاسبه، امتیازات نرمال‌سازی (**Normalize**) می‌شوند تا از رشد بی‌نهایت مقادیر به سمت بی‌نهایت جلوگیری شود و سیستم به بردارهای ویژه اصلی همگرا گردد).

نکته آموزشی

تفاوت HITS با PageRank:

در حالی که **PageRank** معمولاً در زمان خزش وب (**Crawling**) و برای کل اینترنت پیش‌محاسبه (**Pre-computed**) می‌شود (به صورت **Query-Independent**)، الگوریتم **HITS** غالباً به صورت پویا و در زمان پاسخ به جستجوی کاربر اجرا می‌شود (**Query-Dependent**).

مکانیزم **HITS** به این شکل است که ابتدا یک زیرگراف (**Sub-graph**) کوچک از صفحاتی که کلمه کلیدی جستجو شده را دارند تشکیل داده (مجموعه ریشه یا **Root Set** و سپس **Base Set**) و سپس محاسبات **Hub** و **Authority** را فقط روی همان زیرگراف انجام می‌دهد. این امر **HITS** را برای تشخیص موضوعات تخصصی بسیار دقیق‌تر، اما از نظر پردازشی کندتر و پرهزینه‌تر از **PageRank** می‌کند.

پاسخنامه مسئله 21.11: مجموعه ریشه و پایه (Root Set and Base Set)

مقدمه و هدف مسئله

برخلاف الگوریتم **PageRank** که کل شبکه اینترنت را به صورت آفلاین پردازش می‌کند، الگوریتم **HITS** یک روش «وابسته به پرس‌وجو» (**Query-dependent**) است. این یعنی محاسبات گراف تنها روی بخش کوچکی از وب که مرتبط با جستجوی کاربر است، انجام می‌شود. هدف این مسئله، بررسی فاز اولیه یا «فاز مقداردهی» در الگوریتم **HITS** است؛ جایی که زیرگراف (**Sub-graph**) مورد نیاز برای محاسبات، طی 2 مرحله و با ساختن «مجموعه ریشه» و «مجموعه پایه» شکل می‌گیرد.

گام 1) تعریف مجموعه ریشه (Root Set)

هنگامی که کاربر یک کلمه یا عبارت را جستجو می‌کند، الگوریتم **HITS** ابتدا دست به دامن یک موتور جستجوی متنی کلاسیک (مبتنی بر مدل‌هایی مانند **TF-IDF** یا **Boolean**) می‌شود.

نحوه تشکیل: موتور جستجوی متنی، صفحات وب را بر اساس میزان تکرار و ارتباط کلمات کلیدی رتبه‌بندی می‌کند.

الگوریتم **HITS**، تعداد مشخصی از برترین نتایج این جستجو (مثلاً **t = 200** صفحه اول) را استخراج می‌کند.

مفهوم: به این مجموعه اولیه که صرفاً بر اساس «ارتباط متنی» (**Textual Relevance**) به دست آمده است، **مجموعه**

ریشه (**Root Set**) می‌گویند که آن را با R نشان می‌دهیم.

گام 2) چرا مجموعه ریشه به تنهایی کافی نیست؟

شاید این سوال پیش بیاید که چرا محاسبات **Hub** و **Authority** را مستقیماً روی همین مجموعه R انجام نمی‌دهیم؟ 2 دلیل ریاضی و ساختاری برای این محدودیت وجود دارد:

1. **فقدان مراجع معتبر (Authorities):** بسیاری از صفحات مرجع و بسیار معتبر، لزوماً شامل کلمه کلیدی جستجو

شده نیستند (به عنوان مثال، ممکن است در صفحه اصلی یک دانشگاه معتبر، نام یک رشته خاص به دفعات تکرار

نشده باشد، اما معتبرترین منبع برای آن باشد). بنابراین در نتایج اولیه متنی (مجموعه R) ظاهر نمی‌شوند.

2. **کمبود پیوندها (Sparsity):** صفحات موجود در مجموعه R صرفاً به دلیل داشتن کلمات مشابه دور هم جمع

شده‌اند و ممکن است در گراف واقعی وب، هیچ لینک خروجی یا ورودی به یکدیگر نداشته باشند. بدون وجود پیوند

بین صفحات، ساخت ماتریس مجاورت و محاسبه **Hub** و **Authority** غیرممکن است.

گام 3) نحوه ساخت مجموعه پایه (Base Set)

برای حل مشکل پراکندگی و وارد کردن مراجع واقعی به گراف، الگوریتم **HITS** مجموعه ریشه (R) را گسترش داده و **مجموعه پایه**

(**Base Set**) که آن را با S نشان می‌دهیم، می‌سازد. این گسترش گرافیکی در 3 مرحله انجام می‌شود:

1. **کپی اولیه:** در ابتدا تمام اعضای مجموعه ریشه به مجموعه پایه منتقل می‌شوند ($S = R$).

2. **افزودن لینک‌های خروجی (Out-links):** برای هر صفحه موجود در R ، تمام صفحاتی که از آن لینک دریافت کرده‌اند

(صفحاتی که R به آن‌ها اشاره می‌کند) به مجموعه S اضافه می‌شوند. (این کار **Authority**‌های احتمالی را وارد گراف

می‌کند).

3. **افزودن لینک‌های ورودی (In-links):** برای هر صفحه موجود در R ، صفحاتی که به آن لینک داده‌اند نیز به مجموعه

S اضافه می‌شوند. (این کار **Hub**‌های احتمالی را وارد گراف می‌کند).

این تجمیع گرافیکی را می‌توان به شکل دقیق در نظریه مجموعه‌ها با رابطه ریاضی زیر نشان داد:

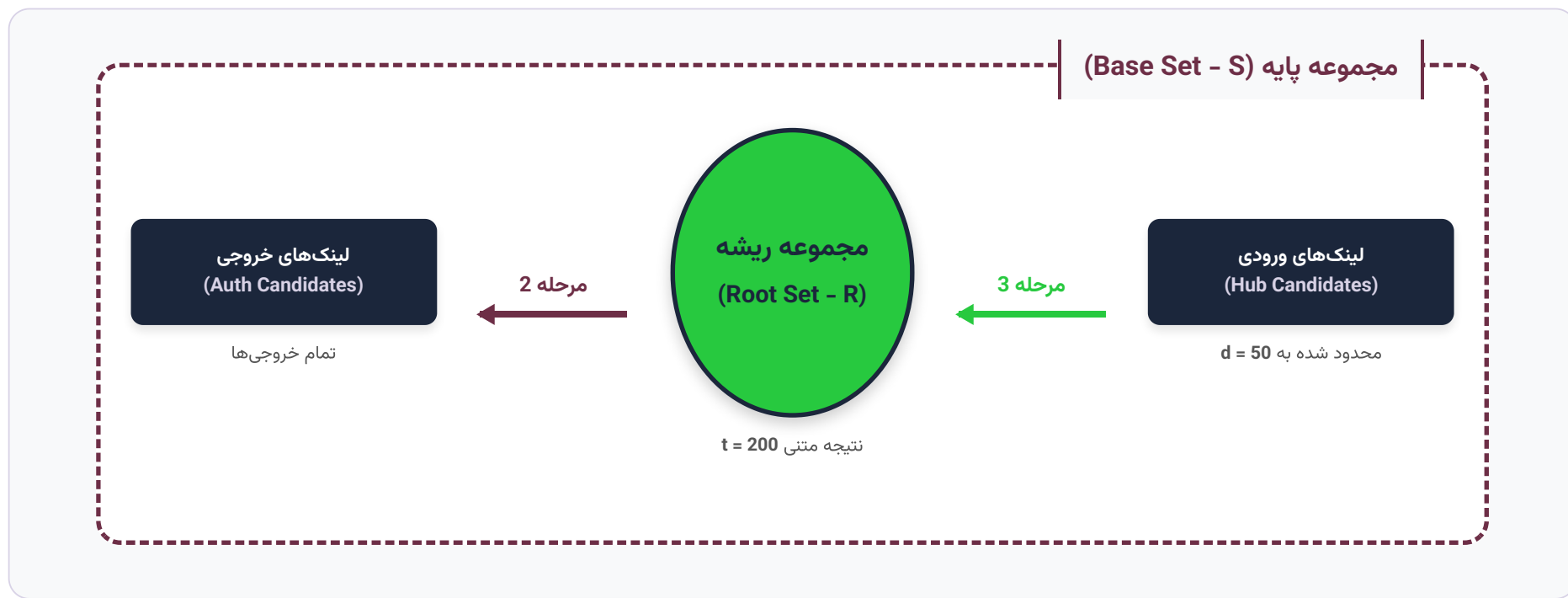
$$S = R \cup \{v \mid \exists u \in R : u \rightarrow v\} \cup \{v \mid \exists u \in R : v \rightarrow u\}$$

نکته: از آنجا که یک صفحه ممکن است هزاران لینک ورودی داشته باشد (مثل سایت‌های خبری بزرگ)، برای جلوگیری از بزرگ شدن بیش از حد

گراف و افت سرعت پردازش آنلین، معمولاً تعداد لینک‌های ورودی اضافه شده به یک سقف مجاز (مثلاً **d = 50** گره به ازای هر صفحه) محدود

می‌شود.

نمودار شماتیک: فرآیند بسط مجموعه ریشه به مجموعه پایه



گام 4) مقایسه ساختاری

جدول زیر تفاوت‌های کلیدی این دو مجموعه را از منظر سیستمی مقایسه می‌کند:

ویژگی	مجموعه ریشه (Root Set)	مجموعه پایه (Base Set)
معیار تشکیل	ارتباط متنی کلمات (Textual)	ساختار پیوندها (Topological)
اندازه (تعداد گره)	کوچک و ثابت (مثلاً 200 صفحه)	به مراتب بزرگتر (1000 تا 5000 صفحه)
تضمین وجود مرجع	ضعیف (ممکن است شامل Authority نباشد)	قوی (Authorities از طریق پیوندها وارد شده‌اند)
نقش در الگوریتم	نقطه شروع و هسته اولیه جستجو	بستر نهایی برای انجام محاسبات ماتریسی HITS

نکته آموزشی (Takeaway)

جداسازی متن از ساختار: الگوریتم HITS با این روش هوشمندانه، بهترین ویژگی‌های دو دنیای متفاوت را ترکیب کرده است.

استفاده از موتور متنی برای ساخت مجموعه ریشه، تضمین می‌کند که در موضوع مورد نظر (**Topic**) باقی مانده‌ایم؛ و استفاده از پیوندها برای ساخت مجموعه پایه، تضمین می‌کند که محاسبات رتبه‌بندی ما بر اساس «تأییدات ساختاری جامعه وب» انجام می‌شود، نه صرفاً ترفندهای کلمه‌سازی توسط نویسندگان سایت‌ها. این رویکرد، نتایج را هم از نظر موضوعی و هم از نظر اعتباری بهینه‌سازی می‌کند.

پاسخنامه مسئله 21.12: همگرایی در الگوریتم (Algorithm Convergence)

مقدمه و هدف مسئله

الگوریتم‌های رتبه‌بندی وب مانند **PageRank** و **HITS**، به دلیل ابعاد نجومی ماتریس‌های گراف وب، نمی‌توانند با روش‌های مستقیم جبر خطی (مانند محاسبه دترمینان یا معکوس ماتریس) حل شوند. به همین دلیل، این الگوریتم‌ها از «روش‌های

تکرارشونده» (Iterative Methods)، به طور خاص روش توان (**Power Iteration**)، استفاده می‌کنند. هدف این مسئله، درک مفهوم نقطه پایان این تکرارها (همگرایی) و بررسی رفتار تجربی این دو الگوریتم در رسیدن به پایداری است.

گام 1) تعریف مفهوم «همگرایی» در الگوریتم‌های تکرارشونده

در ابتدا، الگوریتم‌ها به تمام گره‌های گراف یک امتیاز اولیه (معمولاً برابر و یکنواخت، مثلاً $\frac{1}{N}$) اختصاص می‌دهند. سپس طی یک حلقه محاسباتی (**Iteration**)، امتیاز جدید هر گره بر اساس امتیاز همسایگانش در مرحله قبل محاسبه و به‌روزرسانی می‌شود.

همگرایی (Convergence) به نقطه‌ای از اجرای الگوریتم گفته می‌شود که در آن، انجام یک تکرار جدید، تغییر معناداری در امتیازات ایجاد نمی‌کند. به عبارت دیگر، سیستم به یک «حالت پایدار» (**Steady State**) رسیده و رتبه‌بندی صفحات تثبیت شده است. در این حالت، بردار توزیع احتمال در خروجی مرحله $t + 1$ دقیقاً (یا تقریباً) برابر با ورودی همان مرحله است.

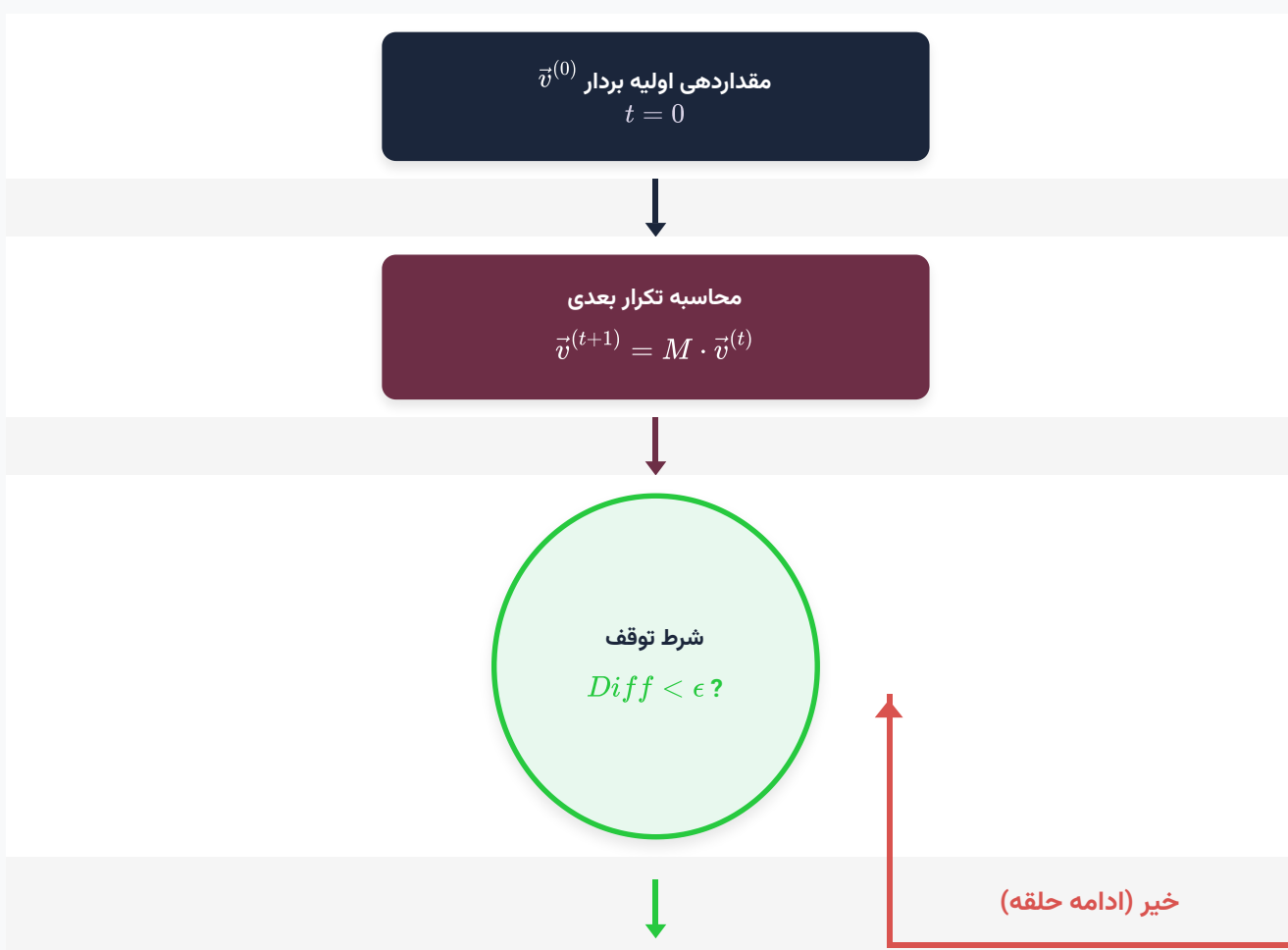
گام 2) شرط ریاضیاتی توقف (آستانه خطا - ϵ)

در پیاده‌سازی عملی، ما نمی‌توانیم تا بی‌نهایت الگوریتم را تکرار کنیم تا تغییرات مطلقاً صفر شود. بنابراین یک متغیر سیستمی به نام «آستانه خطا» یا تولرانس (ϵ) تعریف می‌کنیم (مثلاً 10^{-5}). شرط همگرایی از نظر ریاضی این است که نرم تفاضل (قدر مطلق تغییرات) بردار امتیازات در مرحله t و مرحله بعدی ($t + 1$)، از این مقدار آستانه کمتر شود. این شرط با فرمول ریاضی زیر (معمولاً با استفاده از نرم L_1 یا L_∞) ارزیابی می‌شود:

$$\|\vec{PR}^{(t+1)} - \vec{PR}^{(t)}\| \leq \epsilon$$

وقتی این شرط برقرار شد، می‌گوییم الگوریتم همگرا شده و محاسبات ماتریسی متوقف می‌گردد.

نمودار شماتیک: چرخه حیات و همگرایی در روش توان (Power Iteration)



گام 3) بررسی تجربی تعداد تکرارها در PageRank

الگوریتم **PageRank** روی گراف کل اینترنت (شامل میلیاردها گره) اجرا می‌شود. با وجود این ابعاد عظیم، **PageRank** به شکل شگفت‌انگیزی سریع همگرا می‌شود:

تعداد تکرار تجربی: در عمل، موتورهای جستجو معمولاً بین 50 تا 100 تکرار (**Iteration**) به همگرایی مطلوب می‌رسند (عدد 50 تا 75 بسیار رایج است).

دلیل ریاضی: سرعت همگرایی در **PageRank** وابستگی مستقیمی به پارامتر **Damping Factor** (α) دارد. از نظر جبر خطی، سرعت همگرایی (کاهش خطا در هر مرحله) متناسب با دومین مقدار ویژه بزرگ ماتریس ($|\lambda_2|$) است که ثابت می‌شود محدود به کران $|\lambda_2| \leq \alpha$ است. انتخاب $\alpha = 0.85$ تضمین می‌کند که الگوریتم به صورت لگاریتمی $O(\log N)$ و با سرعت بالا به جواب نهایی همگرا شود (در هر گام خطا تقریباً با ضریب 0.85 کاهش می‌یابد). اگر α به عدد 1 نزدیک شود، تعداد تکرارهای مورد نیاز به شدت افزایش می‌یابد.

گام 4) بررسی تجربی تعداد تکرارها در HITS

شرایط اجرای الگوریتم **HITS** کاملاً متفاوت است، زیرا این الگوریتم **Query-dependent** (وابسته به پرس‌وجو) است و تنها روی یک زیرگراف کوچک اجرا می‌شود.

ابعاد گراف: محاسبات فقط روی «مجموعه پایه» (**Base Set**) انجام می‌شود که معمولاً بین 1000 تا 5000 گره دارد.

تعداد تکرار تجربی: به دلیل کوچک بودن ابعاد گراف هدف، **HITS** بسیار سریع‌تر به پایداری می‌رسد و معمولاً تنها به 10 تا 20 تکرار برای همگرایی کامل بردارهای **Hub** و **Authority** نیاز دارد.

ویژگی مقایسه‌ای	الگوریتم PageRank	الگوریتم HITS
میانگین تکرار تا همگرایی	50 الی 100 تکرار	10 الی 20 تکرار
ابعاد فضای پردازشی	کل وب (میلیاردها گره)	مجموعه پایه (هزاران گره)
عامل کلیدی در سرعت	عامل میرایی (α)	کوچک بودن اندازه ماتریس

نکته آموزشی

مفهوم «بردار ویژه اصلی» (Principal Eigenvector):

در پس‌زمینه تمام این تکرارها، الگوریتم‌ها در حال حل یک مسئله بنیادی جبر خطی هستند. زمانی که می‌گوییم الگوریتم همگرا شده است، از نظر ریاضی به این معناست که بردار امتیازات نهایی صفحات، دقیقاً همان «بردار ویژه اصلی» (بردار ویژه متناظر با بزرگترین مقدار ویژه که برابر 1 است) برای ماتریس گذار (در **PageRank**) یا ماتریس مجاورت (در **HITS**) است.

در واقع، این الگوریتم‌ها (روش توان) یک ابزار و روش عددی (**Numerical Method**) بسیار کارآمد برای تقریب زدن و محاسبه این بردار ویژه در گراف‌های گول‌پیکر محسوب می‌شوند، جایی که محاسبات جبری دقیق غیرممکن است.

پاسخنامه مسئله 21.13: سناریوی تغییرات متن لینک (Anchor Text Variations Scenario)

مقدمه و هدف مسئله

در مباحث پیشرفته بازیابی اطلاعات (**IR**)، متن پیوند (**Anchor Text**) یکی از قوی‌ترین سیگنال‌ها برای درک محتوای صفحه مقصد است. با این حال، کاربران و طراحان وب همیشه از کلمات توصیفی و دقیق (مانند «گوشی ارزان») استفاده نمی‌کنند و غالباً عبارات عمومی و دستوری (مانند «کلیک کنید»، «اینجا»، «ادامه مطلب») را به کار می‌برند.

هدف این مسئله، طراحی یک معماری الگوریتمی از دیدگاه مهندسی جستجو است تا اثر این عبارات عمومی (به عنوان نویز اطلاعاتی) کاهش یافته و ارزش عبارات تخصصی حفظ شود.

گام 1) رویکرد آماری با استفاده از مدل TF-IDF (محاسبه فراوانی)

اولین و منطقی‌ترین خط دفاعی یک موتور جستجو در برابر کلمات عمومی، استفاده از منطق ریاضی نهفته در فرمول TF-IDF (به‌ویژه بخش **IDF**) است.

مفهوم IDF (Inverse Document Frequency): این معیار نشان می‌دهد که یک کلمه چقدر در کل شبکه وب نادر و متمایز است.

سناریوی اجرا: کلماتی مانند «خرید موبایل» تنها در لینک‌هایی که به فروشگاه‌های الکترونیکی ختم می‌شوند یافت می‌شوند، بنابراین **IDF** بالایی دارند. اما عبارت «کلیک کنید» در میلیون‌ها صفحه وب با موضوعات کاملاً نامرتبط (از سایت‌های پزشکی تا وبلاگ‌های آشپزی) برای لینک دادن استفاده می‌شود.

نتیجه‌گیری ریاضی: چون فراوانی داکيومنت (**Document Frequency**) برای عبارت «کلیک کنید» به شدت بالاست و تقریباً به اندازه کل صفحات وب (N) میل می‌کند، مقدار **IDF** آن در الگوریتم بر اساس فرمول لگاریتمی به سمت 0 میل می‌کند:

$$IDF = \log\left(\frac{N}{DF}\right) \approx \log(1) = 0$$

بنابراین، موتور جستجو به صورت خودکار وزن این عبارت را در ماتریس رتبه‌بندی خنثی می‌کند.

گام 2) فیلترینگ با لیست توقف پیوندها (Anchor Stop-words List)

علاوه بر مدل‌های آماری، مهندسان جستجو از روش‌های اکتشافی (**Heuristic**) نیز استفاده می‌کنند.

سناریوی اجرا: یک دیکشنری تخصصی از کلمات توقف (**Stop-words**) مختص به لینک‌ها ساخته می‌شود. این لیست شامل کلماتی است که صرفاً «فعل حرکتی» یا «ارجاعی» هستند (مانند: اینجا، کلیک، ورود، وبسایت، لینک).

اقدام الگوریتم: هر زمان که خزنده‌های وب (**Crawlers**) یک **Anchor Text** را استخراج می‌کنند، ابتدا آن را با این دیکشنری تطبیق می‌دهند. اگر کل عبارت در این لیست قرار داشت، وزن مستقیم آن به عنوان کلمه کلیدی صراحتاً 0 در نظر گرفته می‌شود.

گام 3) تحلیل پنجره متنی مجاور (Surrounding Text Context)

حذف ارزش عبارت «کلیک کنید» یک مشکل جدید ایجاد می‌کند: موتور جستجو سیگنال محتوایی آن لینک را به طور کامل از دست می‌دهد! برای جبران این موضوع، مهندسان از تکنیک بسیار کارآمد «تحلیل مجاورت» استفاده می‌کنند.

سناریوی اجرا: زمانی که الگوریتم تشخیص می‌دهد **Anchor Text** به دلیل عمومی بودن بی‌ارزش است، تمرکز خود را به N کلمه قبل و بعد از لینک (**Text Window**) معطوف می‌کند.

مثال عملی: فرض کنید در صفحه مبدأ کد زیر نوشته شده است:

"برای مشاهده مشخصات و خرید گوشی هوشمند اقتصادی، [اینجا کلیک کنید]."

الگوریتم کلمات "[اینجا کلیک کنید]" را بی‌اثر می‌کند، اما عبارات مجاور آن یعنی «**خرید گوشی هوشمند اقتصادی**» را استخراج کرده و به عنوان متن لینک مجازی (**Virtual Anchor Text**) برای صفحه مقصد در نظر می‌گیرد.

نمودار شماتیک: معماری استخراج متن مجاور (Context Windowing)



گام 4) خوشه‌بندی معنایی و مدل‌های زبانی (Semantic Clustering)

در موتورهای جستجوی مدرن، از مدل‌های پیشرفته پردازش زبان طبیعی (**NLP**) و تعبیه‌سازی بردارهای کلمه (**Word Embeddings**) استفاده می‌شود.

سناریوی اجرا: الگوریتم، کلمات «خرید موبایل» و «گوشی ارزان» را در فضای برداری به عنوان مفاهیمی بسیار نزدیک به یکدیگر در خوشه (**Cluster**) «تجهیزات الکترونیک/تلفن همراه» قرار می‌دهد.

عبارت «کلیک کنید» در هیچ خوشه موضوعی خاصی قرار نمی‌گیرد، زیرا با مفاهیم بسیار پراکنده‌ای هم‌نشینی دارد. در نتیجه، الگوریتم به صورت معنایی (**Semantic**) متوجه می‌شود که این پیوند حامل بار اطلاعاتی تخصصی برای صفحه فروشگاه نیست و تنها ارزش ساختاری (انتقال **PageRank**) دارد، نه ارزش محتوایی (**Relevance**).

نکته آموزشی (Takeaway)

تفاوت ارزش ساختاری و ارزش محتوایی:

دانشجویان باید دقت کنند که وقتی می‌گوییم وزن لینک «کلیک کنید» کاهش می‌یابد، منظورمان صرفاً ارزش محتوایی

از نظر ساختاری و در محاسبات گراف (مانند الگوریتم **PageRank**)، این لینک همچنان یک «یال» معتبر محسوب شده و اعتبار (**Authority**) صفحه مبدأ را به صورت ریاضی به صفحه مقصد (مثلاً «گوشی هوشمند») منتقل می‌کند؛ تنها تفاوت این است که موتور جستجو آن صفحه را برای کلمه کلیدی عمومی "کلیک کنید" در نتایج رتبه‌بندی نخواهد کرد.

پاسخنامه مسئله 21.14: محاسبه حالت پایدار در گراف کوچک

(Steady-State Calculation Scenario)

مقدمه و هدف مسئله

در شبکه‌های کوچک، می‌توانیم به جای استفاده از روش‌های تکرارشونده (**Iterative**) که برای میلیاردها صفحه استفاده می‌شود، مستقیماً از تکنیک‌های قطعی جبر خطی برای یافتن توزیع احتمال حالت پایدار (**Steady-State**) استفاده کنیم. هدف این مسئله، تبدیل ساختار گرافیکی یک شبکه وب کوچک به یک «دستگاه معادلات خطی» بر اساس فرمول استاندارد **PageRank** است.

گام 1) استخراج پارامترهای گراف (تحلیل درجات)

ابتدا باید گراف را از نظر تعداد گره‌ها (N)، لینک‌های خروجی (**Out-degree**) و لینک‌های ورودی (**In-links**) به صورت دقیق تحلیل کنیم:

تعداد کل صفحات: $N = 3$

بررسی صفحه A:

لینک‌های خروجی: به **B** و **C** می‌رود. $\rightarrow Out(A) = 2$

لینک‌های ورودی: فقط از **C** دریافت می‌کند.

بررسی صفحه B:

لینک‌های خروجی: فقط به **C** می‌رود. $\rightarrow Out(B) = 1$

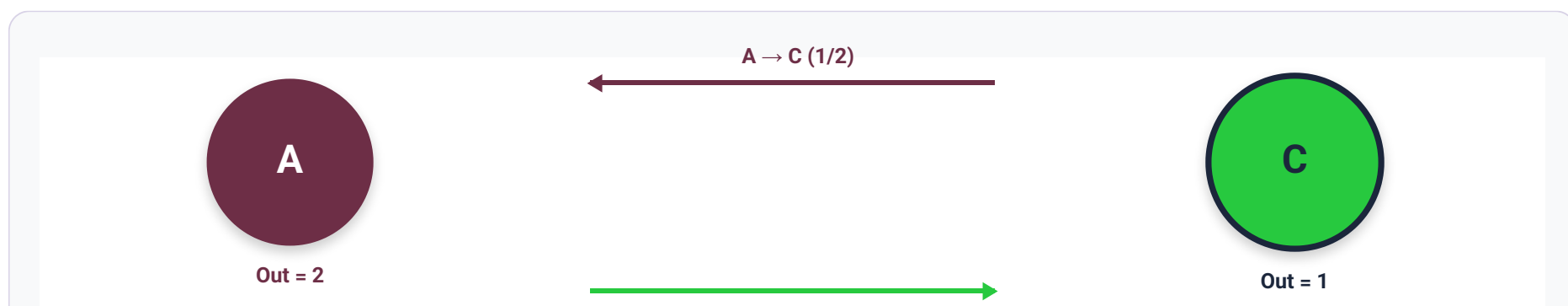
لینک‌های ورودی: فقط از **A** دریافت می‌کند.

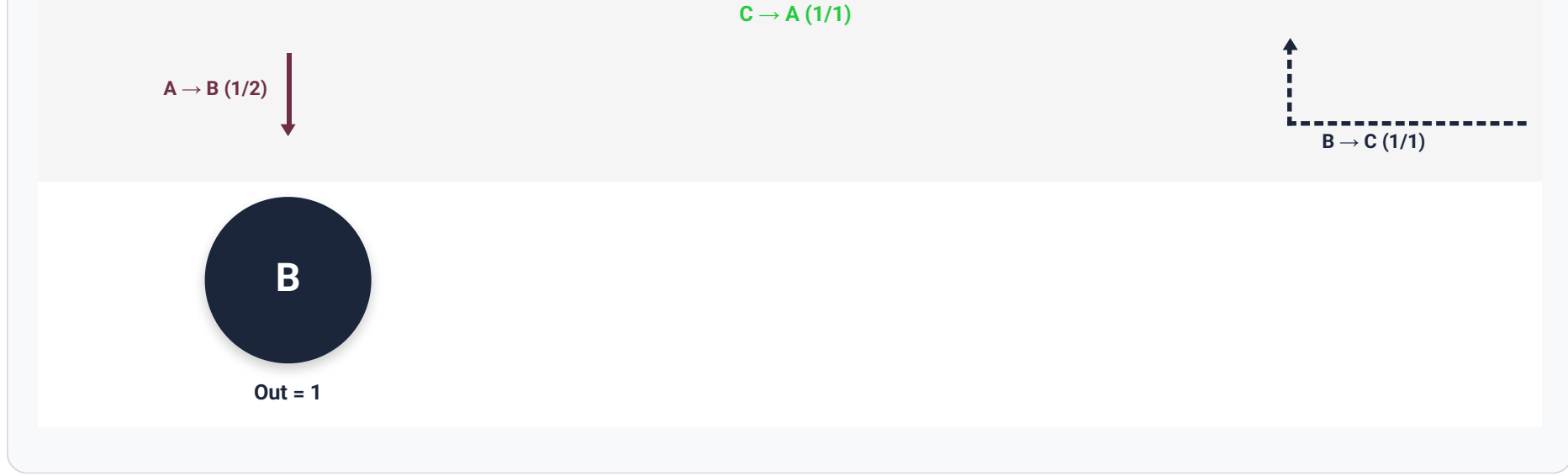
بررسی صفحه C:

لینک‌های خروجی: فقط به **A** می‌رود. $\rightarrow Out(C) = 1$

لینک‌های ورودی: از **A** و **B** دریافت می‌کند.

نمودار شماتیک: نقشه‌برداری توپولوژیک و جریان اعتبار گراف





گام 2) تعیین مقادیر ثابت در فرمول PageRank

فرمول استاندارد PageRank برای یک گره مانند i با احتساب عامل میرایی و جهش تصادفی به صورت زیر مدل سازی می شود:

$$PR(i) = \frac{1 - d}{N} + d \sum_{j \in M(i)} \frac{PR(j)}{Out(j)}$$

بر اساس صورت مسئله، مقادیر ثابت عبارتند از:

عامل میرایی (Damping Factor): $d = 0.85$

احتمال جهش تصادفی (بخش ثابت فرمول):

$$\frac{1 - d}{N} = \frac{1 - 0.85}{3} = \frac{0.15}{3} = 0.05$$

گام 3) تشکیل دستگاه معادلات خطی

اکنون با جایگذاری پارامترهای استخراج شده در فرمول کلی، معادله اختصاصی هر صفحه را استخراج می کنیم:

1. معادله صفحه A:

تنها گره ای که به A لینک داده، گره C است (با 1 لینک خروجی).

$$PR(A) = 0.05 + 0.85 \times \frac{PR(C)}{1}$$

2. معادله صفحه B:

تنها گره ای که به B لینک داده، گره A است (با 2 لینک خروجی).

$$PR(B) = 0.05 + 0.85 \times \frac{PR(A)}{2}$$

3. معادله صفحه C:

گره های A (با 2 لینک خروجی) و B (با 1 لینک خروجی) به C لینک داده اند.

$$PR(C) = 0.05 + 0.85 \times \left(\frac{PR(A)}{2} + \frac{PR(B)}{1} \right)$$

گام 4) نمایش نهایی دستگاه معادلات

برای حل این مسئله از طریق روش‌های جبری (مانند جایگذاری خطی، روش حذفی گاوس یا ماتریس معکوس)، دستگاه معادلات استخراج‌شده را به فرمت استاندارد متغیرها در یک سمت و مقادیر ثابت در سمت دیگر مرتب می‌کنیم:

$$\begin{cases} PR(A) - 0.85PR(C) = 0.05 \\ -0.425PR(A) + PR(B) = 0.05 \\ -0.425PR(A) - 0.85PR(B) + PR(C) = 0.05 \end{cases}$$

نکته آموزشی

تضمین صحت محاسبات (Validation):

در هر سیستم **PageRank**، از آنجا که توزیع امتیازات بر اساس احتمالات است، همواره یک قانون پنهان (معادله کمکی شرطی) وجود دارد که می‌تواند برای بررسی صحت جواب‌های نهایی استفاده شود: **مجموع امتیازات تمام صفحات گراف باید دقیقاً برابر با 1 باشد.**

بنابراین، پس از حل این دستگاه 3 معادله و 3 مجهول، باید رابطه خطی زیر به صورت قطعی برقرار باشد:

$$PR(A) + PR(B) + PR(C) = 1$$

این ویژگی کلیدی به ما اجازه می‌دهد در سیستم‌های به مراتب بزرگتر، پاسخ‌های به دست آمده از نرم‌افزار را اعتبارسنجی (**Validate**) کرده و نشت احتمال در سیستم را رصد کنیم.

پاسخنامه مسئله 21.15: سناریوی تغییر عامل میرایی (Damping Factor Variation)

مقدمه و هدف مسئله

الگوریتم **PageRank** در ابتدا برای شبکه جهانی وب (**WWW**) با پارامتر استاندارد **d = 0.85** طراحی شد. اما شبکه‌های سازمانی (**Intranet**) ساختار توپولوژیک و الگوی کاربری کاملاً متفاوتی دارند. هدف این مسئله، تحلیل ریاضی تأثیر طول مسیر وب‌گردی بر روی انتخاب مقدار بهینه برای پارامتر «عامل میرایی» (**Damping Factor**) است.

گام 1) تحلیل رفتار «موج‌سوار تصادفی» در اینترنت

در شبکه جهانی وب، کاربران غالباً زمان زیادی را صرف گشت‌وگذار می‌کنند و از طریق لینک‌ها به اعماق سایت‌های مختلف می‌روند. اما در یک اینترنت سازمانی، هدف کاربر معمولاً یافتن یک سند خاص (مثل یک بخشنامه، فرم مرخصی یا آیین‌نامه) در سریع‌ترین زمان ممکن است.

در این محیط محدود، کاربر یک جستجو انجام می‌دهد و نهایتاً روی یک یا دو لینک کلیک می‌کند؛ اگر به جواب نرسید، مسیر را در عمق ادامه نمی‌دهد، بلکه بلافاصله به صفحه اصلی یا نوار جستجو بازمی‌گردد تا کلمه کلیدی خود را تغییر دهد. این بازگشت به نقطه شروع یا جستجوی مجدد، در مدل زنجیره مارکوف دقیقاً معادل «جهش تصادفی» (Teleportation) است.

گام 2) رابطه ریاضی طول مسیر و عامل میرایی

در مدل موج‌سوار تصادفی، متغیر d نمایانگر احتمال کلیک روی یک لینک (ادامه مسیر) و متغیر $1 - d$ نمایانگر احتمال جهش (خاتمه مسیر فعلی و پرش تصادفی) است. از نظر علم آمار و توزیع هندسی (Geometric Distribution)، امید ریاضی یا میانگین طول مسیر وب‌گردی ($E[L]$) که نشان‌دهنده تعداد کلیک‌های متوالی قبل از وقوع یک پرش است، با فرمول زیر محاسبه می‌شود:

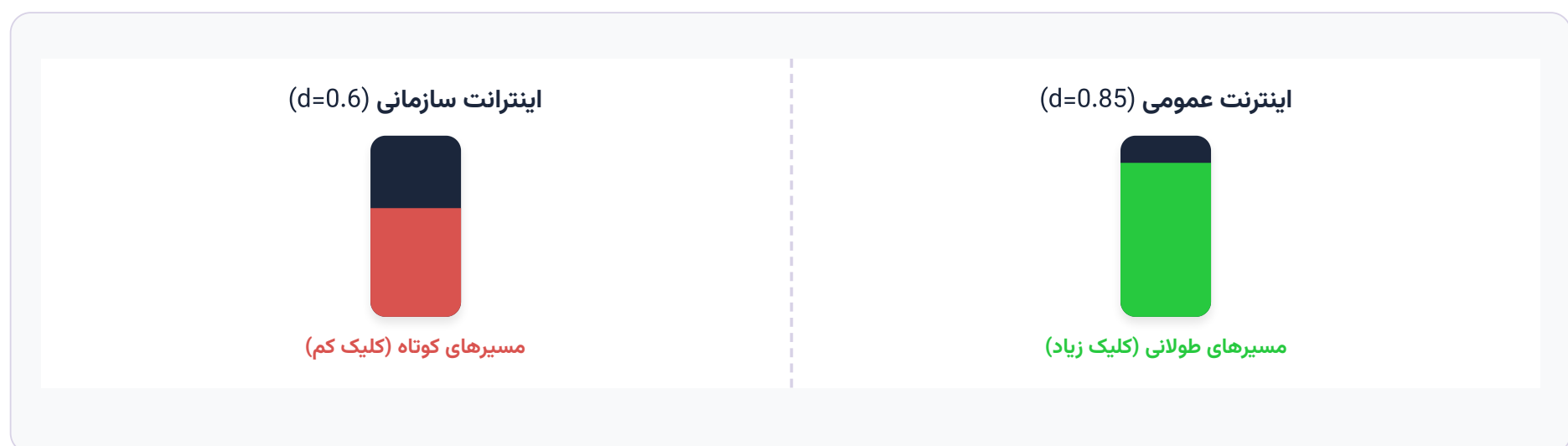
$$E[L] = \frac{d}{1 - d}$$

در وب جهانی با مقدار استاندارد $d = 0.85$ ، میانگین طول مسیر موج‌سوار برابر است با:

$$E[L] = \frac{0.85}{1 - 0.85} = \frac{0.85}{0.15} \approx 5.66$$

این یعنی در اینترنت عمومی، انتظار می‌رود کاربر پیش از خسته شدن و تایپ آدرس جدید، حدود 6 تا 7 لینک متوالی را دنبال کند.

نمودار شماتیک: تأثیر تغییر d بر رفتار کاربر



گام 3) تصمیم‌گیری مهندسی (محاسبه مقدار بهینه d)

بر اساس صورت مسئله، طول مسیر وب‌گردی در اینترنت «بسیار کوتاه‌تر» است. فرض کنید تحلیل‌های آماری (Log Analysis) سازمان نشان می‌دهد که کاربران به طور میانگین تنها 3 کلیک متوالی انجام می‌دهند ($E[L] = 3$). با قرار دادن این مقدار در معادله، مقدار d بهینه محاسبه می‌شود:

$$3 = \frac{d}{1 - d} \implies 3(1 - d) = d \implies 3 - 3d = d \implies 4d = 3 \implies d = 0.75$$

نتیجه‌گیری نهایی: به عنوان یک مهندس جستجو، شما باید پارامتر میرایی (d) را نسبت به مقدار استاندارد **0.85 کاهش** دهید (مثلاً روی **0.75** تنظیم کنید). کاهش d بازتاب‌دهنده این واقعیت رفتاری است که احتمال انصراف از کلیک و پرش تصادفی در اینترانت به مراتب بیشتر از وب آزاد است.

گام 4) پیامدهای الگوریتمی کاهش عامل میرایی

1. **کاهش شعاع نفوذ لینک‌ها:** اعتبار (**PageRank**) یک صفحه مرجع، کمتر به همسایگان دورتر (درجه 2 و 3) منتقل

می‌شود. در نتیجه، ارزش لینک‌های مستقیم (**Direct Links**) افزایش می‌یابد.

2. **افزایش وزن جهش تصادفی:** عبارت $\frac{1-d}{N}$ در فرمول **PageRank** مقدار بزرگتری به خود می‌گیرد. این بدان معناست

که ساختار گرافیکی اهمیت نسبی خود را تا حدودی از دست داده و پایگاه داده مسطح‌تر (**Flatter**) ارزیابی می‌شود.

3. **تسریع سرعت همگرایی:** از منظر جبر خطی، هرچه مقدار d کوچکتر باشد، الگوریتم تکرارشونده با سرعت بسیار

بیشتری به توزیع ایستای نهایی همگرا می‌شود و هزینه پردازشی سرور به شدت کاهش می‌یابد.

نکته آموزشی

هیچ پارامتری مقدس نیست!

این مسئله یک درس بنیادی در طراحی موتورهای جستجو به همراه دارد: مقادیر استاندارد مانند $d = 0.85$ صرفاً اعدادی تجربی برای یک پلتفرم خاص هستند. یک مهندس علوم کامپیوتر باید به جای حفظ کردن این اعداد، منطق ریاضی پشت آن‌ها را درک کند تا بتواند پارامترهای سیستم را با واقعیت‌های آماری، الگوهای رفتاری کاربران و توپولوژی همان شبکه خاص کالیبره و تنظیم نماید.

پاسخنامه مسئله 21.16: تحلیل ماتریس تله‌پورت (Teleportation Matrix Analysis)

مقدمه و هدف مسئله

یکی از زیباترین پیوندهای بین «نظریه گراف» و «جبر خطی» در معماری موتورهای جستجو، نحوه حل مشکل انزوای صفحات وب است. گراف واقعی اینترنت یکپارچه نیست و دارای جزایر منزوی، بن‌بست‌ها و تله‌هاست. این گسستگی باعث می‌شود سیستم نتواند به یک رتبه‌بندی واحد برسد. هدف این مسئله، اثبات این موضوع است که چگونه ترفند ریاضی «تله‌پورت» (**Teleportation**)، گراف ازهم‌گسیخته وب را به یک ساختار بی‌نقص ریاضی تبدیل کرده و ضامن همگرایی الگوریتم **PageRank** می‌شود.

گام 1) مشکل گراف اولیه (ماتریس گذار پایه)

ماتریس گذار پایه (P) که تنها بر اساس لینک‌های واقعی وب ساخته می‌شود، دارای درایه‌های (عناصر) صفر بسیار زیادی است. اگر از صفحه i لینکی به صفحه j وجود نداشته باشد، آنگاه $P_{ij} = 0$ خواهد بود.

به دلیل وجود این صفرهای بی‌شمار، گراف وب به بخش‌های مجزا تقسیم می‌شود. کاربری که در یک جزیره از وب‌سایت‌ها قرار دارد، ممکن است هیچ مسیری برای رسیدن به بخش دیگری از اینترنت نداشته باشد. در زبان زنجیره مارکوف، این یعنی سیستم **تقلیل‌پذیر (Reducible)** است و نمی‌تواند به یک توزیع پایدار سراسری برسد.

گام 2) معرفی ماتریس گوگل (تلفیق با بردار تله پورت)

برای حل این مشکل، ماتریس گذار پایه (P) با یک ماتریس پرش تصادفی یا تله پورت ترکیب می شود تا ماتریس گوگل (G) شکل بگیرد. فرمول ریاضی و کامل ساخت این ماتریس به شکل زیر است:

$$G = d \cdot P + \frac{1-d}{N} \cdot E$$

در این معادله:

P: ماتریس گذار پایه (بر اساس لینک های واقعی).

d: عامل میرایی (Damping Factor).

N: تعداد کل صفحات گراف.

E: یک ماتریس $N \times N$ که تمامی درایه های آن عدد 1 است.

گام 3) اثبات تبدیل به گراف کاملاً متصل (Fully Connected)

برای اینکه نشان دهیم گراف کاملاً متصل شده است، باید ثابت کنیم از هر گره ای به هر گره دیگر (حتی به خودش) یک یال جهت دار با احتمال غیرصفر وجود دارد. به هر درایه دلخواه از ماتریس گوگل (G_{ij}) نگاه می کنیم:

$$G_{ij} = d \cdot P_{ij} + \frac{1-d}{N}$$

حتی اگر در گراف واقعی هیچ لینکی بین صفحه i و صفحه j وجود نداشته باشد ($P_{ij} = 0$)، باز هم مقدار درایه G_{ij} صفر نخواهد شد، زیرا معادله به شکل زیر ساده می شود:

$$G_{ij} = (d \times 0) + \frac{1-d}{N} = \frac{1-d}{N} > 0$$

از آنجا که عامل میرایی d اکیداً کوچکتر از 1 است (مثلاً 0.85)، عبارت $\frac{1-d}{N}$ همواره یک عدد مثبت (بزرگتر از 0) خواهد بود.

نتیجه گیری ساختاری: چون تمام درایه های ماتریس G اکیداً مثبت هستند ($G_{ij} > 0$)، یعنی احتمال حرکت از هر گره

به هر گره دیگری در سیستم وجود دارد. در نظریه گراف، ماتریسی که هیچ درایه صفری نداشته باشد، معادل یک **گراف**

کاملاً متصل (یک گراف کامل جهت دار) است که در آن بین هر دو گره دلخواه، حداقل یک یال مجازی وجود دارد.

نمودار شماتیک: تکامل ریاضیاتی ماتریس گذار به ماتریس گوگل

ماتریس گوگل (G)

0.05	0.05	0.05
0.05	0.05	0.05
0.05	0.05	0.05

=

ماتریس پرش ($\frac{1-d}{N} E$)

0.05	0.05	0.05
0.05	0.05	0.05
0.05	0.05	0.05

+

ماتریس پایه (P)

1/2	1/2	0
0	0	1
0	0	0

گام 4) حل مشکل ارگودیک نبودن (Non-ergodicity)

در زنجیره های مارکوف، برای اینکه سیستم به یک حالت پایدار یکتا (**Unique Steady-State**) برسد، زنجیره باید ارگودیک (**Ergodic**) باشد. یک زنجیره مارکوف زمانی ارگودیک است که 2 شرط زیر را همزمان داشته باشد:

1. تقلیل ناپذیری (Irreducibility):

یعنی امکان رسیدن از هر حالت به هر حالت دیگری وجود داشته باشد. همان طور که در گام سوم اثبات شد، چون گراف به یک گراف کاملاً متصل تبدیل شده، موج سوار می تواند (با احتمال تله پورت) از هر صفحه ای به هر صفحه دیگری برود. پس شرط اول برقرار است.

2. نامتناوب بودن (Aperiodicity):

تناوب یعنی موج سوار در یک حلقه گیر کند و فقط در مضاربی از یک زمان خاص (مثلاً گام های زوج) به یک گره مشخص برسد. بردار تله پورت این مشکل را نیز حل می کند. از آنجا که $G_{ii} > 0$ (احتمال پرش موج سوار از صفحه i به خود همان صفحه i بزرگتر از صفر است)، هر گره در این گراف دارای یک «حلقه به خود» (**Self-loop**) است. وجود حتی یک **Self-loop** در گراف متصل، تمام تناوب های ممکن را در هم می شکند و گراف را کاملاً نامتناوب می کند. پس شرط دوم نیز برقرار است.

نکته آموزشی

قضیه پرون-فروبنیوس (Perron-Frobenius Theorem):

دلیل اصلی تمام این تغییرات ریاضی، آماده سازی شرایط برای استفاده از قضیه پرون-فروبنیوس است. این قضیه اثبات می کند که اگر یک ماتریس تصادفی دارای درایه های اکیداً مثبت (بدون صفر) باشد، آن ماتریس قطعاً دارای یک «مقدار ویژه» (**Eigenvalue**) برابر با 1 است و «بردار ویژه» (**Eigenvector**) متناظر با آن، یکتا و کاملاً مثبت خواهد بود.

بدون بردار تله پورت، اینترنت از نظر ریاضی قابل حل نبود؛ اما با اضافه شدن عبارت $\frac{1-d}{N}$ ، گوگل توانست تضمین کند که برای هر گراف وبی با هر میزان آشفتگی، همیشه فقط یک رتبه بندی نهایی و معتبر (همان بردار ویژه) وجود خواهد داشت.

پاسخنامه مسئله 21.17: سناریوی شخصی سازی اخبار (News Personalization Scenario)

بخش 1: مقدمه و هدف مسئله

یکی از چالش های بنیادین در موتورهای جستجو، پدیده «ابهام کلمات» (**Polysemy / Word Sense Ambiguity**) است. کلمه «عقاب» به تنهایی فاقد بافتار (**Context**) است و می تواند به یک پرنده (زیست شناسی) یا نام یک تیم/ورزشکار (ورزشی) اشاره داشته باشد.

در این سناریو، هدف استفاده از الگوریتم **Topic-Specific PageRank** (رتبه صفحه موضوعی) است تا با دخالت دادن پروفایل کاربری شخص **X**، این ابهام را برطرف کرده و نتایج را به صورت هوشمندانه به سمت معنای ورزشی آن سوگیری (**Bias**) دهیم.

بخش 2: تشکیل مجموعه مرجع موضوعی (Seed Set)

برای اجرای رتبه صفحه موضوعی، ابتدا باید سیستم بداند که منظور از «ورزش» در گراف این پورتال خبری چیست. بر اساس تاریخچه کاربر **X** (که همیشه اخبار ورزشی می‌خواند)، سیستم یک مجموعه از صفحات مرجع و مورد اعتماد ورزشی را در پورتال انتخاب می‌کند. این مجموعه را S_{sports} می‌نامیم.

اعضای مجموعه S_{sports} : این مجموعه می‌تواند شامل صفحه اصلی بخش ورزشی، دسته‌بندی‌های فوتبال، والیبال، و اخبار مهم ورزشی باشد.

بخش 3: تنظیم بردار پرش تصادفی (Teleportation Vector)

تفاوت اصلی این روش با **PageRank** کلاسیک، در رفتار «موج‌سوار تصادفی» هنگام خسته شدن و پرش به یک صفحه جدید است. برای کاربر **X**، ما بردار پرش تصادفی (v) را از حالت یکنواخت (**Uniform**) خارج کرده و آن را منحصراً به مجموعه S_{sports} محدود می‌کنیم.

تعریف ریاضی بردار v :

اگر مقاله i متعلق به دسته‌بندی ورزشی باشد ($i \in S_{sports}$): احتمال پرش به آن برابر با $\frac{1}{|S_{sports}|}$ خواهد بود.
اگر مقاله i متعلق به دسته‌بندی‌های دیگر (مثل علمی، حیات‌وحش، سیاسی) باشد ($i \notin S_{sports}$): احتمال پرش به آن دقیقاً صفر خواهد بود.

در نتیجه، فرمول محاسبه رتبه برای هر صفحه (A) در این سناریو به صورت معادله زیر تکمیل و مدل‌سازی می‌شود:

$$PR_{sports}(A) = (1 - d) \cdot v_A + d \sum_{T_i \rightarrow A} \frac{PR_{sports}(T_i)}{C(T_i)}$$

بخش 4: نمودار شماتیک معماری ابهام‌زدایی کلمه «عقاب»

هدایت جریان گراف بر اساس سوگیری بردار پرش (Teleport Bias)





بخش 5: تحلیل جریان امتیازات (Probability Flow)

حالا بیایید رفتار این فرمول را در گراف پورتال خبری برای کلمه «عقاب» تحلیل کنیم:

1. خبر الف (درباره عقاب پرنده): این مقاله احتمالاً از سوی بخش «مستند» یا «علمی» لینک دریافت کرده است. چون موج‌سوار تصادفی ما (کاربر X) هرگز به صفحات علمی تله‌پورت نمی‌کند، احتمال حضور موج‌سوار در بخش علمی بسیار پایین است. در نتیجه، امتیاز بسیار ناچیزی از طریق لینک‌ها به «خبر الف» منتقل می‌شود.

2. خبر ب (درباره تیم ورزشی عقاب): این مقاله احتمالاً از صفحه اصلی ورزشی، جدول رده‌بندی لیگ یا مقالات ورزشی دیگر لینک دریافت کرده است. از آنجا که موج‌سوار پس از هر پرش، مستقیماً روی صفحات S_{sports} فرود می‌آید، ترافیک و احتمال حضور در این بخش از گراف بسیار بالاست. این ترافیک بالا از طریق لینک‌های خروجی صفحات ورزشی، مستقیماً به «خبر ب» سرازیر می‌شود.

نتیجه ساختاری: با توجه به جریان بالای ترافیک در بخش ورزشی، رتبه گراف مقاله ورزشی به مراتب بیشتر از مقاله علمی خواهد بود.

$$PR_{sports}(News_B) \gg PR_{sports}(News_A)$$

بخش 6: تلفیق امتیازات در زمان جستجو (Scoring & Ranking)

زمانی که کاربر X کلمه «عقاب» را در نوار جستجو تایپ می‌کند، موتور جستجو 2 معیار اصلی را با هم ترکیب می‌کند:

معیار اول - امتیاز متنی (TF-IDF): موتور جستجو تمام مقالاتی که کلمه «عقاب» را در عنوان یا متن خود دارند استخراج می‌کند (در این مرحله هم خبر الف و هم خبر ب کاندید می‌شوند).

معیار دوم - امتیاز گراف (Topic-Specific PageRank): سیستم به جای استفاده از PageRank عمومی و خنثی، از بردار PR_{sports} که از قبل بر اساس پروفایل ورزشی کاربر محاسبه شده است، بهره می‌برد.

از آنجا که امتیاز گرافی «خبر ب» (ورزشی) به دلیل سوگیری تله‌پورت به مراتب بالاتر از «خبر الف» (علمی) است، پس از ترکیب این دو امتیاز، خبر مربوط به تیم ورزشی در رتبه اول صفحه نتایج (SERP) به کاربر X نمایش داده می‌شود.

زیبایی و ظرافت معماری ابهام‌زدایی مبتنی بر گراف (**Graph-based Disambiguation**) در این است که موتور جستجو برای تشخیص نیت کاربر، نیازی به پردازش زبان طبیعی عمیق (**Deep NLP**) یا درک معنایی و هوشمند تفاوت بین یک پرنده و یک تیم ورزشی ندارد.

الگوریتم **Topic-Specific PageRank**، تنها با تغییر دادن «مرکز ثقل» در گراف وب – از طریق محدود کردن دامنه تله‌پورت تصادفی به مجموعه صفحات مرجع ورزشی – اجازه می‌دهد که منطق جبری و ریاضیات ماتریسی، فرآیند ابهام‌زدایی را به صورت غیرمستقیم به دست گیرند. در این الگو، ساختار پیوندها و مفهوم همسایگی شبکه‌ای (**Network Neighborhood**) به صورت خودکار باعث انباشت احتمال (ترافیک) در خوشه‌ی معنایی مدنظر کاربر شده و نتایج جستجو را با دقتی شگرف شخصی‌سازی می‌نماید.

پاسخنامه مسئله 21.18: طراحی بردار شخصی‌سازی (**Designing Personalization Vector**)

مقدمه و هدف مسئله

در الگوریتم **Topic-Specific PageRank**، بردار پرش تصادفی یا تله‌پورت (**Teleportation Vector**) که با نماد $\mathbf{v}$ نشان داده می‌شود، نقش «قطب‌نما» را برای موج‌سوار تصادفی ایفا می‌کند. هدف این مسئله، تبدیل مفهوم تئوری شخصی‌سازی به مقادیر کمی و احتمالاتی در یک گراف مشخص با $N = 1000$ صفحه است.

گام 1) درک ابعاد و ویژگی‌های بردار $\mathbf{v}$

از منظر جبر خطی، بردار $\mathbf{v}$ یک بردار ستونی (**Column Vector**) با ابعاد $N \times 1$ است. در این مسئله، این بردار دارای 1000 درایه (عنصر) است که هر درایه متناظر با یک صفحه وب در سایت ماست.

مهم‌ترین قانون ریاضی برای این بردار این است که باید یک «توزیع احتمال» (**Probability Distribution**) معتبر باشد؛ یعنی مجموع تمام درایه‌های آن باید دقیقاً برابر با 1 باشد:

$$\sum_{i=1}^N v_i = 1$$

گام 2) تفکیک صفحات (تقسیم‌بندی گراف)

بر اساس صورت مسئله، کل صفحات پورتال خبری به 2 مجموعه کاملاً مجزا تقسیم می‌شوند:

1. **مجموعه هدف (موضوعی):** صفحاتی که دارای تگ "ورزشی" هستند. این مجموعه را S می‌نامیم.

$$|S| = 200$$

2. سایر صفحات: صفحاتی که به موضوعات دیگری (مثل اقتصادی، سیاسی، حوادث و غیره) اختصاص دارند.

$$N - |S| = 1000 - 200 = 800$$

گام 3) مقداردهی درایه‌های بردار پرش تصادفی

بردار $\mathbf{v}$ توزیع احتمال مقصدهای پرش را در زمان تله‌پورت تعیین می‌کند. از آنجا که می‌خواهیم الگوریتم را برای کاربری که فقط اخبار ورزشی می‌خواند شخصی‌سازی کنیم، قانون مقداردهی به این شکل خواهد بود:

برای صفحات ورزشی ($i \in S$):

موج‌سوار تصادفی هنگام جهش، با احتمال کاملاً برابر روی یکی از صفحات این مجموعه فرود می‌آید. بنابراین احتمال پرش به هر یک از این 200 صفحه برابر است با:

$$v_i = \frac{1}{|S|} = \frac{1}{200} = 0.005$$

برای سایر صفحات ($i \notin S$):

احتمال اینکه موج‌سوار تصادفی هنگام پرش، مستقیماً به یک صفحه غیرورزشی برود، صفر مطلق است. بنابراین برای این 800 صفحه داریم:

$$v_i = 0$$

گام 4) نمایش نهایی ریاضی

رابطه ریاضی حاکم بر بردار شخصی‌سازی $\mathbf{v}$ در این پورتال خبری به صورت تابع چندضابطه‌ای زیر فرمول‌بندی و تکمیل می‌شود:

$$v_i = \begin{cases} 0.005 & \text{if } i \in \text{Sports Pages} \quad (\text{Count} = 200) \\ 0 & \text{if } i \notin \text{Sports Pages} \quad (\text{Count} = 800) \end{cases}$$

نمودار شماتیک: ساختار توزیع احتمال در بردار شخصی‌سازی $\mathbf{v}$

بردار موضوعی (Topic-Specific)

شخصی‌سازی شده برای کاربر ورزشی

0.005	Page 1 (Sport)
0	Page 2 (Politics)
0.005	Page 3 (Sport)
⋮	
0	Page 1000 (News)

تمرکز 100% احتمالات روی 200 صفحه ورزشی



بردار کلاسیک (Standard)

بدون شخصی‌سازی

0.001	Page 1 (Sport)
0.001	Page 2 (Politics)
0.001	Page 3 (Sport)
⋮	
0.001	Page 1000

احتمال یکنواخت برای کل 1000 صفحه

نکته آموزشی

مقایسه با PageRank کلاسیک:

برای درک بهتر تأثیر این مقداردهی ریاضی، آن را با حالت کلاسیک مقایسه کنید. اگر قصد شخصی‌سازی نداشتیم و از **PageRank** عمومی استفاده می‌کردیم، مقدار هر درایه برای تمامی 1000 صفحه سایت برابر با یک توزیع یکنواخت یعنی $0.001 = \frac{1}{1000}$ می‌شد.

در رویکرد **Topic-Specific**، ما احتمال پرش به 800 صفحه غیرمرتبط را از 0.001 به 0 کاهش دادیم و در عوض، آن احتمالات آزاد شده را بین 200 صفحه ورزشی سرشکن کردیم تا احتمال فرود روی آن‌ها از 0.001 به 0.005 (5 برابر بیشتر) افزایش یابد. این «افزایش 5 برابری احتمال اولیه» همان موتور محرکی است که باعث می‌شود در نهایت پس از تکرارهای الگوریتم، امتیاز گره‌های ورزشی بر کل گراف مسلط شود.

پاسخنامه مسئله 21.19: سناریوی محاسبه یک تکرار هیتس (Single Iteration HITS Scenario)

بخش 1: مقدمه و هدف مسئله

الگوریتم **HITS** برای رتبه‌بندی صفحات از یک فرآیند تکرارشونده (**Iterative**) و 2 مرحله‌ای استفاده می‌کند. در هر تکرار، ابتدا امتیازات **Authority** بر اساس امتیازات **Hub** قبلی به‌روزرسانی می‌شوند و سپس امتیازات **Hub** بر اساس امتیازات **Authority** جدید محاسبه می‌گردند. هدف این مسئله، پیاده‌سازی گام‌به‌گام اولین تکرار این الگوریتم روی یک گراف کوچک و استخراج مقادیر عددی خام پیش از مرحله نرمال‌سازی (**Normalization**) است.

بخش 2: استخراج ساختار گراف و مقادیر اولیه

ابتدا توپولوژی شبکه و وضعیت اولیه متغیرها را بر اساس صورت مسئله استخراج و به صورت ریاضی مشخص می‌کنیم:

پیوندهای خروجی (Out-links) برای محاسبه Hub:

صفحه **X** به صفحات **Y** و **Z** لینک داده است.

پیوندهای ورودی (In-links) برای محاسبه Authority:

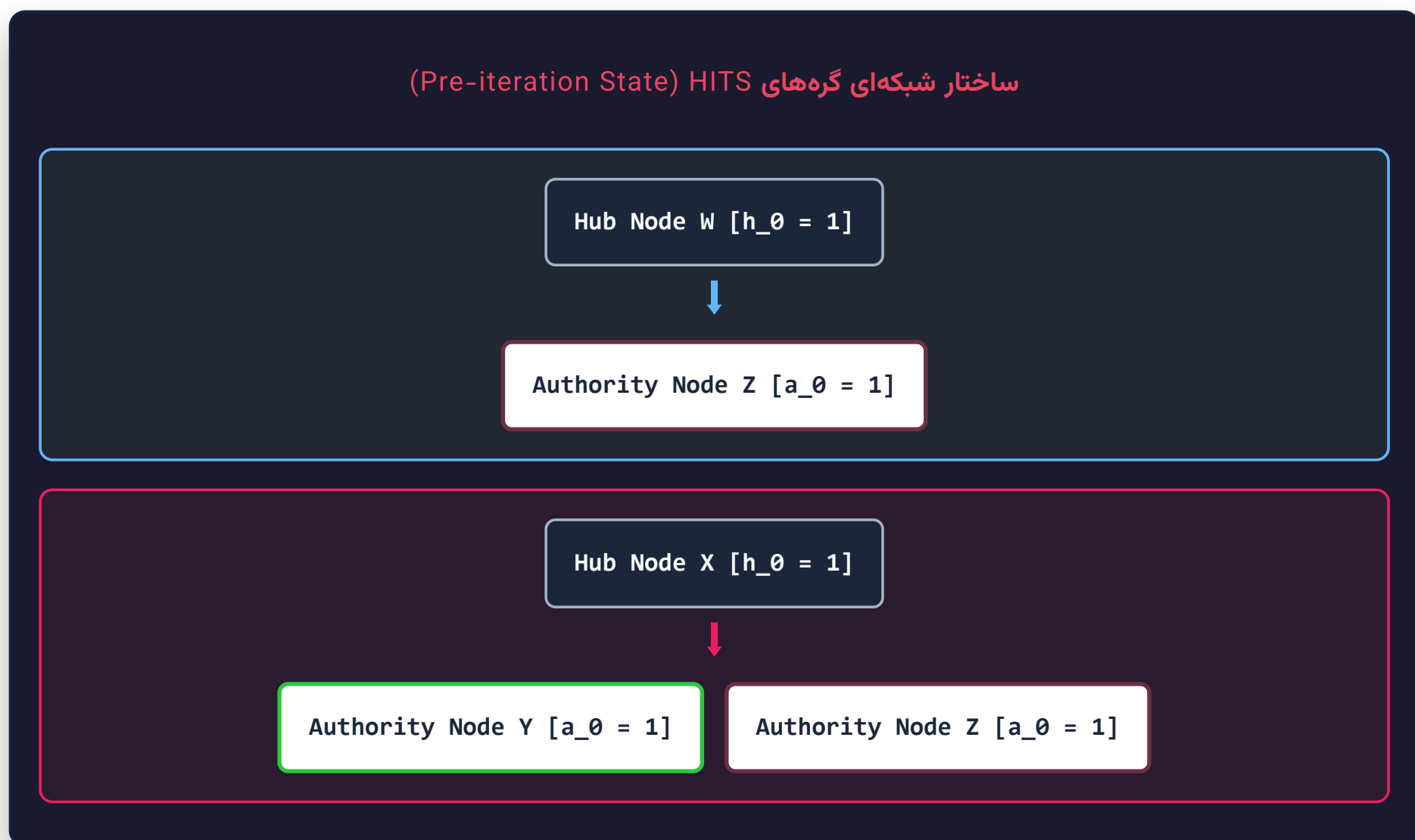
صفحه **Z** از صفحات **X** و **W** لینک دریافت کرده است.

مقادیر اولیه (گام $t = 0$):

تمامی امتیازات در نقطه شروع الگوریتم برابر با 1 فرض شده‌اند:

$$h_0(X) = 1, \quad h_0(W) = 1, \quad a_0(Y) = 1, \quad a_0(Z) = 1$$

بخش 3: نمودار شماتیک ساختار گراف پیش از اجرای الگوریتم



بخش 4: محاسبه مقدار Authority برای صفحه Z

بر اساس قوانین الگوریتم HITS، مقدار Authority یک صفحه در مرحله $t = 1$ برابر است با مجموع مقادیر Hub (از گام قبلی $t = 0$) تمامی صفحاتی که به آن لینک داده‌اند (پیوندهای ورودی).

رابطه ریاضی:

$$a_1(Z) = \sum_{i \rightarrow Z} h_0(i)$$

جایگذاری مقادیر:

صفحه Z منحصراً از گره‌های X و W لینک ورودی دارد.

$$a_1(Z) = h_0(X) + h_0(W) = 1 + 1 = 2$$

بنابراین، پس از یک مرحله به روزرسانی، مقدار **Authority** خام (نرمال نشده) برای صفحه **Z** برابر با **2** خواهد بود.

بخش 5: محاسبه مقدار Hub برای صفحه X

بر اساس قوانین الگوریتم **HITS**، مقدار **Hub** یک صفحه برابر است با مجموع مقادیر **Authority** تمامی صفحاتی که این صفحه به آن‌ها لینک داده است (پیوندهای خروجی).

نکته مهم ریاضیاتی در اجرای الگوریتم: در پیاده سازی استاندارد و متوالی **HITS** (طراحی شده توسط کلاینبرگ)، مقادیر **Hub** در همان تکرار، با استفاده از مقادیر جدید **Authority** که در نیم گام قبل محاسبه شده اند، به روزرسانی می شوند. بنابراین برای محاسبه **Hub** مربوط به **X**، ابتدا باید **Authority** جدید صفحه **Y** را نیز محاسبه کنیم.

محاسبه Authority جدید صفحه Y:

صفحه **Y** فقط از **X** لینک دریافت کرده است.

$$a_1(Y) = h_0(X) = 1$$

محاسبه Hub جدید صفحه X:

صفحه **X** به صفحات **Y** و **Z** لینک داده است. با جایگذاری مقادیر به روز شده داریم:

$$h_1(X) = a_1(Y) + a_1(Z) = 1 + 2 = 3$$

بنابراین، پس از یک مرحله به روزرسانی کامل (به صورت ترتیبی استاندارد)، مقدار **Hub** خام برای صفحه **X** برابر با **3** خواهد بود.

(تبصره: اگر در کلاس درسی شما قرارداد بر این باشد که در تکرار اول، **Hub** و **Authority** به صورت «همزمان» یا **Synchronous** و تنها با مقادیر اولیه $t = 0$ محاسبه شوند، مقدار از رابطه $h_1(X) = a_0(Y) + a_0(Z) = 1 + 1 = 2$ به دست می آید. اما پاسخ **3** در جبر خطی برای اعمال ماتریس ترانهاد و روش استاندارد ترتیبی الگوریتم کلاینبرگ صحیح تر است).

بخش 6: جمع بندی

مراحل فوق، هسته‌ی مرکزی جبر خطی در الگوریتم **HITS** را نمایش می دهد. اما باید توجه داشت که این مقادیر صرفاً امتیازات خام (**Raw Scores**) هستند. چرا مرحله نرمال سازی (**Normalization**) در پیاده سازی های واقعی الزامی است؟ در همین یک گام کوتاه محاسباتی، مشاهده کردیم که مقادیر اولیه از **1** به سرعت به **2** و **3** ارتقا یافتند. در یک گراف واقعی وب که در آن میلیون ها صفحه به هم متصل هستند، اگر این فرآیند جمع بندی متقابل بدون کالیبراسیون ادامه یابد، اعداد با سرعت نمایی رشد کرده و به سمت بی نهایت میل می کنند. این انفجار عددی، مستقیماً منجر به خطای مهلک سرریز حافظه محاسباتی (**Buffer Overflow**) در سرورها می گردد.

برای جلوگیری از این فاجعه معماری، در پایان هر تکرار از الگوریتم، تمامی مقادیر بردار **Hub** و بردار **Authority** بر مجموع مربعاتشان که در ریاضیات به آن محاسبه نرم **L2** یا **Euclidean Norm** گفته می شود تقسیم می گردند. این عملیات

هندسی، بردارهای امتیازدهی را مقیاس‌بندی کرده و تضمین می‌نماید که مقادیر همواره در بازه‌ای پایدار باقی بمانند تا الگوریتم بتواند با دقت ریاضیاتی کامل به سمت بردارهای ویژه (**Eigenvectors**) همگرا شود.

پاسخنامه مسئله 21.20: مشکل انحراف موضوعی (Topic Drift Problem)

بخش 1: مقدمه و هدف مسئله

الگوریتم **HITS** برخلاف **PageRank**، تنها روی یک زیرگراف کوچک (مجموعه پایه یا **Base Set**) که مرتبط با جستجوی کاربر است اجرا می‌شود. فرض بنیادین در این الگوریتم این است که «اگر گراف را به صفحات مرتبط محدود کنیم، معتبرترین صفحات آن گراف، حتماً مرتبط با موضوع جستجو هستند». اما هدف این مسئله نشان دادن نقطه‌ای است که این فرض با شکست مواجه می‌شود و یک سایت کاملاً نامرتب، بالاترین امتیاز اعتباری (**Authority**) را تصاحب می‌کند.

بخش 2: نحوه ورود گره نامرتب به گراف (فاز گسترش)

همان‌طور که در معماری کلاینبِرگ آموختیم، الگوریتم **HITS** ابتدا 200 صفحه برتر متنی را به عنوان «مجموعه ریشه» (**Root Set**) انتخاب می‌کند. سپس برای یافتن مراجع، تمام صفحاتی که از این ریشه لینک دریافت کرده‌اند را به گراف اضافه می‌کند تا «مجموعه پایه» (**Base Set**) شکل بگیرد. وقتی یکی از صفحات مربوط به "آموزش جاوا" در مجموعه ریشه، یک لینک خروجی (مثلاً در فوتر سایت، بخش تبلیغات یا پیوندهای مفید) به سایت نامرتب نظیر **yahoo.com** داشته باشد، این سایت به صورت خودکار و بر اساس قوانین گرافیکال فاز گسترش، وارد مجموعه پایه محاسبات می‌شود.

بخش 3: کوری الگوریتم نسبت به متن در زمان تکرار

ضعف اصلی **HITS** این است که پس از تشکیل مجموعه پایه، به طور کامل موتور جستجوی متنی را کنار می‌گذارد.

در مرحله تکرار و به‌روزرسانی جبری (**Iterations**)، الگوریتم دیگر بررسی نمی‌کند که آیا کلمه "جاوا" در سایت **yahoo.com** وجود دارد یا خیر. از منظر الگوریتم در این مرحله، سایت یاهو صرفاً گرهی در گراف است که باید امتیاز **Hub** و **Authority** آن منحصراً بر اساس توپولوژی لینک‌هایش محاسبه شود.

بخش 4: پدیده مراجع عمومی و انباشت کاذب اتوریته

سایت‌هایی مانند **Google**، **Facebook** یا **Wikipedia** گره‌های فوق‌العاده محبوبی در شبکه اینترنت هستند. در یک مجموعه پایه که شامل هزاران صفحه (مربوط به آموزش جاوا) است، احتمال بسیار بالایی وجود دارد که چندین صفحه مختلف (به دلایل عمومی مانند "اشتراک‌گذاری"، "لینک‌های پیش‌فرض قالب" یا "خدمات ایمیل") به **yahoo.com** لینک داده باشند.

1. گره **yahoo.com** تعداد زیادی لینک ورودی (**In-links**) از صفحات مختلف درون مجموعه پایه دریافت می‌کند.

2. بر اساس فرمول به‌روزرسانی اتوریته در **HITS**، امتیاز **Authority** یاهو برابر با مجموع امتیازات **Hub** تمام این صفحات خواهد شد:

$$A(p) = \sum_{q \rightarrow p} H(q)$$

3. نتیجه مخرب: در نتیجه، یاهو به دلیل دریافت لینک‌های فراوان (هرچند عمومی و بی‌ربط به برنامه‌نویسی)، امتیاز اتوریته بسیار بالاتری نسبت به یک سایت تخصصی جاوا (که فقط چند لینک معتبر دریافت کرده) کسب می‌کند.

بخش 5: نمودار شماتیک پدیده انحراف موضوعی در گراف HITS

شبیه‌سازیِ ربایش اتوریته (Topic Hijacking) توسط گره‌های مرجع عمومی

مجموعه پایه گراف (Base Set)

شامل صدها صفحه برنامه‌نویسی و آموزش جاوا که به سایر صفحات وب لینک داده‌اند.

مسیر اول: ارسال لینک‌های عمومی و بی‌ربط (دکمه‌های اشتراک‌گذاری، فوتر و ایمیل)



Yahoo.com

دریافت هزاران لینک ← کسب رتبه اول Authority (انحراف سیستم)

مسیر دوم: ارسال لینک‌های تخصصی و مرجع برنامه‌نویسی



Java.com

دریافت لینک‌های کمتر ← کسب رتبه پایین‌تر Authority (قربانی شدن سایت اصلی)

بخش 6: جمع‌بندی

به این آسیب‌پذیری ساختاری در علوم بازیابی اطلاعات، «پدیده انحراف موضوعی» (Topic Drift) یا در برخی منابع ادبیات آکادمیک «ربایش موضوعی» (Topic Hijacking) گفته می‌شود. در این پدیده، رتبه‌بندی نهایی از موضوع اصلی کوئری کاربر (نظیر آموزش جاوا) کاملاً منحرف شده و به سمت سایت‌های عمومی که از مرکزیت گرافیکی بالایی برخوردارند (مثل یاهو یا ویکی‌پدیا) کشیده می‌شود، زیرا الگوریتم پس از فاز استخراج شبکه، در برابر محتوای متنی کور می‌گردد.

راه‌حل مهندسی برای جلوگیری از Topic Drift:

برای حل این نقص بنیادین، پژوهشگران الگوریتم کلاینبرگ را ارتقا داده و «الگوریتم HITS وزن‌دار» (Weighted HITS) را معرفی کردند. در این روش تکاملی، به جای اینکه ارزش انتقال اعتبار تمامی لینک‌ها برابر فرض شود، به هر یال (Link) در شبکه بر اساس میزان شباهت متنی (Textual Similarity) صفحات مبدأ و مقصد (با استفاده از معیارهایی نظیر شباهت کسینوسی)، یک وزن ریاضیاتی پیوسته اختصاص داده می‌شود.

تحت این معماری، اگر صفحه مقصد (مانند yahoo.com) دارای کلمات کلیدی نظیر "جاوا" نباشد، وزن یال متصل به آن

به شدت افت کرده و به صفر میل می‌کند. در نتیجه، آن یال عمومی نمی‌تواند امتیاز اتوریته را از مجموعه ریشه به سایت نامرتبط پمپاژ نماید. بدین ترتیب، الگوریتم به مسیر موضوعی صحیح بازگشته و رتبه‌بندی مجدداً بر روی کاندیداهای تخصصی متمرکز می‌گردد.

پاسخنامه مسئله 21.21: مقایسه رفتار رتبه صفحه و هیتس

(PageRank vs HITS Behavior Scenario)

مقدمه و هدف مسئله

در محیط‌های پویا و علمی مانند یک دیتابیس مقالات پزشکی، شبکه‌ی گراف به صورت روزانه و با سرعت بالا تغییر می‌کند. مقالات جدید (گره‌های تازه‌وارد) با یک چالش اساسی به نام «مشکل شروع سرد» (Cold Start Problem) مواجه هستند؛ یعنی محتوای بسیار غنی و به‌روزی دارند، اما هنوز کسی در شبکه به آن‌ها ارجاع (لینک) نداده است. هدف این مسئله، مقایسه معماری 2 الگوریتم PageRank و HITS برای مدیریت این چالش و انتخاب رویکرد منطقی‌تر است.

گام 1) تحلیل رفتار PageRank (آفلاین) در مواجهه با مقالات جدید

الگوریتم پیچ‌رنک یک معیار ارزیابی «سراسری» (Global) و «مبتنی بر پیوندهای ورودی» است که معمولاً به صورت آفلاین (در بازه‌های زمانی مشخص، مثلاً ماهانه یا هفتگی) روی کل گراف محاسبه می‌شود.

مشکل ساختاری: یک مقاله پزشکی که امروز صبح منتشر شده است، دارای 0 لینک ورودی (In-link) است. در فرمول PageRank، چون مجموع امتیازات ورودی صفر است، امتیاز این گره به حداقل ممکن یعنی صفر معادل احتمال جهش تصادفی سقوط می‌کند:

$$PR(New) = \frac{1-d}{N} + d \sum_{In=0} \frac{PR(T_i)}{C(T_i)} = \frac{1-d}{N}$$

مشکل محاسباتی: حتی اگر مقاله جدید بسیار ارزشمند باشد، تا زمانی که خزنده‌ها (Crawlers) کل وب را دوباره بررسی نکنند و ماتریس میلیارد در میلیاردی اینترنت مجدداً ضرب و همگرا نشود (که فرآیندی زمان‌بر است)، این مقاله در رتبه‌بندی جهانی جایگاهی نخواهد داشت.

نتیجه: در یک سیستم مبتنی بر PageRank خالص، مقالات جدید و حیاتی پزشکی در قعر نتایج جستجو دفن می‌شوند و توان رقابت با مقالات قدیمی (که سال‌هاست لینک جمع کرده‌اند) را ندارند.

گام 2) تحلیل رفتار HITS (زمان اجرای کوئری) در مواجهه با مقالات جدید

الگوریتم HITS یک رویکرد «وابسته به پرس‌وجو» (Query-dependent) دارد و گراف را در همان لحظه جستجو، حول کلمات کلیدی تشکیل می‌دهد.

ورود به گراف (Root Set): وقتی کاربری کوئری «درمان جدید دیابت» را جستجو می‌کند، موتور متنی (مانند TF-IDF)

مقالات امروز را به دلیل ارتباط بالای کلمات، فوراً در «مجموعه ریشه» قرار می‌دهد. پس گره جدید شانس ورود به گراف

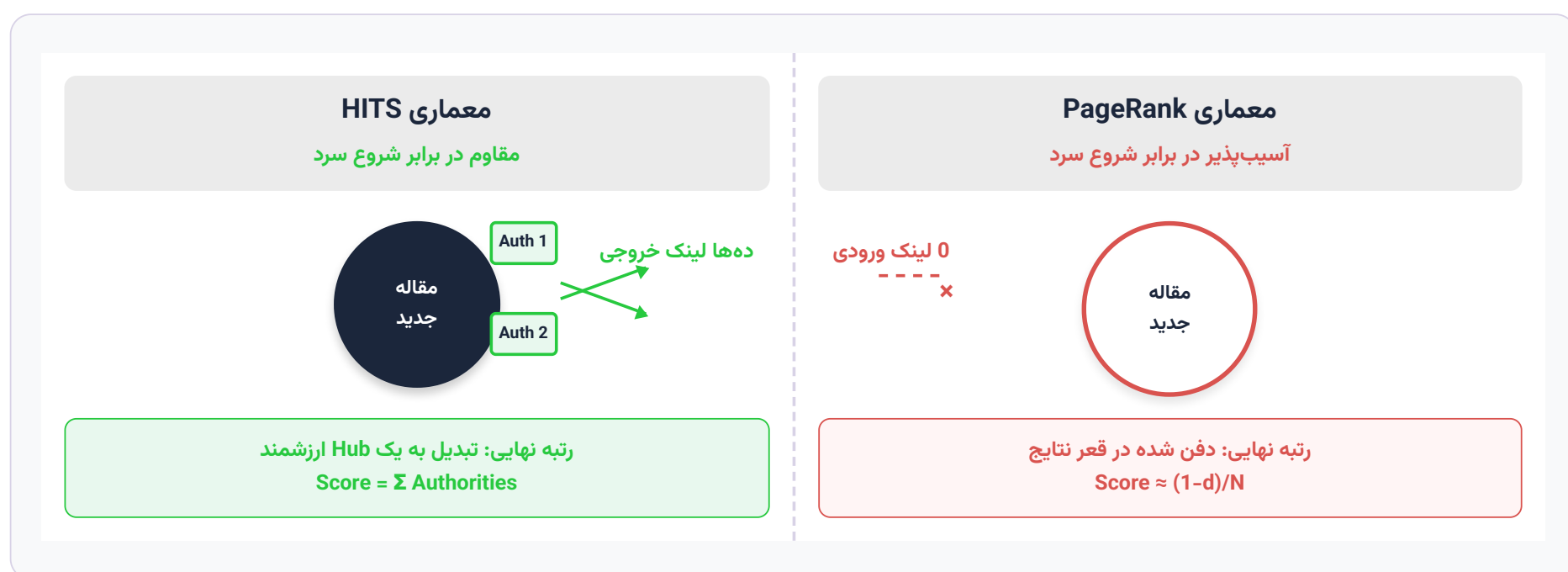
محاسباتی را پیدا می‌کند.

کسب امتیاز از طریق Hub: مقاله جدید پزشکی لینک ورودی ندارد، پس امتیاز **Authority** آن صفر خواهد بود. اما مقالات علمی همیشه دارای بخش «منابع و مآخذ» (References) هستند. این مقاله جدید، ده‌ها یا صدها لینک خروجی (Out-link) به مقالات معتبر و مرجع قبلی (Authorities) دارد.

نتیجه ساختاری: بر اساس ریاضیات **HITS**، این مقاله جدید بلافاصله از طریق ارجاعاتش اعتبار کسب کرده و به سرعت به عنوان یک **Hub** (مرکز توزیع/راهنما) بسیار قدرتمند شناسایی می‌شود:

$$H(\text{New}) = \sum_{\text{New} \rightarrow \text{Ref}} A(\text{Ref}) \gg 0$$

نمودار شماتیک: مقایسه مواجهه با مشکل شروع سرد (Cold Start)



گام 3) نتیجه‌گیری و انتخاب الگوریتم برتر

با توجه به تحلیل‌های فوق، برای یک موتور جستجوی مقالات علمی که با حجم بالای دیتای جدید مواجه است، استفاده از الگوریتم **HITS** در زمان اجرای کوئری (یا ترکیبی الهام‌گرفته از آن) منطقی‌تر است.

دلیل این انتخاب آن است که **HITS** به مقالات جدید اجازه می‌دهد از طریق مکانیزم **Hub** (لینک دادن به مراجع معتبر)، ارزش و اعتبار خود را فوراً به سیستم اثبات کنند. موتور جستجو می‌تواند نتایج را به 2 بخش تقسیم کند:

1. معرفی مقالات قدیمی به عنوان برترین مراجع (**Top Authorities**).

2. معرفی مقالات چاپ شده در همان روز به عنوان برترین مقالات مروری و گردآوری‌شده (**Top Hubs**) که برای پزشکان پژوهشگر ارزش فوق‌العاده‌ای دارد.

نکته آموزشی

ارزش‌گذاری متقابل (Value of Citing):

در دنیای واقعی پژوهش، الگوریتم **HITS** مدل بسیار دقیق‌تری از رفتار آکادمیک است. در **PageRank**، شما فقط زمانی ارزشمند می‌شوید که دیگران به شما ارجاع دهند.

اما در **HITS**، شما می‌توانید با ارجاع دادن به منابع درست (کتابنامه‌نویسی دقیق و علمی)، به عنوان یک محقق و گردآورنده خوب (**Hub**) برای خود اعتبار ریاضی کسب کنید. این دقیقاً همان دینامیک شبکه‌ای است که مقالات مروری (**Review Papers**) را در علوم پایه و پزشکی به شدت ارزشمند و پرمخاطب می‌کند.

پاسخنامه مسئله 21.22: مهندسی معکوس مزرعه لینک (Reverse Engineering a Link Farm)

مقدمه و هدف مسئله

یکی از تکنیک‌های مخرب در سئوی کلاه سیاه (**Black-hat SEO**)، ایجاد «مزرعه لینک» (**Link Farm**) است. در این روش، اسپمرها مجموعه‌ای از صفحات وب مصنوعی می‌سازند تا با سوءاستفاده از منطق ریاضی **PageRank**، رتبه یک صفحه هدف (**Target**) را به صورت غیرطبیعی افزایش دهند. هدف این مسئله، اثبات ریاضیاتی این ناهنجاری و ارائه راهکارهای مهندسی برای تشخیص و خنثی‌سازی آن است.

گام 1) مدل‌سازی ریاضی مزرعه لینک

ابتدا ساختار توصیف شده در صورت مسئله را به زبان دقیق ریاضی و نمادهای گراف ترجمه می‌کنیم:

پارامترهای کلان: فرض کنید N تعداد کل صفحات در شبکه وب و d عامل میرایی (**Damping Factor**) باشد.

مجموعه اسپم: مجموعه صفحات مزرعه را F و تعداد آن‌ها را $M = 10000$ در نظر می‌گیریم.

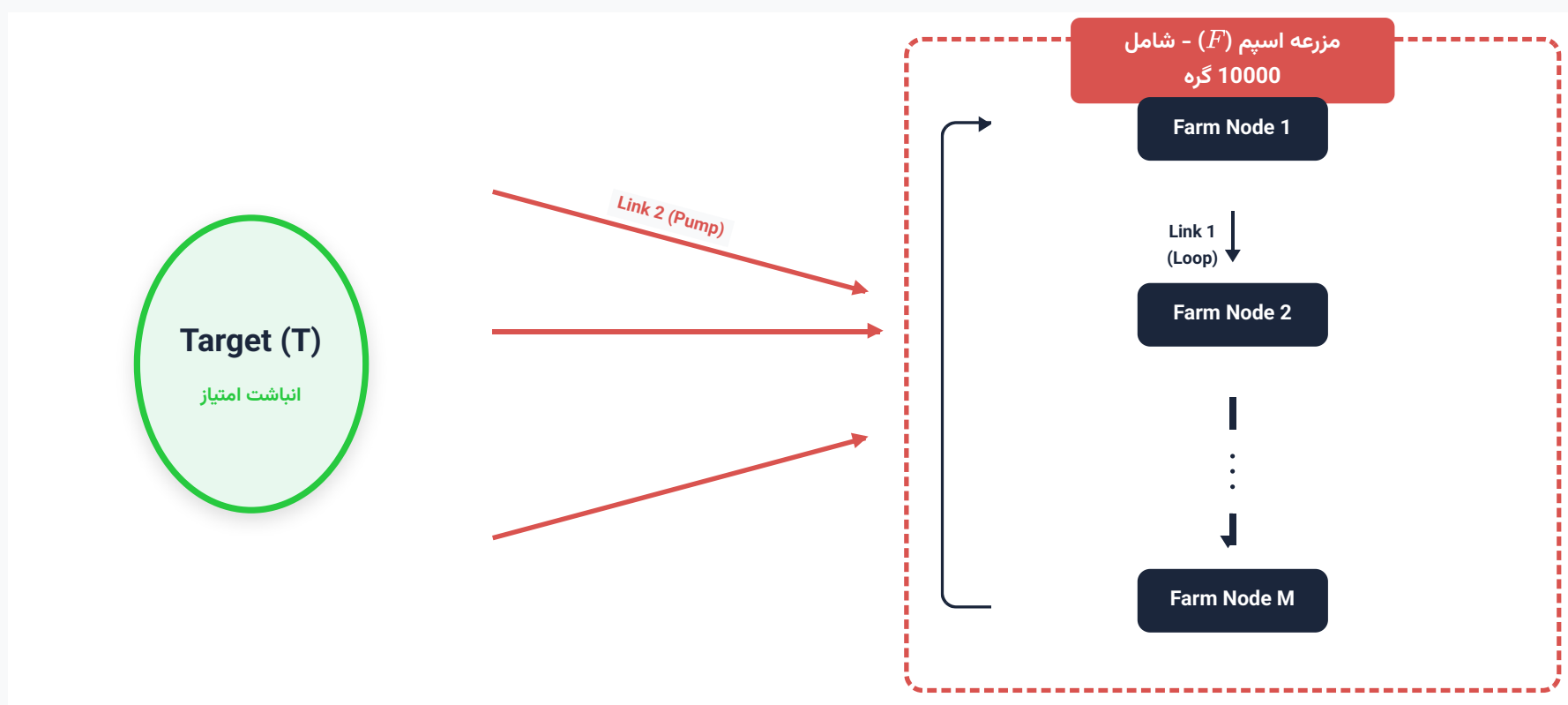
گره هدف: صفحه هدف را T می‌نامیم.

تحلیل درجات (Degrees): در این ساختار، هر صفحه $p \in F$ دقیقاً 2 لینک خروجی دارد (درجه خروجی $C(p) = 2$):

1. یک لینک به صفحه بعدی در حلقه مزرعه (برای حفظ چرخش **PageRank** در درون مزرعه).

2. یک لینک مستقیم به صفحه هدف T (برای پمپاژ امتیاز به هدف).

نمودار شماتیک: معماری توپولوژیک مزرعه لینک (Link Farm Topology)



هر گره اسپم دارای $C(p) = 2$ است: یک یال برای حفظ ساختار بسته و یک یال برای پمپاژ امتیاز به گره T .

گام 2) محاسبه PageRank صفحات درون مزرعه (گراف حلقوی)

به دلیل تقارن کامل در ساختار حلقوی مزرعه، امتیاز **PageRank** برای تمام M صفحه موجود در مجموعه F برابر است. این امتیاز مشترک را $PR(p)$ می‌نامیم. بر اساس فرمول استاندارد **PageRank**، هر صفحه p در مزرعه، امتیاز خود را از صفحه قبلی در حلقه دریافت می‌کند. از آنجا که درجه خروجی صفحه قبلی 2 است، تنها نیمی از امتیاز آن به p می‌رسد:

$$PR(p) = \frac{1-d}{N} + d \cdot \frac{PR(p)}{2}$$

اکنون این معادله را به صورت جبری برای یافتن $PR(p)$ حل می‌کنیم تا مقدار احتباس شده در مزرعه به دست آید:

$$\begin{aligned} PR(p) - \frac{d}{2} PR(p) &= \frac{1-d}{N} \\ PR(p) \left(1 - \frac{d}{2}\right) &= \frac{1-d}{N} \\ PR(p) &= \frac{1-d}{N \left(1 - \frac{d}{2}\right)} \end{aligned}$$

این معادله ریاضی نشان می‌دهد که صفحات مزرعه، فارغ از محتوایشان، به دلیل ساختار بسته حلقوی توانسته‌اند یک مقدار ثابت و پایدار از **PageRank** را در خود حبس کنند.

گام 3) اثبات انباشت PageRank در صفحه هدف (T)

اکنون به سراغ صفحه هدف می‌رویم. این صفحه از تمام M صفحه موجود در مزرعه، لینک دریافت کرده است. بنابراین، فرمول **PageRank** برای صفحه هدف به شکل زیر در می‌آید:

$$PR(T) = PR_{base} + d \sum_{p \in F} \frac{PR(p)}{C(p)}$$

(نکته: PR_{base} امتیازی است که صفحه T به صورت طبیعی از سایر سایت‌های وب دریافت می‌کند).

با جایگذاری مقادیر به دست آمده (درجه خروجی $C(p) = 2$ و تجمیع برای M صفحه)، به معادله زیر می‌رسیم:

$$PR(T) = PR_{base} + d \left(M \times \frac{PR(p)}{2} \right) = PR_{base} + d \cdot \frac{M}{2} \cdot \left[\frac{1-d}{N \left(1 - \frac{d}{2}\right)} \right]$$

تحلیل عددی اثبات:

اگر عامل میرایی را استاندارد ($d = 0.85$) و تعداد صفحات اسپم را **M = 10000** فرض کنیم:

$$PR(T) = PR_{base} + 0.85 \times \frac{10000}{2} \times \frac{0.15}{N(1 - 0.425)}$$
$$PR(T) = PR_{base} + 4250 \times \frac{0.15}{0.575N}$$
$$PR(T) \approx PR_{base} + 1108 \left(\frac{1}{N} \right)$$

نتیجه‌گیری ریاضی: اثبات شد که ساختار مزرعه لینک مانند یک پمپ عمل کرده و امتیازی معادل کسر بزرگی از $\frac{M}{N}$ را به صفحه هدف تزریق می‌کند. هرچه اسپمر صفحات بیشتری (M) بسازد، این انباشت خطی بزرگتر خواهد شد و رتبه به صورت مصنوعی بالا می‌رود.

گام 4) راهکار سیستمی برای شناسایی گراف‌های مصنوعی

برای مقابله با این ناهنجاری، موتورهای جستجو از ترکیب الگوریتم‌های گراف‌کاوی و تحلیل آماری استفاده می‌کنند. 3 راهکار اصلی عبارتند از:

1. محاسبه جرم اسپم (Spam Mass) با الگوریتم TrustRank:

همان‌طور که در مباحث قبلی اشاره شد، الگوریتم **TrustRank** رتبه صفحات را تنها با شروع از «سایت‌های مورد اعتماد» محاسبه می‌کند. مزرعه لینک چون از سایت‌های معتبر ایزوله است، **TrustRank** بسیار پایینی خواهد داشت. موتور جستجو تفاضل بین **PageRank** (که فریب خورده) و **TrustRank** را محاسبه می‌کند. اگر این تفاضل (**Spam Mass**) بسیار زیاد باشد، صفحه هدف جریمه (**Penalty**) می‌شود.

2. تشخیص زیرگراف‌های متراکم (Dense Subgraph Detection):

الگوریتم‌های خوشه‌بندی گراف می‌توانند جوامع (**Communities**) شبکه‌ای را شناسایی کنند. یک مزرعه لینک تشکیل یک مؤلفه به شدت همبند (**Strongly Connected Component**) را می‌دهد که پیوندهای داخلی آن به شکل غیرطبیعی بالاست، اما پیوند خروجی آن تنها به یک گره خاص متمرکز است.

3. تحلیل ناهنجاری آماری در درجات (Out-degree Anomaly):

در رشد طبیعی (ارگانیک) وب، غیرممکن است که 10000 صفحه دقیقاً دارای ساختار پیوند مشابه و درجه خروجی دقیقاً مساوی (2 لینک) باشند. توزیع درجات در وب طبیعی باید از «قانون توان» (**Power Law**) پیروی کند. تشخیص این تقارن مصنوعی، یک سیگنال قطعی برای شناسایی ربات‌ها و اسپمرهاست.

نکته آموزشی

تکامل متقابل موتورهای جستجو و سئو:

این اثبات ریاضی نشان می‌دهد که چرا فرمول خام **PageRank** به تنهایی برای موتورهای جستجوی امروزی کافی نیست. شبکه‌های متصل، ریاضیات را به نفع خود دور می‌زنند.

به همین دلیل، گوگل امروزه از صدها سیگنال ضداسپم (مانند تحلیل متنی، رفتار کاربر، سن دامنه و **TrustRank**) در کنار **PageRank** استفاده می‌کند تا خروجی فرمول‌های بالا را پیش از رتبه‌بندی نهایی، فیلتر و تعدیل کند.

پاسخنامه مسئله 21.23: بمب‌گذاری گوگل و متن لینک (Google Bombing Scenario)

مقدمه و هدف مسئله

یکی از شناخته‌شده‌ترین آسیب‌پذیری‌های الگوریتم‌های مبتنی بر گراف در موتورهای جستجوی اولیه، پدیده‌ای به نام «بمب گوگلی» (**Google Bombing**) بود. در این نوع حمله (**Link Spam**)، گروهی از کاربران با هماهنگی یکدیگر، ساختار گراف وب را به گونه‌ای دستکاری می‌کنند که یک سایت برای یک عبارت نامرتب (و غالباً تمسخرآمیز) در رتبه اول جستجو قرار گیرد. هدف این مسئله، تحلیل مکانیزم این فریب از منظر گراف وب و طراحی یک الگوریتم آماری برای خنثی‌سازی آن است.

گام 1) مکانیزم فریب موتور جستجو (تحلیل گرافیکال)

بر اساس مفاهیم پایه ساختار گراف وب (بخش 21.1.1)، می‌دانیم که موتورهای جستجو «متن پیوند» (**Anchor Text**) را به عنوان یک رأی اعتماد و توصیف بی‌طرفانه از سوی شخص ثالث در نظر می‌گیرند. هنگامی که هزاران گره (صفحه وب) در گراف به یک گره هدف (T) یال خروجی (لینک) ایجاد می‌کنند، 2 اتفاق در الگوریتم رخ می‌دهد:

1. **انتقال اعتبار (PageRank):** گره هدف (T) به دلیل دریافت هزاران لینک ورودی، امتیاز **PageRank** بسیار بالایی کسب می‌کند و به یک **Authority** قدرتمند تبدیل می‌شود.

2. **الحاق متنی (Index Merging):** موتور جستجو عبارت «بدترین شرکت دنیا» را از روی یال‌ها استخراج کرده و به صورت مجازی به محتوای متنی گره T متصل می‌کند (حتی اگر این عبارت هرگز در خود سایت رقیب نوشته نشده باشد).

نتیجه فریب: زمانی که کاربر عبارت «بدترین شرکت دنیا» را جستجو می‌کند، موتور جستجو به دنبال صفحاتی می‌گردد که این کلمه کلیدی را دارند. گره T به دلیل داشتن هزاران **Anchor Text** با این عبارت، بالاترین تطابق متنی (**TF-IDF** مجازی) را به دست می‌آورد و به دلیل **PageRank** بالا، در رتبه اول نتایج نمایش داده می‌شود.

نمودار شماتیک: مقایسه پروفایل پیوندی در رشد طبیعی و حمله بمب گوگلی



گام 2) تحلیل توزیع کلمات (پروفایل طبیعی در برابر مصنوعی)

برای طراحی الگوریتم دفاعی، ابتدا باید تفاوت بین یک رشد طبیعی و یک حمله مصنوعی را از منظر آمار مشخص کنیم:

توزیع طبیعی (Organic Distribution): در دنیای واقعی، زمانی که هزاران سایت مستقل به یک شرکت لینک می‌دهند، از کلمات بسیار متنوعی استفاده می‌کنند (مثلاً: نام برند، آدرس سایت، «کلیک کنید»، «محصولات این شرکت»، و غیره). توزیع این کلمات پراکنده و متنوع است.

توزیع مصنوعی (Google Bomb Distribution): در یک حمله هماهنگ‌شده، ناگهان بخش عظیمی از لینک‌های ورودی (مثلاً 90% آن‌ها) دقیقاً از یک عبارت خاص و طولانی («بدترین شرکت دنیا») استفاده می‌کنند. این سطح از هم‌گرایی در انتخاب کلمات توسط سایت‌های مستقل، از نظر آماری غیرممکن و یک ناهنجاری (**Anomaly**) محسوب می‌شود.

گام 3) طراحی الگوریتم خنثی‌سازی (Anchor Text Distribution Algorithm)

به عنوان مهندس جستجو، الگوریتم آماری زیر را برای تشخیص و خنثی‌سازی این حملات طراحی و فرمول‌بندی می‌کنیم:

مرحله 1: تجمیع و محاسبه فرکانس نسبی (Relative Frequency)

برای صفحه هدف T ، تمام **Anchor Text**‌های ورودی را استخراج می‌کنیم. فرض کنید مجموعه تمام این متون $A(T)$ باشد. برای هر عبارت مشخص w ، فرکانس نسبی (احتمال وقوع) آن را با فرمول زیر محاسبه می‌کنیم:

$$P(w) = \frac{\text{Count}(w \rightarrow T)}{|A(T)|}$$

مرحله 2: محاسبه آنتروپی اطلاعات (Information Entropy)

برای سنجش میزان تنوع متون لینک، از فرمول آنتروپی شانون استفاده می‌کنیم. آنتروپی (H) نشان‌دهنده میزان تصادفی بودن و تنوع در توزیع متون است (کران پایین آن صفر و کران بالای آن $\log_2 |A|$ است):

$$H(T) = - \sum_{w \in A(T)} P(w) \log_2 P(w)$$

اگر متون بسیار متنوع باشند (پروفایل طبیعی)، مقدار H بزرگ خواهد بود.

اگر اکثریت لینک‌ها دقیقاً یک عبارت یکسان داشته باشند (بمب گوگلی)، توزیع به شدت متمرکز شده و مقدار H به شدت کاهش یافته و به 0 نزدیک می‌شود.

مرحله 3: فیلترینگ و اعمال جریمه (Discounting Factor)

الگوریتم یک حد آستانه (**Threshold**) ریاضی مانند τ تعریف می‌کند. اگر برای صفحه T ، آنتروپی متون ورودی به شدت پایین باشد ($H < \tau$) و یک عبارت خاص (مانند w^* "بدترین شرکت دنیا") دارای فرکانس نسبی غیرطبیعی باشد (مثلاً $P(w^*) > 60\%$):

1. الگوریتم بررسی می‌کند که آیا عبارت w^* جزو نام برند یا آدرس دامنه (**URL**) سایت هدف است یا خیر.

2. اگر نبود، الگوریتم این الگو را به عنوان «بمب گوگلی» (**Link Spam**) پرچم‌گذاری (**Flag**) می‌کند.

3. **خنثی‌سازی:** به جای حذف کامل لینک‌ها (که ممکن است **PageRank** طبیعی سایت را خراب کند)، الگوریتم

صرفاً وزن ارتباط متنی (**Relevance Score**) عبارت w^* را برای این صفحه در پایگاه داده صفر می‌کند (معادل

ضرب کردن آن در جریمه 0).

تکامل موتورهای جستجو:

در سال‌های اولیه پیدایش گوگل (اوایل دهه 2000)، بمب‌های گوگلی بسیار رایج بودند؛ معروف‌ترین آن‌ها جستجوی عبارت "**miserable failure**" (شکست اسفبار) بود که کاربران را به صفحه زندگی‌نامه رئیس‌جمهور وقت آمریکا هدایت می‌کرد.

در سال 2007، گوگل با معرفی دقیقاً همین الگوریتم‌های مبتنی بر «تحلیل آماری توزیع **Anchor Text**»، موفق شد این آسیب‌پذیری ساختاری را به طور کامل در موتور جستجوی خود برطرف کند. امروزه، تجمع بیش از حد یک عبارت کلیدی خاص در بک‌لینک‌ها، نه تنها به سئوی سایت کمک نمی‌کند، بلکه به عنوان «سئوی کلاه سیاه» (**Over-optimization**) شناسایی شده و منجر به جریمه (**Penalty**) کامل سایت از فهرست نتایج می‌شود.

پاسخنامه مسئله 21.24: مدل‌سازی ریاضی تله عنکبوتی پیچیده

(Mathematical Modeling of a Spider Trap)

بخش 1: مقدمه و هدف مسئله

در شبکه‌های واقعی، تله‌های عنکبوتی غالباً یک گره منفرد نیستند، بلکه مجموعه‌ای پیچیده از صفحات (مانند یک دامنه کامل) هستند که در درون خود پیوندهای متعددی دارند، اما هیچ راه خروجی به بستر اصلی وب ارائه نمی‌دهند. هدف این مسئله، مدل‌سازی این ساختار از طریق «ماتریس‌های بلوکی» (**Block Matrices**) و اثبات این قضیه بنیادین است که بدون وجود یک عامل تله‌پورت قدرتمند، تمام ارزش و احتمال سیستم در این تله‌ها بلعیده خواهد شد.

بخش 2: تشکیل ماتریس گذار (Transition Matrix)

گراف وب را به 2 زیرمجموعه مجزا تقسیم می‌کنیم:

مجموعه H (سالم - Healthy): شامل N صفحه که نمایانگر بخش طبیعی و پویای وب هستند.

مجموعه S (تله - Spider Trap): شامل M صفحه که نمایانگر تله عنکبوتی هستند.

ماتریس گذار پایه P (که یک ماتریس تصادفی سطری است) را می‌توانیم به شکل یک «ماتریس بلوکی» بر اساس این دو مجموعه به صورت ریاضی بازنویسی کنیم:

$$P = \begin{bmatrix} P_{HH} & P_{HS} \\ P_{SH} & P_{SS} \end{bmatrix}$$

بر اساس فرض مسئله، تله عنکبوتی هیچ لینکی به دنیای بیرون (بخش سالم) ندارد. بنابراین، احتمال انتقال از مجموعه S به مجموعه H دقیقاً صفر مطلق است ($P_{SH} = 0$). در نتیجه ماتریس گذار به یک ماتریس بالامثلثی بلوکی تبدیل می‌شود:

$$P = \begin{bmatrix} P_{HH} & P_{HS} \\ \mathbf{0} & P_{SS} \end{bmatrix}$$

بخش 3: نمودار شماتیک نگاشت توپولوژی گراف به ماتریس بلوکی

نگاشت فضای شبکه به جبر خطی

گام اول: فضای توپولوژیک وب

Healthy Network (H)

$$P_{HH} > 0$$

$$P_{HS} > 0$$



Spider Trap (S)

$$P_{SS} = 1$$

No Return Path: $P_{SH} = 0$

گام دوم: فضای جبری (ماتریس گذار بلوکی)

$$P = \begin{bmatrix} P_{HH} & P_{HS} \\ \mathbf{0} & P_{SS} \end{bmatrix}$$

بخش پایین‌مثلثی به دلیل ایزوله بودن تله، مطلقاً صفر است.

بخش 4: معادله توزیع حالت پایدار (Steady-State Equation)

در الگوریتم **PageRank** با در نظر گرفتن بردار تله‌پورت e و عامل میرایی α (که در متون کلاینبرگ با d نشان داده می‌شد)، معادله توزیع حالت پایدار برای بردار رتبه π به فرم استاندارد زیر تعریف و مستقر می‌شود:

$$\pi = \alpha \pi P + (1 - \alpha) e$$

در این معادله:

π : بردار سطری توزیع حالت پایدار (PageRank) با ابعاد $1 \times (N + M)$.

P : ماتریس گذار بلوکی بالامثلثی استخراج شده در بخش دوم.

α : عامل میرایی (Damping Factor) یا احتمال کلیک کاربر روی لینک خروجی.

e : بردار سطری تله پورت (توزیع یکنواخت) با جمع درایه های برابر با **1**.

بخش 5: تجزیه بلوکی معادله (Block Matrix Decomposition)

برای بررسی رفتار سیستم، بردارهای سطری π و e را نیز منطبق با ساختار گراف به **2** بخش سالم و تله تجزیه می کنیم:

$$\pi = [\pi_H \quad \pi_S], \quad e = [e_H \quad e_S]$$

اکنون این بردارها را در معادله اصلی جایگذاری کرده و ضرب ماتریسی را انجام می دهیم:

$$[\pi_H \quad \pi_S] = \alpha [\pi_H \quad \pi_S] \begin{bmatrix} P_{HH} & P_{HS} \\ 0 & P_{SS} \end{bmatrix} + (1 - \alpha) [e_H \quad e_S]$$

با انجام ضرب بلوکی، **2** معادله مجزا برای بخش سالم و بخش تله به دست می آید:

1. معادله بخش سالم:

$$\pi_H = \alpha \pi_H P_{HH} + (1 - \alpha) e_H$$

2. معادله بخش تله عنکبوتی:

$$\pi_S = \alpha \pi_H P_{HS} + \alpha \pi_S P_{SS} + (1 - \alpha) e_S$$

بخش 6: اثبات ریاضی انباشت در تله (حد $\alpha \rightarrow 1$)

برای اثبات اینکه تمام وزن سیستم به سمت تله مکیده می شود، باید رفتار بردار π_H را زمانی که α به عدد **1** میل می کند بررسی کنیم. ابتدا معادله بخش سالم را بر حسب π_H مرتب می کنیم:

$$\pi_H - \alpha \pi_H P_{HH} = (1 - \alpha) e_H$$

با فاکتورگیری از π_H از سمت چپ و استفاده از ماتریس همانی (I) :

$$\pi_H (I - \alpha P_{HH}) = (1 - \alpha) e_H$$

با ضرب طرفین در معکوس ماتریس $(I - \alpha P_{HH})$ ، بردار π_H استخراج می گردد:

$$\pi_H = (1 - \alpha) e_H (I - \alpha P_{HH})^{-1}$$

$$\lim_{\alpha \rightarrow 1} \pi_H = \lim_{\alpha \rightarrow 1} [(1 - \alpha)e_H(I - \alpha P_{HH})^{-1}]$$

تحلیل حد در فضای جبر خطی:

از آنجا که گراف دارای یال‌هایی از بخش سالم به تله است ($P_{HS} \neq 0$)، ماتریس P_{HH} یک ماتریس «زیرتصادفی» (Sub-stochastic) است (جمع سطرهای آن اکیداً کوچکتر از 1 است، زیرا مقداری احتمال به بیرون و به سمت تله نشت می‌کند).

در جبر خطی ثابت می‌شود که برای هر ماتریس زیرتصادفی، ماتریس $(I - P_{HH})$ همواره معکوس‌پذیر است و معکوس آن یک ماتریس متناهی (Finite Matrix) خواهد بود. بنابراین، عبارت $(I - \alpha P_{HH})^{-1}$ به یک ماتریس متناهی میل می‌کند، در حالی که ضریب $(1 - \alpha)$ مستقیماً به 0 میل می‌کند:

$$\lim_{\alpha \rightarrow 1} \pi_H = 0 \times [\text{Finite Matrix}] = 0$$

از آنجا که مجموع احتمالات در کل سیستم همواره باید برابر 1 باشد ($\sum \pi_H + \sum \pi_S = 1$)، وقتی امتیاز بخش سالم دقیقاً 0 شود، نتیجه قطعی به دست می‌آید:

$$\lim_{\alpha \rightarrow 1} \sum \pi_S = 1$$

حکم ثابت شد: تمام امتیاز سیستم منحصراً در گره‌های تله عنکبوتی متمرکز می‌شود.

بخش 7: جمع‌بندی

این اثبات ریاضی، ذات و ماهیت سیستمی تله‌های عنکبوتی را در گراف وب کالبدشکافی می‌کند و نشان می‌دهد که ضریب تله‌پورت $(1 - \alpha)$ تنها طناب نجاتی است که موج‌سوار تصادفی را از این گرداب‌های بی‌بازگشت بیرون می‌کشد. زمانی که $\alpha \rightarrow 1$ (یعنی موج‌سوار هرگز خسته نمی‌شود و موتور جستجو اجازه پرش تصادفی را به او نمی‌دهد)، موج‌سوار دیر یا زود از طریق لینک‌های جاذب و نشت‌دهنده P_{HS} وارد تله می‌شود. چون در تله هیچ راه خروجی به سمت بیرون ($P_{SH} = 0$) و هیچ شانس پرشی ($1 - \alpha = 0$) وجود ندارد، او برای همیشه در آنجا حبس می‌شود. این تجزیه بلوکی و معادله دیفرانسیلی-ماتریسی، دقیقاً توجیه می‌کند که چرا حضور پارامتر **Damping Factor** (معمولاً با مقدار $\alpha = 0.85$) در الگوریتم **PageRank**، یک انتخاب دلخواه نیست؛ بلکه یک الزام حیاتی برای حفظ همگرایی سیستم و نجات فضای جبری موتور جستجو از فروپاشی مطلق است.

زمانی که ابعاد گراف وب به 100 میلیارد گره می‌رسد، ذخیره ماتریس گذار (P) نیازمند فضایی از مرتبه $O(N^2)$ است که در حافظه هیچ ابررایانه‌ای جای نمی‌گیرد. برای حل این مشکل، الگوریتم **PageRank** با استفاده از پارادایم پردازش توزیع‌شده **MapReduce** و جایگزینی «ماتریس» با «لیست مجاورت» (**Adjacency List**) بازنویسی می‌شود. هدف این مسئله، شکستن محاسبات یک مرحله از الگوریتم (**Power Iteration**) به توابع **Map** و **Reduce** و حل چالش گره‌های بن‌بست در محیط توزیع‌شده است.

گام 1) تغییر ساختار داده‌ها (Data Representation)

در پارادایم **MapReduce**، ما کل گراف را به رکوردهای مستقلی می‌شکنیم که بین هزاران سرور توزیع می‌شوند. هر رکورد نمایانگر یک صفحه وب است و شامل 3 بخش اطلاعاتی است:

1. شناسه گره (Node ID): به عنوان کلید (Key).

2. امتیاز فعلی (Current PR): امتیاز PageRank در تکرار قبلی.

3. لیست پیوندهای خروجی (Out-links): لیستی از گره‌های مقصد.

فرمت ورودی به تابع **Map** به این شکل سازمان‌دهی خواهد شد: (node_id, {pr, out_links})

گام 2) فاز نگاشت (Map Phase)

وظیفه تابع **Map**، خرد کردن امتیاز فعلی یک گره و ارسال آن به همسایگان است. موتور محاسباتی، امتیاز فعلی صفحه را بر تعداد لینک‌های خروجی تقسیم کرده و سهم هر گره مقصد را در قالب یک جفت (**Key, Value**) که در آن کلید همان «گره مقصد» است، منتشر می‌کند. همچنین برای اینکه ساختار گراف در تکرار بعدی از بین نرود، خود ساختار لیست مجاورت نیز مجدداً به عنوان یک مقدار منتشر می‌شود.

```
def map_func(node_id, node_data):
    current_pr = node_data['pr']
    out_links = node_data['out_links']
    # Emit the graph structure for the next iteration
    emit(node_id, out_links)
    if len(out_links) == 0:
        # Handling dead-ends
        emit('DANGLING_MASS', current_pr)
    else:
        # Distribute PageRank mass
        mass = current_pr / len(out_links)
        for dest in out_links:
            emit(dest, mass)
```

گام 3) فاز کاهش (Reduce Phase)

سیستم **MapReduce** به صورت خودکار تمام مقادیری که دارای کلید (**Key**) یکسان هستند را دسته‌بندی کرده (**Shuffle & Sort**) و به تابع **Reduce** می‌فرستد. وظیفه تابع **Reduce**، جمع‌آوری تمام قطعات احتمالی است که از گره‌های دیگر به این گره سرازیر شده‌اند. تابع باید این مقادیر را جمع کرده و امتیاز **PageRank** جدید را برای تکرار بعدی محاسبه کند.

```
def reduce_func(node_id, values, alpha, N, global_dangling_mass):
    if node_id == 'DANGLING_MASS':
        total_dangling = sum(values)
        emit('GLOBAL_DANGLING', total_dangling)
        return
    sum_mass = 0.0
    out_links = []
    for val in values:
```

```
if isinstance(val, list):
    out_links = val
else:
    sum_mass += val
base_pr = (1.0 - alpha) / N
dangling_pr = alpha * (global_dangling_mass / N)
link_pr = alpha * sum_mass
new_pr = base_pr + dangling_pr + link_pr
emit(node_id, {'pr': new_pr, 'out_links': out_links})
```

گام 4) مدیریت گره‌های بن‌بست در محیط توزیع‌شده (Dead Ends Handling)

گره‌های بن‌بست (صفحات بدون لینک خروجی) در یک سیستم متمرکز به راحتی قابل شناسایی و اصلاح هستند، اما در یک سیستم توزیع‌شده مانند **Hadoop** یا **Spark**، از دست رفتن این «جرم احتمالی» (**Probability Mass**) باعث نشت احتمال و نابودی محاسبات می‌شود. مدیریت این مشکل در فازهای **Map** و **Reduce** با استراتژی زیر انجام می‌پذیرد:

1. تجمیع در فاز Map (کلید مجازی):

همان‌طور که در کد تابع **Map** مشاهده می‌کنید، اگر گرهی بن‌بست باشد، امتیاز خود را دور نمی‌ریزد؛ بلکه آن را با یک کلید ویژه و رزرو شده (مانند 'DANGLING_MASS') منتشر می‌کند.

2. محاسبه جرم گمشده جهانی (Global Dangling Mass):

در فاز **Reduce**، یک گره پردازشی (**Reducer**) خاص، تمام مقادیری که با کلید ویژه ارسال شده‌اند را دریافت کرده و با هم جمع می‌کند. این مقدار مجموع، نشان‌دهنده کل احتمالی است که در این تکرار از گراف نشت کرده است (فرض کنیم آن را M_D بنامیم).

3. بازتوزیع یکنواخت (Uniform Redistribution):

از آنجا که گره‌های بن‌بست معادل جهش تصادفی عمل می‌کنند، این جرم گمشده (M_D) باید به صورت کاملاً مساوی بین تمام N گره سیستم تقسیم شود.

در نتیجه، فرمول محاسبه **PageRank** در تابع **Reduce** (که در کد بالا نیز پیاده‌سازی شده) برای هر گره i به شکل ریاضی زیر اصلاح و تکمیل می‌شود:

$$PR_i = \underbrace{\frac{1 - \alpha}{N}}_{\text{Teleportation}} + \underbrace{\alpha \frac{M_D}{N}}_{\text{Dangling Mass}} + \underbrace{\alpha \sum_{j \rightarrow i} \frac{PR_j}{C_j}}_{\text{In-links Mass}}$$

در این معادله:

بخش اول: احتمال جهش تصادفی استاندارد (Teleportation).

بخش دوم: سهم جبران‌شده از گره‌های بن‌بست که به طور مساوی به همه داده شده است.

بخش سوم: امتیاز به دست آمده از لینک‌های ورودی واقعی گراف.

نمودار شماتیک: معماری پیشرفته جریان داده در MapReduce PageRank

فاز Reduce (تجمیع)



فاز Shuffle (گروه‌بندی)



فاز Map (خرد کردن)



نکته آموزشی

تأخیر یک مرحله‌ای (One-Step Delay):

در پیاده‌سازی واقعی خوشه‌های محاسباتی عظیم (مثل گوگل)، محاسبه دقیق M_D در همان تکراری که **PageRank** در حال محاسبه است، نیازمند متوقف کردن کل شبکه توزیع‌شده و یکسان‌سازی (**Synchronization**) است که یک گلوگاه محاسباتی فاجعه‌بار (**Bottleneck**) ایجاد می‌کند.

به همین دلیل، در مهندسی سیستم‌های مقیاس‌پذیر، معمولاً جرم بن‌بست محاسبه شده در تکرار t ، برای توزیع در تکرار $t + 1$ استفاده می‌شود. این تأخیر جزئی هیچ تأثیری در همگرایی نهایی الگوریتم (**Steady-State**) ندارد، اما سرعت پردازش توزیع‌شده را با حذف نیاز به همگام‌سازی ده‌ها برابر افزایش می‌دهد.

پاسخنامه مسئله 21.26: نقض ارگودیسیت در زنجیره مارکوف (Ergodicity Violation Scenario)

مقدمه و هدف مسئله

همگرایی الگوریتم‌های مبتنی بر گشت‌وگذار تصادفی (مانند **PageRank**) نیازمند برقراری شرط «ارگودیسیت» (**Ergodicity**) در گراف است. یکی از پایه‌های ارگودیک بودن، «نامتناوب بودن» (**Aperiodicity**) گراف است. هدف این مسئله، بررسی یک سناریوی حدی است که در آن بخشی از وب به شکل یک گراف دوگانه (**Bipartite**) تشکیل شده و اثبات اینکه چگونه این ساختار، ریاضیات روش تکرار توان (**Power Iteration**) را با نوسان و شکست مواجه می‌کند.

گام 1) مدل‌سازی ماتریس گذار برای گراف دوگانه (Bipartite)

فرض کنید بخش ایزوله‌ای از وب شامل 2 لایه (مجموعه) از صفحات است: مجموعه U و مجموعه V . بر اساس تعریف گراف دوگانه، هیچ لینکی درون اعضای مجموعه U یا درون اعضای مجموعه V وجود ندارد. تمامی لینک‌های خروجی از صفحات لایه U دقیقاً به لایه V می‌روند و بالعکس.

اگر ماتریس گذار P را بر اساس این دو لایه به صورت بلوکی (**Block Matrix**) بنویسیم، ساختار آن به شکل ریاضی زیر خواهد بود:

$$P = \begin{bmatrix} \mathbf{0} & P_{UV} \\ P_{VU} & \mathbf{0} \end{bmatrix}$$

تفسیر ساختاری: درایه‌های روی قطر اصلی صفر هستند، زیرا هیچ ارتباط درون‌گروهی ($U \rightarrow U$ یا $V \rightarrow V$) در این گراف وجود ندارد.

گام 2) اثبات وجود تناوب (Periodicity) در زنجیره مارکوف

رفتار یک موج‌سوار تصادفی را در این گراف بررسی می‌کنیم. فرض کنید موج‌سوار در گام صفر ($t = 0$) در یکی از صفحات لایه U قرار دارد.

گام 1: $t = 1$: موج‌سوار مجبور است از طریق لینک‌ها به لایه V برود.

گام 2: $t = 2$: موج‌سوار مجبور است از لایه V خارج شده و مجدداً به لایه U بازگردد.

از نظر ریاضی، اگر ماتریس P را به توان 2 برسانیم (وضعیت سیستم پس از 2 مرحله محاسباتی)، داریم:

$$P^2 = \begin{bmatrix} 0 & P_{UV} \\ P_{VU} & 0 \end{bmatrix} \begin{bmatrix} 0 & P_{UV} \\ P_{VU} & 0 \end{bmatrix} = \begin{bmatrix} P_{UV}P_{VU} & 0 \\ 0 & P_{VU}P_{UV} \end{bmatrix}$$

نتیجه اثبات: همان‌طور که می‌بینید، در ماتریس P^2 درایه‌های غیرقطری صفر شده‌اند. این یعنی احتمال بازگشت به گره اولیه تنها در گام‌های زوج ($t = 2, 4, 6, \dots$) غیرصفر است. بنابراین، این زنجیره مارکوف دارای **تناوب 2 (Period = 2)** است و یک زنجیره مارکوف متناوب (**Periodic**) محسوب می‌شود.

نمودار شماتیک: ساختار دوگانه و رفتار نوسانی (Oscillation) در گراف



گام 3) فروپاشی همگرایی در متد تکرار توان (Power Iteration)

متد **Power Iteration** بر اساس رابطه $\pi^{(t+1)} = \pi^{(t)}P$ کار می‌کند. برای اینکه این دنباله به یک بردار ثابت همگرا شود، ماتریس P باید دقیقاً یک «مقدار ویژه غالب» (**Dominant Eigenvalue**) داشته باشد؛ یعنی:

$$|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$$

اما در ماتریس‌های متناوبِ گراف‌های دوگانه، این شرط جبری نقض می‌شود:

1. ماتریس P در این حالت دارای 2 مقدار ویژه با قدر مطلق برابر است: $\lambda_1 = 1$ و $\lambda_2 = -1$.

2. به دلیل وجود مقدار ویژه منفی (-1)، توزیع احتمالات هرگز تثبیت نمی‌شود.

3. رفتار نوسانی: اگر در ابتدا تمام امتیاز **PageRank** را در لایه U قرار دهیم، در تکرار اول تمام امتیاز به لایه V منتقل

می‌شود. در تکرار دوم، کل امتیاز دوباره به لایه U برمی‌گردد.

نتیجه نهایی: بنابراین، متد **Power Iteration** در یک نوسان (**Oscillation**) بی‌نهایت بین 2 بردار متفاوت گیر می‌افتد و

سیستم هرگز به یک توزیع پایدار (**Steady-State Distribution**) همگرا نخواهد شد.

نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

تله‌پورت به عنوان شکننده تناوب (Periodicity Breaker):

این مسئله به خوبی نشان می‌دهد که بردار تله‌پورت در الگوریتم **PageRank** تنها برای نجات موج‌سوار از بن‌بست‌ها طراحی نشده است. زمانی که ما ماتریس گوگل (G) را با فرمول $G = \alpha P + (1 - \alpha)E$ می‌سازیم، در واقع در حال اضافه کردن مقادیر مثبت (ولو بسیار کوچک) به تمامی درایه‌های ماتریس، از جمله درایه‌های روی قطر اصلی ($G_{ii} > 0$) هستیم.

از نظر نظریه گراف، این کار معادل قرار دادن یک «حلقه به خود» (**Self-loop**) روی تمامی صفحات وب است. وجود حتی 1 حلقه به خود در یک گراف متصل، کافی است تا تناوب آن از بین برود (تغییر **Period** به 1) و گراف کاملاً «نامتناوب» (**Aperiodic**) شود. در نتیجه این ترفند هوشمندانه، ارگودیسیتته احیا شده و همگرایی ریاضی الگوریتم تضمین می‌گردد.

پاسخنامه مسئله 21.27: بهینه‌سازی بردار پرش تصادفی (Optimizing Random Jump Vector)

مقدمه و هدف مسئله

الگوریتم استاندارد **PageRank**، گراف اینترنت را به صورت دموکراتیک و کاملاً بی‌طرفانه ارزیابی می‌کند؛ به این معنا که در زمان «جهش تصادفی» (**Teleportation**)، احتمال پرش موج‌سوار به تمام صفحات اینترنت برابر است. اما این بی‌طرفی، راه را برای سوءاستفاده سئوکاران کلاه سیاه (تولیدکنندگان وب اسپم) باز می‌کند. هدف این مسئله، ارتقای ریاضی این الگوریتم با جایگزینی «پرش یکنواخت» با «پرش مبتنی بر کیفیت» است تا سیستمی مقاوم در برابر اسپم (**Spam-resistant**) طراحی شود.

گام 1) تحلیل ریاضی پرش یکنواخت در مدل کلاسیک

در فرمول کلاسیک **PageRank**، بخش مربوط به پرش تصادفی با عبارت $\frac{1-d}{N}$ نمایش داده می‌شود:

$$PR(A) = \frac{1-d}{N} + d \sum_{j \in M(A)} \frac{PR(j)}{C(j)}$$

در این حالت، بردار پرش تصادفی (**Teleportation Vector**) که آن را e می‌نامیم، یک توزیع یکنواخت است. یعنی برای گراف با N گره، احتمال فرود موج‌سوار روی صفحه A دقیقاً برابر با $\frac{1}{N}$ است، فارغ از اینکه صفحه A یک مرجع علمی معتبر باشد یا یک صفحه بی‌محتوا و اسپم.

گام 2) معرفی بردار TrustScore (کیفیت محتوا)

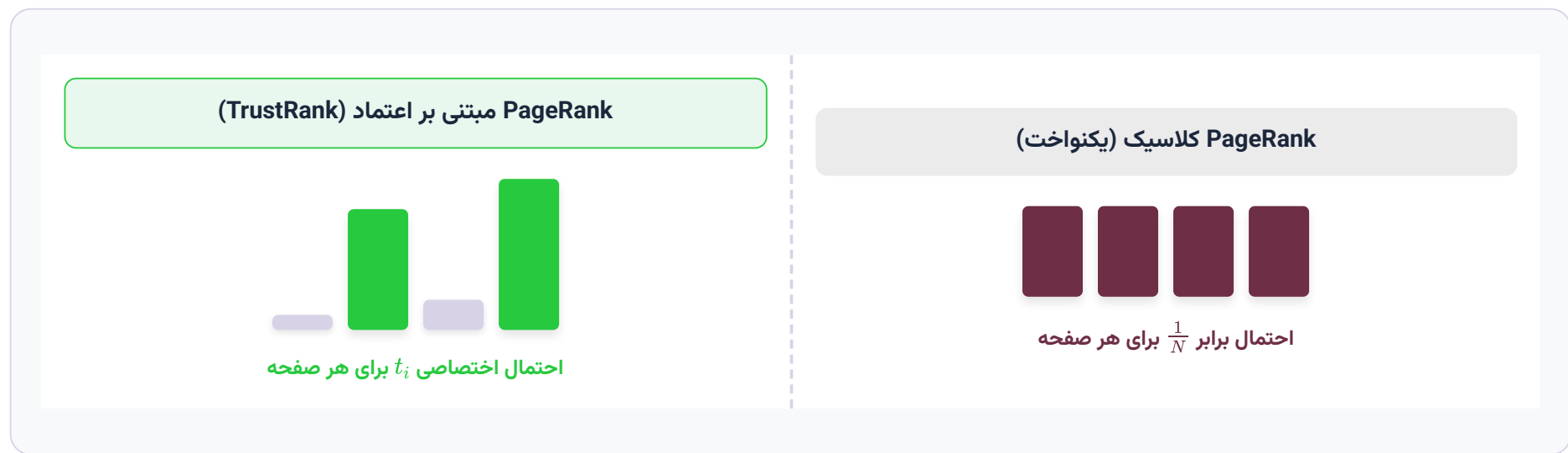
در مدل بهینه‌سازی شده، ما بردار e را با بردار جدیدی به نام t (**TrustScore Vector**) جایگزین می‌کنیم. درایه متناظر با صفحه A در این بردار را t_A می‌نامیم. این مقدار نشان‌دهنده «کیفیت ذاتی و میزان اعتماد» به آن صفحه است (که می‌تواند توسط ارزیاب‌های انسانی، تحلیل الگوریتم‌های پردازش زبان طبیعی، یا لیست سفید دامنه‌های معتبر مانند **.edu** و **.gov** تعیین شود).

شرط ریاضی توزیع احتمال: برای حفظ خاصیت تصادفی بودن (**Stochastic**) ماتریس گذار، مجموع مقادیر

TrustScore برای تمام صفحات شبکه باید دقیقاً برابر با 1 باشد:

$$\sum_{i=1}^N t_i = 1$$

نمودار شماتیک: مقایسه تله‌پورت یکنواخت در برابر تله‌پورت هوشمند



گام 3) بازنویسی فرمول بهینه‌شده PageRank

اکنون با جایگزین کردن توزیع یکنواخت $\frac{1}{N}$ با مقدار اختصاصی t_A در فرمول پایه، سیستم معادلات رتبه‌بندی به شکل زیر بازنویسی می‌شود:

$$PR_t(A) = (1 - d) \cdot t_A + d \sum_{j \in M(A)} \frac{PR_t(j)}{C(j)}$$

عبارت نخست $(1 - d) \cdot t_A$: احتمال اینکه موج‌سوار تصادفی به دلیل خستگی، مستقیماً به صفحه A پرش کند (که اکنون کاملاً وابسته به کیفیت صفحه A است).

عبارت دوم $d \sum \dots$: احتمال اینکه موج‌سوار از طریق لینک‌های موجود در صفحات دیگر به صفحه A برسد.

گام 4) استدلال سیستمی برای مبارزه با وب اسپم

چرا این تغییر ریاضی باعث شکست خوردن اسپمرها می‌شود؟ 3 دلیل مهندسی برای آن وجود دارد:

1. **خنثی‌سازی مزارع لینک (Link Farms):** اسپمرها می‌توانند با ساخت هزاران صفحه مصنوعی و لینک دادن آن‌ها به

یکدیگر، بخش دوم معادله ($\sum \dots$) را متورم کنند. اما آن‌ها هیچ کنترلی روی بخش اول معادله یعنی t_A ندارند.

سیستم، امتیاز **TrustScore** صفحات اسپم را به صورت پیش‌فرض 0 ($t_{spam} = 0$) در نظر می‌گیرد.

2. قطع تزریق احتمال به شبکه‌های ایزوله: در مدل کلاسیک، چون پرش یکنواخت بود، موج‌سوار با احتمال $\frac{1}{N}$ وارد

صفحات اسپم می‌شد و از آنجا چرخش در مزرعه لینک آغاز می‌گردید. اما در فرمول جدید، موج‌سوار پس از هر پرش،

تنها روی صفحات باکیفیت و معتبر فرود می‌آید.

3. جریان و نفوذ اعتبار (Flow of Trust): در این مدل، اعتبار از هسته صفحات معتبر ($t_i > 0$) سرچشمه می‌گیرد و

از طریق لینک‌های خروجی آن‌ها در شبکه توزیع می‌شود. از آنجا که سایت‌های معتبر و دانشگاهی هرگز به مزارع لینک

یا سایت‌های اسپم ارجاع نمی‌دهند، هیچ مسیری (یالی) برای انتقال این اعتبار از بخش سالم وب به بخش اسپم وجود

نخواهد داشت. در نتیجه، رتبه صفحات اسپم پس از چند تکرار الگوریتم، به سمت 0 میل می‌کند.

نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

الگوریتم TrustRank:

معادله‌ای که در این مسئله اثبات و بازنویسی کردید، هسته مرکزی الگوریتمی به نام TrustRank است که در سال 2004 توسط محققان دانشگاه استنفورد و یاهو برای مقابله با اسپم معرفی شد.

منطق بنیادین TrustRank بر یک اصل ساده استوار است: "اعتماد (Trust) از طریق لینک‌ها منتقل می‌شود، اما سایت‌های خوب به سایت‌های بد لینک نمی‌دهند." با تزریق این فرض به ریاضیات ماتریس گذار، موتورهای جستجو توانستند شبکه‌های مخرب را بدون نیاز به بررسی محتوای تک‌تک آن‌ها، شناسایی و بی‌اثر کنند.

پاسخنامه مسئله 21.28: رتبه صفحه موضوعی در لحظه (Real-Time Topic-Specific PR)

مقدمه و هدف مسئله

محاسبه الگوریتم PageRank برای میلیاردها صفحه وب نیازمند منابع پردازشی عظیم و زمان طولانی (فاز آفلاین) است. اگر بخواهیم برای هر کاربر، بر اساس ترکیب منحصربه‌فرد علایق او (مثلاً 60% ورزشی، 40% تکنولوژی)، یک بردار تله‌پورت جدید بسازیم و PageRank را از ابتدا (در لحظه) محاسبه کنیم، سیستم بلافاصله دچار فروپاشی پردازشی خواهد شد.

هدف این مسئله، معرفی یک شاهکار مهندسی بر پایه جبر خطی است: استفاده از خاصیت «خطی بودن» (Linearity) معادله PageRank برای تجزیه این محاسبات سنگین به بردارهای پایه (Basis Vectors) و انجام یک جمع برداری ساده در زمان اجرای کوئری.

گام 1) فاز آفلاین (پیش‌محاسبه بردارهای پایه)

به جای محاسبه PageRank برای ترکیب‌های بی‌نهایت کاربران، ما صرفاً برای موضوعات اصلی (مثلاً 100 موضوع پایه مانند ورزش، تکنولوژی، پزشکی و غیره) محاسبات را انجام می‌دهیم.

1. فرض کنید 100 بردار تله‌پورت پایه داریم: $v_1, v_2, \dots, v_{100}$. (هر بردار v_i فقط به صفحات همان موضوع اشاره می‌کند).

2. موتور جستجو در فاز آفلاین، الگوریتم سنگین PageRank را 100 بار به طور مجزا اجرا می‌کند تا توزیع حالت پایدار برای هر موضوع به دست آید.

3. خروجی این فاز، 100 بردار رتبه صفحه (PageRank Vector) است که آن‌ها را $\pi_1, \pi_2, \dots, \pi_{100}$ می‌نامیم. این‌ها

بردارهای پایه (Basis Vectors) ما هستند و در حافظه سریع (RAM) سرورها ذخیره می‌شوند.

گام 2) فاز آنلاین (مدل‌سازی پروفایل کاربر)

هنگامی که کاربری وارد سیستم می‌شود، پروفایل او به صورت یک ترکیب خطی (Linear Combination) از موضوعات پایه مدل‌سازی می‌شود.

1. فرض کنید کاربر U دارای علائقی با وزن‌های $w_1, w_2, \dots, w_{100}$ است (مثلاً $w_{sports} = 0.6$ و $w_{tech} = 0.4$ و بقیه 0).

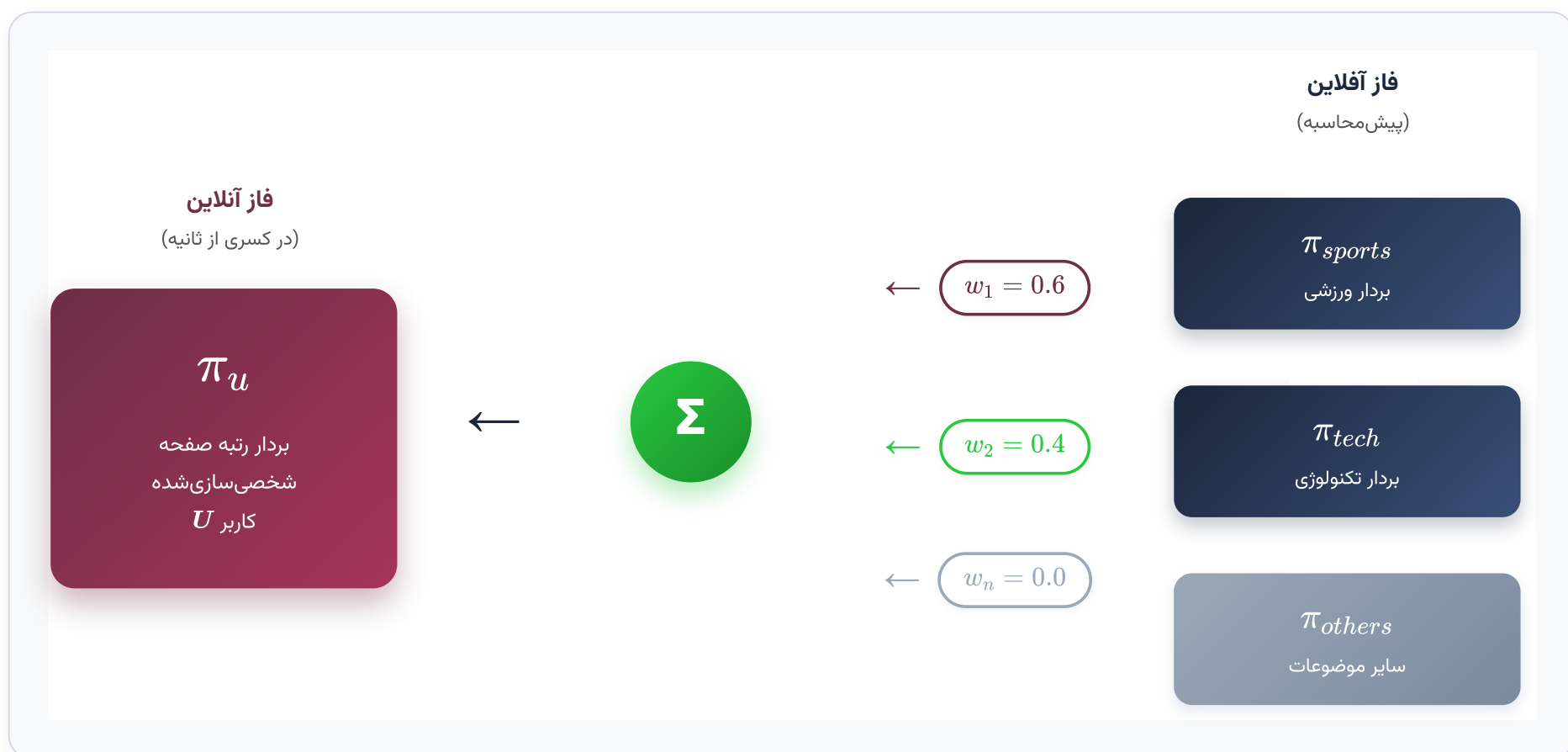
2. شرط ریاضی این است که مجموع وزن‌ها 1 باشد:

$$\sum_{i=1}^{100} w_i = 1$$

3. بردار تله‌پورت شخصی‌سازی‌شده برای این کاربر (v_u) دقیقاً همان ترکیب خطی بردارهای تله‌پورت پایه است:

$$v_u = \sum_{i=1}^{100} w_i v_i$$

نمودار شماتیک گرافیکی: فلوچارت ترکیب خطی بردارهای رتبه صفحه



گام 3) محاسبه در لحظه و خطی (Real-Time Computation)

جادوی ریاضی این روش در اینجا رخ می‌دهد: برای یافتن بردار PageRank نهایی این کاربر (π_u)، نیازی به اجرای مجدد الگوریتم گراف روی بردار v_u نیست. کافی است بردارهای پایه از پیش محاسبه شده را با همان وزن‌های کاربر، با یکدیگر جمع کنیم:

$$\pi_u = \sum_{i=1}^{100} w_i \pi_i$$

این عملیات در لحظه، تنها یک ضرب و جمع اسکالر ساده با پیچیدگی زمانی $O(N)$ است که در کسری از ثانیه انجام می‌شود.

گام 4) اثبات ریاضی خطی بودن ترکیب

برای اثبات صحت این روش، باید نشان دهیم که ترکیب خطی بردارهای پایه π (یعنی $\sum w_i \pi_i$)، دقیقاً در معادله اصلی **PageRank** برای کاربر با بردار تله‌پورت v_u صدق می‌کند.

1. صورت معادلات پایه:

می‌دانیم که هر بردار پایه π_i ، جواب معادله **PageRank** برای تله‌پورت v_i است:

$$\pi_i = \alpha \pi_i P + (1 - \alpha) v_i$$

2. صورت معادله کاربر هدف:

معادله **PageRank** برای بردار تله‌پورت شخصی‌سازی شده کاربر (v_u) باید به شکل زیر باشد:

$$\pi_u = \alpha \pi_u P + (1 - \alpha) v_u$$

3. اثبات با جایگذاری (Substitution):

فرض می‌کنیم ترکیب خطی بردارهای پایه را $\pi^* = \sum w_i \pi_i$ بنامیم. این مقدار را در سمت راست (**RHS**) معادله کاربر هدف قرار می‌دهیم تا ببینیم آیا سمت چپ معادله به دست می‌آید یا خیر. با جایگذاری v_u با معادل خطی آن ($\sum w_i v_i$):

$$\text{RHS} = \alpha \left(\sum w_i \pi_i \right) P + (1 - \alpha) \left(\sum w_i v_i \right)$$

به دلیل خاصیت توزیع‌پذیری (**Distributive Property**) در ضرب ماتریس‌ها، می‌توانیم عملگر جمع ($\sum$) و ضرایب اسکالر (w_i) را از ماتریس P و عبارات خارج کنیم:

$$\text{RHS} = \sum w_i (\alpha \pi_i P) + \sum w_i ((1 - \alpha) v_i)$$

اکنون از w_i فاکتور می‌گیریم:

$$\text{RHS} = \sum w_i \left[\alpha \pi_i P + (1 - \alpha) v_i \right]$$

همان‌طور که در «صورت معادلات پایه» دیدیم، عبارت داخل کروشه دقیقاً برابر با π_i است. پس آن را جایگزین می‌کنیم:

$$\text{RHS} = \sum w_i \pi_i = \pi^*$$

نتیجه‌گیری اثبات: نشان دادیم که $\pi^* = \alpha \pi^* P + (1 - \alpha) v_u$ از آنجا که قضیه مارکوف ثابت کرده است توزیع حالت پایدار برای یک گراف با پرش تصادفی (تله‌پورت) **یکتا (Unique)** است، پس π^* لزوماً همان π_u (بردار رتبه صفحه نهایی کاربر) است. حکم ثابت شد.

نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

تغییر پارادایم از گراف به بردار:

این اثبات ریاضی نشان می‌دهد که چگونه می‌توان یک مسئله پیچیده و سنگین «پیمایش گراف» (Graph Traversal) را به یک مسئله بسیار سبک و سریع «جمع برداری» (Vector Addition) تبدیل کرد.

این تکنیک، پایه و اساس بسیاری از سیستم‌های توصیه‌گر (**Recommender Systems**) مدرن است که می‌توانند نتایج را به صورت آنی (**Real-time**) و مقیاس‌پذیر برای میلیون‌ها کاربر با سلاقی بی‌نهایت متنوع، محاسبه و شخصی‌سازی کنند.

پاسخنامه مسئله 21.29: تحلیل مقادیر ویژه در هیتس (Eigenvalue Analysis in HITS)

مقدمه و هدف مسئله

الگوریتم **HITS (Hubs and Authorities)** در نگاه اول مجموعه‌ای از جمع‌های ساده به نظر می‌رسد که در یک حلقه تکرار می‌شوند. اما در پس‌زمینه، این الگوریتم ریشه عمیقی در «نظریه طیفی گراف» (**Spectral Graph Theory**) و جبر خطی دارد. هدف این مسئله، اثبات این موضوع است که روابط تکرارشونده **HITS** در واقع فرمول‌های پنهانی برای یافتن مقادیر ویژه یک ماتریس بسیار خاص هستند و تحلیل این ماتریس، ماهیت گراف‌کاوی الگوریتم را افشا می‌کند.

گام 1) جایگذاری معادلات و کشف ماتریس پنهان

ابتدا معادلات به‌روزرسانی (**Update Rules**) در الگوریتم **HITS** را به زبان ماتریسی می‌نویسیم. فرض کنید A ماتریس مجاورت (**Adjacency Matrix**) گراف باشد. در این ماتریس، درایه $A_{ij} = 1$ است اگر از گره i به گره j یال وجود داشته باشد و در غیر این صورت 0 است.

معادلات در تکرار t به شکل زیر تعریف می‌شوند:

1. به‌روزرسانی Hub:

$$h^{(t)} = A a^{(t-1)}$$

2. به‌روزرسانی Authority:

$$a^{(t)} = A^T h^{(t)}$$

اکنون، معادله h را در معادله a جایگذاری می‌کنیم تا مقدار $a^{(t)}$ را مستقیماً بر حسب مقدار قبلی خودش ($a^{(t-1)}$) به دست آوریم:

$$a^{(t)} = A^T(Aa^{(t-1)}) = (A^T A)a^{(t-1)}$$

نتیجه‌گیری اولیه: این معادله دقیقاً همان فرمول استاندارد روش «تکرار توان» (Power Iteration) در جبر خطی برای ماتریس $(A^T A)$ است. بنابراین، وقتی الگوریتم **HITS** همگرا می‌شود، بردار **Authority** (a) در معادله زیر صدق خواهد کرد:

$$(A^T A)a = \lambda a$$

این یعنی بردار **Authority**، در واقع بردار ویژه (Eigenvector) متناظر با بزرگترین مقدار ویژه (Principal Eigenvalue) برای ماتریس $A^T A$ است.

(به طور مشابه با جایگذاری برعکس، ثابت می‌شود که بردار **Hub** (h)، بردار ویژه اصلی برای ماتریس AA^T است).

گام 2) تقارن ماتریس $A^T A$

در جبر خطی، بررسی تقارن این ماتریس بسیار مهم است:

$$(A^T A)^T = A^T (A^T)^T = A^T A$$

از آنجا که ماتریس $A^T A$ یک ماتریس متقارن (**Symmetric**) و نیمه‌معین مثبت (**Positive Semi-definite**) است، قضیه طیفی (**Spectral Theorem**) تضمین می‌کند که مقادیر ویژه آن همگی حقیقی و نامنفی بوده و بردارهای ویژه آن بر هم عمود (**Orthogonal**) هستند. این ویژگی جبری، همگرایی قطعی ریاضی الگوریتم **HITS** را تضمین می‌کند.

گام 3) بینش گراف‌کاوی (تفسیر معنایی ماتریس‌ها)

مهم‌ترین بخش این تحلیل، تفسیر این ریاضیات در دنیای واقعی وب و گراف‌کاوی است. ماتریس‌های $A^T A$ و AA^T در تحلیل شبکه‌های استنادی (**Citation Networks**) مفاهیم بسیار مشهوری دارند:

1. ماتریس $A^T A$ (ماتریس هم‌ارجاعی - Co-citation Matrix):

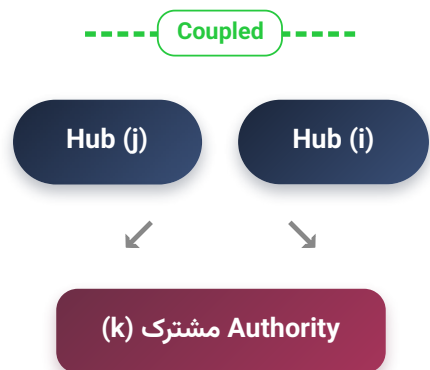
اگر عناصر این ماتریس را بررسی کنیم، درایه $(A^T A)_{ij}$ نشان‌دهنده تعداد گره‌هایی است که همزمان هم به i و هم به j لینک داده‌اند. وقتی ثابت کردیم بردار **Authority** بردار ویژه این ماتریس است، این بینش عمیق به دست می‌آید: الگوریتم **HITS** صرفاً صفحاتی که لینک زیاد دارند را پیدا نمی‌کند؛ بلکه متراکم‌ترین کلاستر (خوشه) از صفحاتی را پیدا می‌کند که توسط **Hub**های مشترک، به صورت همزمان ارجاع داده شده‌اند (Co-cited).

2. ماتریس AA^T (ماتریس اشتراک مراجع - Bibliographic Coupling):

درایه $(AA^T)_{ij}$ نشان‌دهنده تعداد گره‌هایی است که هم گره i و هم گره j مشترکاً به آن‌ها لینک داده‌اند. بنابراین یک **Hub** خوب، صفحه‌ای است که بیشترین اشتراک منابع (لینک‌های خروجی یکسان) را با سایر **Hubs**های قدرتمند شبکه داشته باشد.

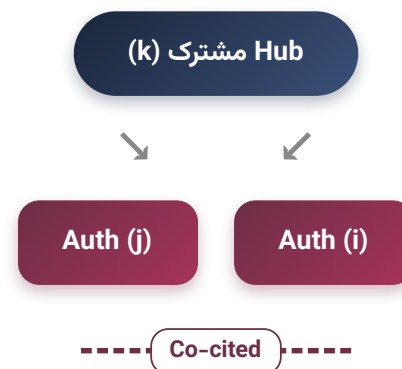
ماتریس اشتراک مراجع AA^T

(Bibliographic Coupling – هم‌گرایی Hubs)



ماتریس هم‌ارجاعی $A^T A$

(Co-citation – کشف Authorities متراکم)



نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

کشف جوامع متراکم (Dense Bipartite Cores):

این اثبات ریاضی نشان می‌دهد که الگوریتم **HITS** در ذات خود یک الگوریتم خوشه‌بندی (**Clustering**) است. اگر کاربر کلمه‌ای با 2 معنی متفاوت را جستجو کند (مثلاً کلمه "جاوا" که هم یک جزیره و هم یک زبان برنامه‌نویسی است)، گراف پایه شامل دو خوشه کاملاً مجزا از صفحات خواهد شد.

از آنجا که **HITS** بر اساس قانون تکرار توان به سمت «بزرگترین» مقدار ویژه ماتریس هم‌ارجاعی ($A^T A$) همگرا می‌شود، الگوریتم به صورت خودکار خوشه‌ای که متراکم‌تر است (تعداد صفحات و پیوندهای مشترک بیشتری دارد - غالباً زبان برنامه‌نویسی) را انتخاب کرده و بردار **Authority** را منحصراً روی آن خوشه متمرکز می‌کند، در حالی که خوشه دیگر (جزیره جاوا) امتیاز ریاضیاتی نزدیک به 0 دریافت خواهد کرد.

پاسخنامه مسئله 21.30: سناریوی فیلترینگ مجموعه پایه (Base Set Filtering Scenario)

مقدمه و هدف مسئله

هنگامی که یک کلمه دارای ابهام معنایی (**Polysemy**) است، مجموعه پایه (**Base Set**) در الگوریتم **HITS** به جای یک گراف یکپارچه، تبدیل به 2 یا چند زیرگراف (جامعه) تقریباً مجزا می‌شود که پیوند بسیار کمی با یکدیگر دارند (مثلاً جامعه سایت‌های فناوری در برابر جامعه سایت‌های کشاورزی و تغذیه).

هدف این مسئله، بررسی نحوه استفاده از «نظریه طیفی گراف» (**Spectral Graph Theory**) برای استخراج این جوامع پنهان و ابهام‌زدایی از کوئری کاربر است.

گام 1) سلطه بردار ویژه اصلی (The Principal Eigenvector)

همان‌طور که در مسائل قبلی اثبات شد، فرمول استاندارد **HITS** همواره به سمت بردار ویژه اصلی (متناظر با بزرگترین مقدار ویژه λ_1) ماتریس هم‌ارجایی $A^T A$ همگرا می‌شود.

در گراف وب، معمولاً یکی از معانی یک کلمه، حضور پررنگ‌تری دارد. از آنجا که سایت‌های مربوط به کمپانی اپل (تکنولوژی) بسیار بیشتر از سایت‌های میوه اپل به هم لینک داده‌اند، این جامعه به یک خوشه به شدت متراکم (**Dense Core**) تبدیل می‌شود.

نتیجه محاسباتی: بردار ویژه اصلی (a_1) تمام وزن و امتیاز **Authority** خود را روی سایت‌های تکنولوژی متمرکز می‌کند و امتیاز سایت‌های میوه در این بردار به سمت 0 میل می‌کند. در نتیجه، در خروجی استاندارد **HITS**، معنای دوم (میوه) کاملاً سرکوب شده و حذف می‌شود.

گام 2) نقش بردارهای ویژه غیرنخستین (Non-principal Eigenvectors)

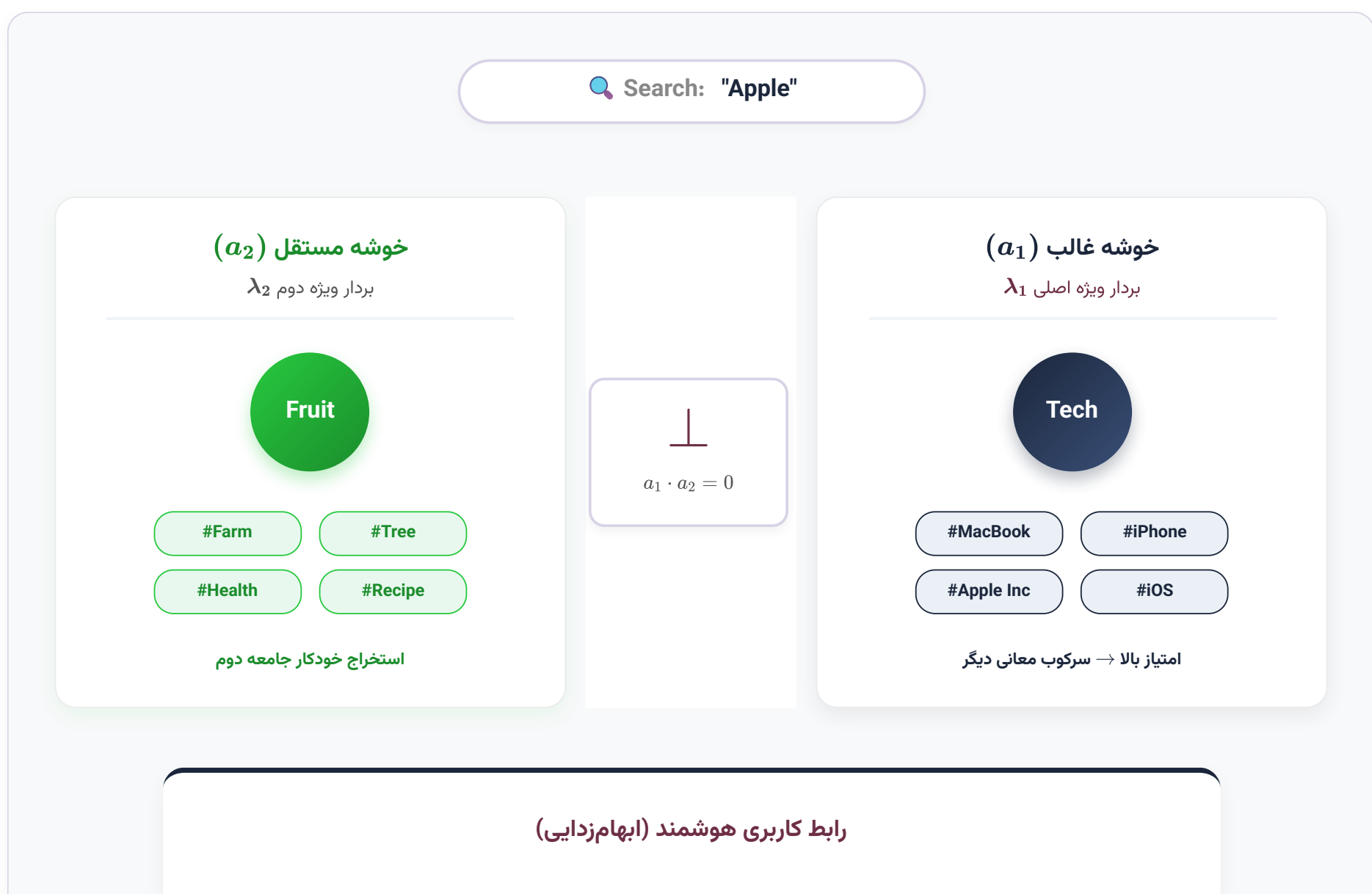
از جبر خطی می‌دانیم که ماتریس $A^T A$ یک ماتریس متقارن است. یکی از مهم‌ترین قضایای ماتریس‌های متقارن این است که بردارهای ویژه آن بر یکدیگر عمود (Orthogonal) هستند. این تعامد ریاضی، در گراف‌کاوی به معنای «استقلال موضوعی» است:

تعامد بردارهای ویژه: اگر بردار a_1 (فناوری) و بردار a_2 (متناظر با دومین مقدار ویژه بزرگ λ_2) بر هم عمود باشند ($a_1 \cdot a_2 = 0$)، بدین معناست که درایه‌هایی (صفحاتی) که در a_1 مقدار بالایی دارند، در a_2 باید مقداری نزدیک به 0 داشته باشند و بالعکس.

کشف جامعه مستقل: بنابراین، بردار ویژه دوم (a_2) به طور طبیعی و خودکار، دومین جامعه متراکم در گراف را پیدا می‌کند که از جامعه اول کاملاً مستقل است. در این سناریو، a_2 بالاترین امتیازات **Authority** را به سایت‌های «میوه و کشاورزی» اختصاص می‌دهد.

تعمیم به معانی بیشتر: به همین ترتیب، بردار ویژه سوم (a_3) ممکن است جامعه مستقل دیگری (مثلاً فیلمی با نام اپل یا یک شرکت ناشناخته دیگر) را استخراج کند.

نمودار شماتیک گرافیکی: کالبدشکافی طیفی گراف برای ابهام‌زدایی



🍏 کشاورزی و تغذیه (Fruit)

📱 تکنولوژی (iPhone/Mac)

گام 3) تحلیل علامت‌ها و تفکیک جوامع (Spectral Partitioning)

در بردارهای ویژه دوم به بعد، درایه‌ها می‌توانند مقادیر منفی یا مثبت داشته باشند. الگوریتم از این ویژگی جبری برای دسته‌بندی استفاده می‌کند:

1. در بردار a_2 ، صفحاتی که بالاترین مقادیر مثبت (مثلاً $x > 0.5$) را دارند، به عنوان هسته اصلی معنای دوم استخراج می‌شوند.

2. صفحاتی که بالاترین مقادیر منفی (مثلاً $x < -0.5$) را دارند، ممکن است نشان‌دهنده یک قطبیت دیگر یا یک زیرمجموعه متضاد در همان خوشه باشند.

3. صفحاتی که مقدارشان حول و حوش 0 است، به این موضوع تعلق ندارند (این‌ها همان صفحات تکنولوژی هستند که در a_1 فعال بودند).

گام 4) طراحی معماری سیستم برای کاربر (User UI Architecture)

برای ارائه قابلیت انتخاب به کاربر، موتور جستجو به شکل زیر عمل می‌کند:

1. ماتریس $A^T A$ را برای **Base Set** تشکیل می‌دهد.

2. سه بردار ویژه اول (a_1, a_2, a_3) را به صورت ریاضی محاسبه می‌کند.

3. از صفحاتی که در a_1 بالاترین امتیاز را دارند، کلمات کلیدی پرتکرار (مثل: **iPhone, Mac, iOS**) را استخراج می‌کند.

4. از صفحاتی که در a_2 بالاترین امتیاز را دارند، کلمات کلیدی پرتکرار (مثل: **Tree, Farm, Recipe, Health**) را استخراج می‌کند.

5. در صفحه نتایج، موتور جستجو از کاربر می‌پرسد:

"آیا منظور شما اپل (تکنولوژی - iPhone) است یا اپل (میوه و کشاورزی)؟"

و با کلیک کاربر، نتایج استخراج شده از بردار ویژه مربوطه را به او نمایش می‌دهد.

نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

از رتبه‌بندی تا تقطیر موضوعی (Topic Distillation):

این مسئله نشان می‌دهد که ارزش واقعی الگوریتم **HITS** تنها در رتبه‌بندی صفحات نیست، بلکه یک ابزار قدرتمند برای **تقطیر موضوعی** است.

در مقاله اصلی جان کلاینبِرگ (خالق **HITS**)، دقیقاً به همین قابلیت استثنایی اشاره شده است: کاوش در فضاها و ویژه (**Eigenspaces**) ماتریس پیوندها، می‌تواند ساختار پنهان و چندمعنایی شبکه وب را بدون نیاز به هیچ‌گونه پردازش زبان طبیعی (**NLP**) و صرفاً با اتکا به توپولوژی لینک‌ها کشف و استخراج کند.

پاسخنامه مسئله 21.31: حمله سیبل در گراف وب (Sybil Attack in Web Graph)

مقدمه و هدف مسئله

«حمله سیبل» (**Sybil Attack**) در شبکه‌های کامپیوتری و گراف وب زمانی رخ می‌دهد که یک مهاجم با ایجاد تعداد بسیار زیادی هویت جعلی (گره‌های مصنوعی یا صفحات بی‌ارزش)، تلاش می‌کند تا سیستم رتبه‌بندی را فریب داده و نفوذ یک گره هدف (**Target**) را به صورت غیرطبیعی افزایش دهد. هدف این مسئله، تحلیل مقاومت 2 الگوریتم کلاسیک **PageRank** و **HITS** در برابر این نوع حمله و مقایسه آسیب‌پذیری آن‌هاست.

گام 1) تحلیل رفتار **PageRank** در برابر حمله سیبل

الگوریتم **PageRank** بر پایه توزیع سراسری اعتبار (**Global Trust**) کار می‌کند. در این مدل، ارزش یک لینک خروجی، وابستگی مستقیمی به اعتبار صفحه مبدأ دارد.

فرض کنید مهاجم M گره جعلی (**Sybil**) ساخته است که همگی فقط به صفحه هدف (T) لینک داده‌اند. از آنجا که این گره‌های جعلی جدید هستند و هیچ سایت معتبری (از بخش سالم وب) به آن‌ها لینک نداده است، امتیاز **PageRank** آن‌ها در گراف تنها به اندازه حداقل احتمال جهش تصادفی (**Teleportation**) خواهد بود:

$$PR(\text{Sybil}_i) \approx \frac{1-d}{N}$$

تأثیر این M گره جعلی بر روی صفحه هدف به شکل زیر محاسبه می‌شود:

$$PR_{fake}(T) = d \sum_{i=1}^M PR(\text{Sybil}_i) = d \cdot M \cdot \left(\frac{1-d}{N} \right)$$

نتیجه برای **PageRank:** از آنجا که مخرج کسر (N ، تعداد کل صفحات وب) عددی نجومی (مثلاً میلیاردها) است، ارزش عبارت $\frac{1-d}{N}$ به شدت ناچیز است. بنابراین، حتی اگر مهاجم ده‌ها هزار نود جعلی (M) بسازد، تا زمانی که این نودها از سایت‌های معتبر اعتباری دریافت نکنند، تأثیر آن‌ها بر روی صفحه هدف تقریباً 0 خواهد بود. **PageRank** به دلیل داشتن دید «سراسری»، مقاومت بسیار بالایی در برابر افزایش صرف «تعداد» لینک‌های جعلی دارد.

گام 2) تحلیل رفتار **HITS** در برابر حمله سیبل

الگوریتم **HITS** بر روی یک زیرگراف کوچک محلی به نام «مجموعه پایه» (**Base Set**) اجرا می‌شود و به جای توزیع اعتبار، از خاصیت «تقویت متقابل» (**Mutual Reinforcement**) بین **Hub** و **Authority** استفاده می‌کند.

اگر مهاجم بتواند کلمات کلیدی را در M صفحه جعلی خود بگنجانند تا این صفحات وارد **Base Set** شوند (و همگی به صفحه هدف T لینک دهند)، فاجعه ریاضی زیر رخ می‌دهد. در هر تکرار از الگوریتم **HITS**:

1. امتیاز **Authority** صفحه هدف، برابر با مجموع امتیازات **Hub** صفحات جعلی می‌شود:

$$a^{(t)}(T) = \sum_{i=1}^M h^{(t-1)}(\text{Sybil}_i)$$

2. امتیاز **Hub** هر صفحه جعلی، برابر با امتیاز **Authority** صفحه هدف می‌شود (زیرا فقط به آن لینک داده‌اند):

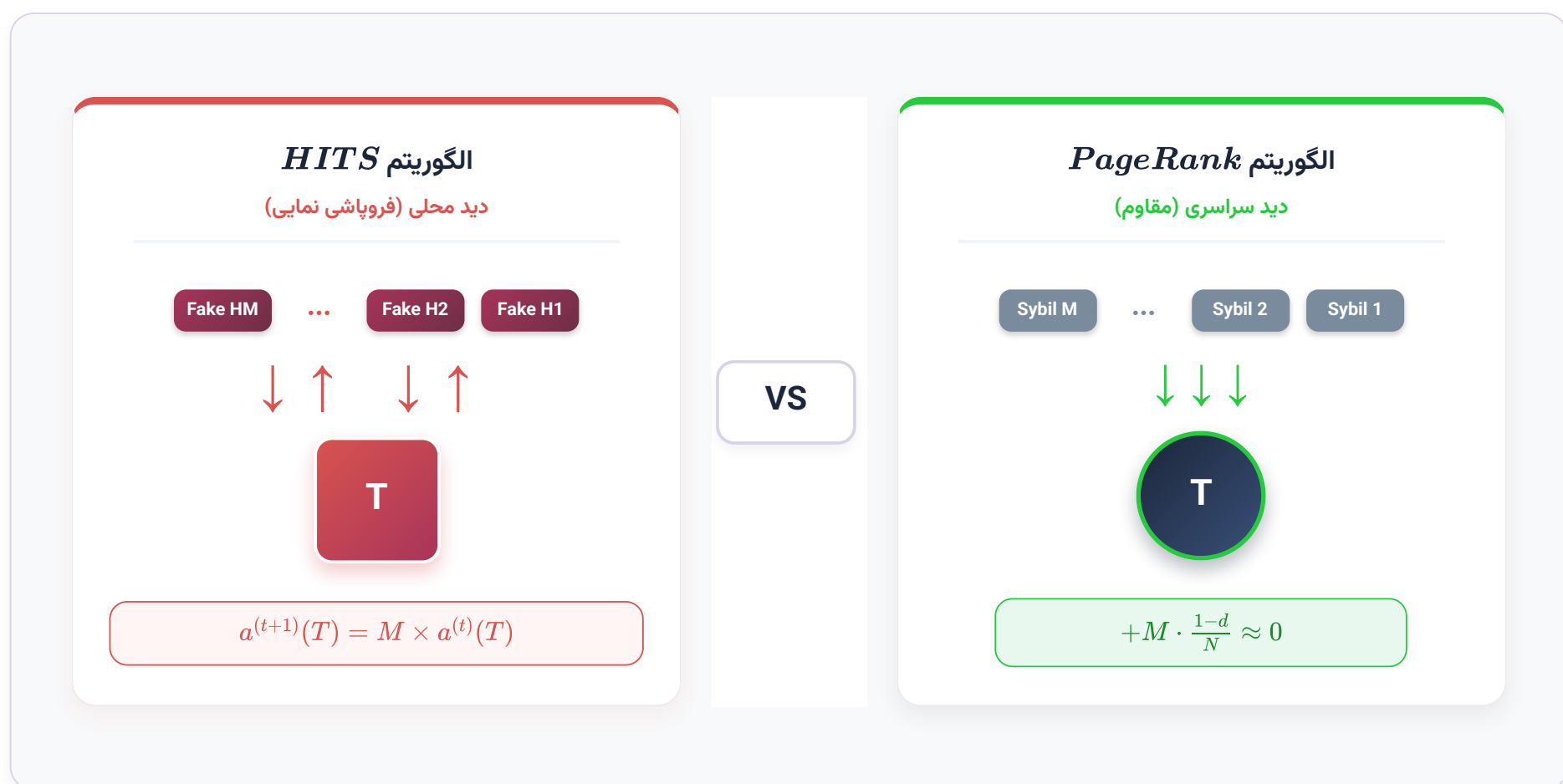
$$h^{(t)}(\text{Sybil}_i) = a^{(t)}(T)$$

با جایگذاری معادله دوم در معادله اول برای مرحله بعدی ($t + 1$) قانون توانی مخرب به دست می‌آید:

$$a^{(t+1)}(T) = M \times a^{(t)}(T)$$

نتیجه برای HITS: امتیاز **Authority** صفحه هدف در هر مرحله از تکرار، دقیقاً در عدد M (تعداد گره‌های جعلی) ضرب می‌شود. این چرخه بازخورد مثبت باعث می‌شود که صفحه T به سرعت و به صورت نمایی، تمام امتیازات گراف محلی را ببلعد و به عنوان برترین **Authority** شناخته شود.

نمودار شماتیک گرافیکی: کالبدشکافی مقاومت الگوریتم‌ها



گام 3) نتیجه‌گیری و مقایسه نهایی

بر اساس تحلیل‌های ریاضی فوق:

آسیب‌پذیرترین الگوریتم: الگوریتم **HITS** به شدت در برابر حمله سیل آسیب‌پذیر است.

تأثیرپذیری از «تعداد» لینک‌ها: الگوریتم **HITS** بسیار سریع‌تر تحت تأثیر صرف کمیّت و تعداد لینک‌های ورودی جعلی قرار می‌گیرد. در **HITS**، لینک‌ها بدون نیاز به داشتن اعتبار قبلی، می‌توانند با ایجاد یک حلقه بازخورد با صفحه هدف، اعتبار (**Hub Score**) تولید کنند. اما در **PageRank**، تا زمانی که کیفیت (اعتبار اولیه) وجود نداشته باشد، کمیّت لینک‌های اسپم بی‌اثر و خنثی می‌شود.

نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

تفاوت ارزیابی محلی (Local) و سراسری (Global):

این مسئله نشان‌دهنده یک اصل بنیادین در طراحی سیستم‌های توزیع‌شده است. الگوریتم‌هایی که اعتبار را به صورت «سراسری» (Global) محاسبه می‌کنند و ریشه در کل شبکه دارند (مانند **PageRank** و **TrustRank**)، بسیار دیرتر فریب می‌خورند.

اما الگوریتم‌هایی که تنها به توپولوژی «محلی» (Local) در زمان اجرای کوئری متکی هستند (مانند **HITS** کلاسیک)، به راحتی توسط یک مهاجم با ایجاد یک زیرگراف متراکم مصنوعی، دچار فروپاشی محاسباتی و ربایش موضوعی (**Topic Hijacking**) می‌شوند. به همین دلیل موتورهای جستجو هیچ‌گاه از **HITS** به صورت خام و محافظت‌نشده برای رتبه‌بندی نهایی وب استفاده نمی‌کنند.

پاسخنامه مسئله 21.32: رتبه صفحه برای محتوای پویا (PageRank for Dynamic Content)

مقدمه و هدف مسئله

در شبکه‌های اجتماعی با جریان داده پیوسته (**Streaming Data**) مانند توییتر، ساختار گراف (توییت‌ها، لایک‌ها، فالوورها) در هر میلی‌ثانیه تغییر می‌کند. اجرای الگوریتم **PageRank** کلاسیک (**Power Iteration**) که دارای پیچیدگی از مرتبه $O(E)$ برای کل گراف است، در این مقیاس زمانی کاملاً غیرممکن است.

هدف این مسئله، طراحی یک «الگوریتم تقریبی و افزایشی» (**Incremental Approximation Algorithm**) است که به جای محاسبه مجدد کل گراف، تنها «تغییرات» را در یک همسایگی محلی (**Local Neighborhood**) منتشر کند تا رتبه نودهای جدید در لحظه (**Real-time**) محاسبه شود.

گام 1) اصل میرایی و شعاع محلی (Decay Principle & Local Radius)

اساس ریاضیاتی که به ما اجازه می‌دهد گراف‌های عظیم را به صورت محلی تقریب بزنیم، تأثیر «عامل میرایی» (**Damping Factor - d**) است. در الگوریتم **PageRank**، تأثیر یک گره بر روی همسایگان با فاصله k ، با ضریب d^k کاهش می‌یابد. اگر $d = 0.85$ باشد:

تأثیر روی همسایه درجه 1: 0.85

تأثیر روی همسایه درجه 3: $0.85^3 \approx 0.61$

تأثیر روی همسایه درجه 6: $0.85^6 \approx 0.37$

این خاصیت نمایی نشان می‌دهد که ایجاد یک گره یا یال جدید، یک «شعاع انفجار محدود» (**Limited Blast Radius**) دارد. بنابراین، برای به‌روزرسانی رتبه یک گره، نیازی به پیمایش کل گراف نیست و بررسی یک همسایگی محلی (مثلاً تا عمق 3 یا 4) تقریب بسیار دقیقی (با خطای کمتر از ϵ) ارائه می‌دهد.

گام 2) طراحی الگوریتم انتشار محلی (Local Push Algorithm)

برای پیاده‌سازی این تقریب در لحظه، از الگوریتمی بر پایه «بردار پسماند» (Residual Vector) استفاده می‌کنیم. در این سیستم، ما 2 مقدار برای هر گره i ذخیره می‌کنیم:

1. مقدار $PR(i)$: رتبه صفحه تثبیت‌شده فعلی.

2. مقدار $R(i)$: پسماند (مقدار تغییری که گره دریافت کرده اما هنوز به همسایگانش منتقل نکرده است).

سناریوی ایجاد یال جدید (مثال: کاربر A کاربر B را فالو می‌کند):

1. محاسبه تغییر اولیه: با ایجاد لینک جدید $A \rightarrow B$ ، توزیع امتیازات خروجی A تغییر می‌کند. سهم جدیدی از امتیاز

A باید به B برسد. این سهم به بردار پسماند B یعنی $R(B)$ اضافه می‌شود.

2. عملیات انتشار (Push): سیستم دارای یک حد آستانه خطای مجاز ϵ (مثلاً 0.001) است. الگوریتم به صورت

بازگشتی گره‌هایی که $R(i) > \epsilon$ دارند را انتخاب می‌کند.

3. تخلیه پسماند: ریاضیات تخلیه و انتشار پسماند به این شکل اعمال می‌شود:

$$PR_{new}(i) = PR_{old}(i) + \alpha \cdot R(i)$$
$$R_{new}(j) += \frac{(1 - \alpha) \cdot R(i)}{N_{out}(i)} \quad \forall j \in Out(i)$$
$$R_{new}(i) = 0$$

(بخشی از پسماند به رتبه خود گره اضافه شده، مابقی مساوی بین همسایگان تقسیم می‌شود و در نهایت پسماند گره صفر می‌گردد).

4. شرط توقف: این عملیات به صورت آبخاری در گراف محلی ادامه می‌یابد تا زمانی که پسماند تمام گره‌های آن ناحیه

از ϵ کوچکتر شود.

نمودار شماتیک گرافیکی: معماری پردازش گراف بلادرنگ (تراز هندسی مطلق)



گام 3) تقریب مونت کارلو برای محتوای زنده (Monte Carlo Approximation)

برای توییت‌های کاملاً جدید (**New Nodes**) که در همان ثانیه تولید شده‌اند، حتی الگوریتم **Push** نیز ممکن است کند باشد. در موتورهای جستجوی بی‌درنگ، از روش گشت‌های تصادفی مونت کارلو (**Monte Carlo Random Walks**) استفاده می‌شود:

1. به محض تولید محتوای جدید (گره v)، موتور محاسباتی K گشت تصادفی کوتاه (با طول متوسط $\frac{1}{1-d}$) را در جهت معکوس (**Reverse Random Walk**) آغاز می‌کند.

2. این گشت‌ها از گره v شروع شده و از طریق لینک‌های ورودی (کاربرانی که این توییت را ریتوییت کرده‌اند یا نویسنده اصلی) در گراف حرکت می‌کنند.

3. اگر این گشت‌های تصادفی به گره‌هایی برسند که در سیستم دارای **PageRank** بسیار بالایی هستند (مثل اکانت‌های مرجع)، گره جدید v بلافاصله بخش کسری از آن اعتبار را به دست می‌آورد.

4. رتبه نهایی گره v متناسب با مجموع رتبه‌های گره‌های ملاقات‌شده در این شبیه‌سازی محلی خواهد بود.

نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

تغییر پارادایم از Pull به Push:

در جبر خطی کلاسیک، الگوریتم **PageRank** یک الگوریتم **Pull-based** است (هر گره در هر تکرار، امتیازات را از همسایگان ورودی خود «بیرون می‌کشد»). این روش برای گراف‌های ایستا عالی است.

اما الگوریتم تقریب محلی، یک رویکرد **Push-based** است. یعنی گرهی که تغییر در آن رخ داده (منبع تغییر)، موج این تغییر را به سمت همسایگان خروجی خود «هل می‌دهد». در مهندسی سیستم‌های بزرگ‌مقیاس توزیع‌شده، تغییر زاویه دید از **Pull** به **Push**، کلید معماری حل مشکل گراف‌های پویا و پردازش در لحظه (**Streaming Processing**) است.

پاسخنامه مسئله 21.33: وزن‌دهی به یال‌ها بر اساس رفتار کاربر

(Edge Weighting via User Behavior)

مقدمه و هدف مسئله

در فرمول کلاسیک **PageRank**، فرض بر این است که «موج‌سوار تصادفی» (**Random Surfer**) کورکورانه عمل می‌کند و احتمال کلیک روی هر لینک خروجی از یک صفحه کاملاً برابر است. اما در دنیای واقعی، کاربران روی لینک‌هایی که در بالای صفحه، با فونت بزرگتر، یا در بافت متنی جذاب‌تری قرار دارند، بیشتر کلیک می‌کنند. هدف این مسئله، گذار از مدل «موج‌سوار تصادفی» به مدل «موج‌سوار واقع‌گرایانه/هوشمند» با تزریق لاگ‌های رفتار کاربر (**Click-through Rate - CTR**) به درون ماتریس مارکوف است.

گام 1) بازنویسی مدل مارکوف (Transition Probabilities)

در مدل استاندارد، احتمال انتقال از صفحه i به صفحه j به صورت توزیع یکنواخت تعریف می‌شود:

$$P_{ij} = \frac{1}{N_{out}(i)}$$

که در آن $N_{out}(i)$ تعداد کل لینک‌های خروجی از صفحه i است. برای وزن‌دهی بر اساس رفتار کاربر، فرض کنید نرخ کلیک (CTR) لینک متصل‌کننده صفحه i به صفحه j را با $CTR(i, j)$ نشان دهیم. برای اینکه این مقادیر تبدیل به یک احتمال معتبر شوند، باید آن‌ها را بر مجموع کلیک‌های خروجی از صفحه i نرمال‌سازی کنیم. بنابراین، فرمول جدید احتمال گذار به شکل زیر بازنویسی می‌شود:

$$P_{ij} = \frac{CTR(i, j)}{\sum_k CTR(i, k)}$$

در این فرمول جدید، اگر یک لینک 80% کلیک‌های خروجی یک صفحه را به خود اختصاص داده باشد، موج‌سوار مارکوف نیز با احتمال 0.8 از آن عبور خواهد کرد و جریان اعتبار از این مسیر به شدت تقویت می‌شود.

گام 2) چالش نقض ماتریس تصادفی (Stochastic Matrix Violation)

برای اینکه الگوریتم رتبه‌بندی (مانند **Power Iteration**) همگرا شود، ماتریس گذار باید «تصادفی سطری» (Row-stochastic) باشد؛ یعنی مجموع احتمالات در هر سطر باید دقیقاً برابر با 1 باشد: $\sum_j P_{ij} = 1$. استفاده خام از فرمول بالا، 2 چالش ویرانگر در ریاضیات سیستم ایجاد می‌کند:

1. تقسیم بر صفر (Zero-Denominator): اگر صفحه‌ای دارای لینک خروجی باشد، اما به هر دلیلی (مثلاً ترافیک پایین

سایت یا قرار گرفتن لینک‌ها در فوتر) تاکنون هیچ کاربری روی هیچ‌کدام از لینک‌های آن کلیک نکرده باشد، مجموع مخرج کسر برابر صفر خواهد شد ($\sum CTR = 0$). این امر باعث تولید مقادیر تعریف‌نشده (NaN) در ماتریس می‌گردد و کل محاسبات را متوقف می‌کند.

2. مشکل شروع سرد (Cold Start) برای لینک‌های جدید: اگر یک لینک حیاتی و ارزشمند همین امروز به صفحه اضافه شده باشد، مقدار CTR آن صفر است. در این مدل خام، احتمال عبور از این لینک 0 در نظر گرفته می‌شود و گراف عملاً دچار قطعی یال (Edge Deletion) می‌گردد.

گام 3) راه‌حل سیستمی (هموارسازی و ترکیب خطی)

برای حفظ خاصیت تصادفی سطری و حل مشکل دیتای ناموجود (**Sparsity**)، در سیستم‌های بازیابی اطلاعات مدرن از تکنیک هموارسازی (**Smoothing**) یا ترکیب خطی مدل‌ها استفاده می‌شود. ما مدل رفتار کاربر را با مدل یکنواخت کلاسیک ترکیب می‌کنیم:

$$P_{ij}^{new} = \gamma \left(\frac{CTR(i, j)}{\sum_k CTR(i, k)} \right) + (1 - \gamma) \left(\frac{1}{N_{out}(i)} \right)$$

تحلیل پارامتر γ (وزن اعتماد):

متغیر $\gamma \in [0, 1]$ یک پارامتر کنترل‌کننده است که «میزان اعتماد» سیستم به دیتای کلیک را مشخص می‌کند. این پارامتر

می‌تواند یک عدد ثابت باشد، یا به صورت پویا (**Dynamic**) و متناسب با حجم ترافیک صفحه i تعریف شود:

برای صفحات پربازدید: اگر صفحه دیتای کلیک بسیار غنی و معتبری داشته باشد، مقدار $1 \rightarrow \gamma$ میل می‌کند و

سیستم کاملاً رفتار محور می‌شود.

برای صفحات کم‌بازدید یا لینک‌های جدید: اگر مجموع کلیک‌های صفحه نزدیک به صفر باشد، مقدار $0 \rightarrow \gamma$ میل

می‌کند. در این حالت بخش اول فرمول خنثی شده و سیستم برای جلوگیری از خطای ریاضی، به همان توزیع یکنواخت

کلاسیک ($1/N_{out}$) عقب‌نشینی (**Fallback**) می‌کند.

نمودار شماتیک گرافیکی: تکامل توزیع احتمال در مدل‌های مارکوف



نکته آموزشی و کاربردی (Takeaway) برای دانشجویان

تکامل از وب‌سندگرا به وب‌کاربرگرا:

این بازنویسی دقیق ریاضی، همان جهش تکنولوژیکی است که موتورهای جستجو در گذر از نسل دوم (مبتنی بر لینک خام مانند پیج‌رنک اولیه) به نسل سوم (مبتنی بر سیگنال‌های کاربری) تجربه کردند.

با این بهینه‌سازی، یک لینک پنهان در فوتر سایت که هیچ کاربری روی آن کلیک نمی‌کند، ارزش انتقال اعتبار (**PageRank Flow**) خود را عملاً از دست می‌دهد، در حالی که لینکی در بدنه اصلی مقاله که ترافیک بالایی جذب می‌کند، به یک مسیر قطور و اصلی برای انتقال اعتبار (**Authority**) در گراف اینترنت تبدیل می‌شود.

پاسخنامه مسئله 21.34: هیتس در گراف‌های دوگانه ناقص

(HITS in Imperfect Bipartite Graphs)

بخش 1: مقدمه و هدف مسئله

در الگوریتم کلاسیک **HITS**، فرض بر این است که صفحات وب به **2** لایه کاملاً مجزا (گراف دوگانه خالص) تقسیم می‌شوند: هاب‌ها (لینک‌دهنده‌ها) و اتوریتی‌ها (محتواها). اما در گراف واقعی وب، سایت‌هایی مانند ویکی‌پدیا یا مقالات مروری (**Review Papers**)، به طور همزمان هم محتوای غنی دارند و هم به منابع معتبر لینک می‌دهند. هدف این مسئله، فرمول‌بندی یک «حلقه بازخورد داخلی» (**Self-feedback Loop**) در ریاضیات **HITS** است تا نشان دهیم چگونه اعتبار ذاتی یک صفحه می‌تواند مستقیماً بر قدرت راهبری آن تأثیر بگذارد.

بخش 2: بازتعریف متغیرها و معرفی ضریب بازخورد

برای اعمال این تغییر در سیستم، یک پارامتر جدید به نام **ضریب بازخورد داخلی** (β) تعریف می‌کنیم که شرط آن $\beta > 0$ است. این ضریب مشخص می‌کند که چه کسری از امتیاز اتوریتی یک صفحه، به صورت مستقیم روی امتیاز هاب بودن همان صفحه تأثیر می‌گذارد (و بالعکس). از نظر توپولوژی گراف، این کار دقیقاً معادل اضافه کردن یک «حلقه به خود» (**Self-loop**) با وزن ریاضی β برای تمامی گره‌های شبکه است.

بخش 3: گسترش معادلات سیگما (Summation Equations)

در حالت کلاسیک، گره i فقط از همسایگانش امتیاز می‌گرفت. در مدل گسترش‌یافته، گره i علاوه بر همسایگان، کسری از امتیاز ویژگی مکمل خود را نیز دریافت می‌کند. معادلات به‌روزرسانی (پیش از مرحله نرمال‌سازی) به شکل زیر بازنویسی می‌شوند:

1. به‌روزرسانی اتوریتی (Authority Update):

امتیاز اتوریتی جدید صفحه i ، برابر است با مجموع هاب تمام صفحاتی که به آن لینک داده‌اند، به علاوه ضریبی از امتیاز هاب خود صفحه i :

$$a^{(t)}(i) = \sum_{j \rightarrow i} h^{(t-1)}(j) + \beta \cdot h^{(t-1)}(i)$$

2. به‌روزرسانی هاب (Hub Update):

امتیاز هاب جدید صفحه i ، برابر است با مجموع اتوریتی تمام صفحاتی که به آن‌ها لینک داده است، به علاوه ضریبی از امتیاز اتوریتی خود صفحه i :

$$h^{(t)}(i) = \sum_{i \rightarrow j} a^{(t)}(j) + \beta \cdot a^{(t)}(i)$$

بخش 4: نمودار شماتیک تکامل گراف (Bipartite to Feedback Graph)

افزودن حلقه بازخورد β در توپولوژی شبکه

گراف دوگانه خالص (مدل کلاسیک HITS)

Hub Node → Authority Node

$$A_{ii} = 0$$



مدل بازخورد داخلی (Regularized HITS)

Node i → Node j

Self-Link: Node i → Node i (+ β)

$$\tilde{A} = A + \beta I$$

بخش 5: مدل سازی ماتریسی (Matrix Formulation)

برای تحلیل این تغییر در جبر خطی، فرض کنید A ماتریس مجاورت گراف پایه باشد. اضافه کردن حلقه به خود برای تمامی گره ها، معادل جمع کردن ماتریس A با یک ماتریس همانی (Identity Matrix) است که در مقدار اسکالر β ضرب شده است. ماتریس مجاورت اصلاح شده ($\tilde{A}$) به این صورت تعریف می شود:

$$\tilde{A} = A + \beta I$$

با جایگذاری ماتریس جدید در فرمول های پایه **HITS**، معادلات ماتریسی گسترش یافته به صورت کاملاً فشرده و کارآمد به دست می آیند:

$$h^{(t)} = \tilde{A} a^{(t-1)}$$

$$a^{(t)} = \tilde{A}^T h^{(t)}$$

بخش 6: جمع بندی

منظم سازی ماتریس (Matrix Regularization):

در کلاس های پیشرفته ی بازایی اطلاعات، این اثبات نشان دهنده یک مفهوم کلیدی برای مهندسی شبکه های پیچیده است؛ این فرمول بندی ریاضی فراتر از یک تغییر ساده است، در علوم شبکه به این تکنیک **Regularization** (منظم سازی) ماتریس مجاورت گفته می شود.

در گراف های وب که توپولوژی بسیار تنک و پراکنده ای (**Sparse**) دارند، الگوریتم خام **HITS** ممکن است دچار نوسان شود یا وزن غیرمنطقی و کاذبی به گره های خاصی بدهد. با اضافه کردن عبارت جبر خطی βI ، ماتریس ها اصطلاحاً «خوش رفتارتر» می شوند.

این حلقه بازخورد (**Self-feedback**) باعث می‌شود صفحاتی که به طور همزمان هر 2 نقش (هاب بودن و اتوریته بودن) را به خوبی ایفا می‌کنند، پاداش مضاعف و همافزا (**Synergistic**) دریافت کنند. در نتیجه، این صفحات غنی و چندبعدی در صفحه نتایج موتور جستجو جایگاه بسیار پایدارتر و مقاوم‌تری نسبت به سایت‌های تک‌بعدی (که فقط محتوا دارند یا فقط لینک می‌دهند) کسب خواهند کرد.

پاسخنامه مسئله 21.35: تقلب در اتوریته با استفاده از هاب‌های جعلی

(Authority Manipulation via Fake Hubs)

بخش 1: مقدمه و هدف مسئله

یکی از ظریف‌ترین و هوشمندانه‌ترین حملات سئوی کلاه سیاه علیه الگوریتم **HITS**، تکنیک «سرقت اعتبار هاب» (**Hub Score Hijacking**) است. در این سناریو، مهاجم به جای اینکه سعی کند مستقیماً برای سایت خود لینک جمع کند، سعی می‌کند ارزش لینک‌های خودش را به صورت مصنوعی بالا ببرد. هدف این مسئله، کالبدشکافی ریاضی این فریب در فضای گراف دوگانه و ارائه پچ‌های الگوریتمی برای مقابله با آن است.

بخش 2: مکانیزم فریب (تورم غیرواقعی امتیاز هاب)

بر اساس قوانین ریاضی و جبر خطی الگوریتم **HITS**، امتیاز هاب یک صفحه، مستقیماً برابر است با مجموع امتیاز اتوریته تمام صفحاتی که آن صفحه به آن‌ها لینک داده است:

$$h^{(t)}(i) = \sum_{i \rightarrow j} a^{(t-1)}(j)$$

مهاجم 100 وبلاگ جعلی می‌سازد. در تمام این وبلاگ‌ها، به سایت‌های مرجع و قدرتمندی نظیر ویکی‌پدیا و بی‌بی‌سی لینک می‌دهد. از آنجا که این مراجع دارای اتوریته‌های نجومی در شبکه وب هستند، بر اساس فرمول فوق، مجموع این امتیازات به سمت وبلاگ‌های جعلی مهاجم سرازیر می‌شود.

نتیجه فاز اول (فریب سیستم):

سیستم فریب می‌خورد و از نظر ریاضی تصور می‌کند که این 100 وبلاگ جعلی، به دلیل ارجاع به منابع بسیار معتبر جهانی، «هاب‌های فوق‌العاده باکیفیت و راهنمایان ارزشمندی» در شبکه هستند. بنابراین، امتیاز هاب (h) برای تک‌تک این وبلاگ‌های اسپم به شدت افزایش یافته و متورم می‌شود.

بخش 3: سرازیر شدن اعتبار مصنوعی به سایت هدف

در گام دوم از تکرار الگوریتم، نوبت به محاسبه امتیاز اتوریته برای سایت کلاهبردار اصلی یا سایت هدف (**Target Site**) می‌رسد. امتیاز اتوریته سایت هدف، برابر است با مجموع امتیاز هاب تمام صفحاتی که به آن لینک داده‌اند:

$$a^{(t)}(Target) = \sum_{i \rightarrow Target} h^{(t)}(i)$$

مهاجم در آن 100 وبلاگ جعلی، علاوه بر لینک‌های ویکی‌پدیا، یک لینک هم به سایت هدف خود داده است. چون در گام قبلی، امتیاز هاب این وبلاگ‌ها به صورت مصنوعی (با سوءاستفاده از نام مراجع) متورم شده بود، اکنون تمام این اعتبار نامشروع، یک‌جا در متغیر $a^{(t)}$ تجمیع شده و به سایت هدف تزریق می‌گردد.

فاجعه ساختاری در رتبه‌بندی:

سایت هدف مهاجم در صفحه نتایج موتور جستجو، در کنار ویکی‌پدیا و بی‌بی‌سی به عنوان یکی از برترین اتوریتی‌های گراف شناخته خواهد شد؛ بدون اینکه حتی یک سایت واقعی و معتبر در کل اینترنت به آن لینک داده باشد.

بخش 4: نمودار شماتیک کالبدشکافی حمله



بخش 5: پیچ کردن سیستم (طراحی راهکارهای دفاعی)

برای رفع این آسیب‌پذیری مهلک ساختاری، مهندسين جستجو تغییرات زیر را در هسته الگوریتم اعمال می‌کنند:

1. پیچ اول: هیتس کسری (Normalized HITS)

بزرگترین باگ HITS خام این است که یک هاب، تمام قدرت خود را بدون هیچ کاهش‌ی به تمامی لینک‌های خروجی‌اش

می‌دهد. برای پچ کردن این باگ، مشابه پیچ‌رنک، امتیاز هاب را بر تعداد کل لینک‌های خروجی (N_{out}) تقسیم می‌کنیم.

$$a^{(t)}(i) = \sum_{j \rightarrow i} \frac{h^{(t-1)}(j)}{N_{out}(j)}$$

با این پچ، اگر وبلاگ جعلی به 100 سایت معتبر لینک بدهد تا هاب خود را بالا ببرد، در زمان انتقال این اعتبار به سایت هدف، قدرت آن بر 100 تقسیم شده و عملاً اثر توری آن خنثی می‌شود.

2. پچ دوم: وزن‌دهی مبتنی بر شباهت متنی (Textual Similarity Weighting)

سیستم به جای پذیرش کورکورانه گراف، متن وبلاگ جعلی را با متن سایت مقصد از طریق بردارسازی مقایسه می‌کند. سایت‌های ویکی‌پدیا و بی‌بی‌سی دارای محتوای بسیار غنی هستند. اگر وبلاگ جعلی صرفاً مجموعه‌ای از لینک‌ها و متن‌های بی‌معنی یا کپی شده باشد، شباهت کسینوسی (**Cosine Similarity**) بردارهای متنی آن‌ها به صفر میل می‌کند. با ضرب این شباهت در وزن یال گراف، ارتباط بین وبلاگ جعلی و ویکی‌پدیا در ریاضیات سیستم قطع شده و وبلاگ نمی‌تواند اعتبار هاب را سرقت کند.

3. پچ سوم: فیلتر ناهنجاری ساختاری (Structural Anomaly Detection)

سیستم‌های گراف‌کاوی به طور پیوسته به دنبال تقارن‌های غیرطبیعی می‌گردند. اینکه دقیقاً 100 سایت مختلف، دقیقاً به سه سایت یکسان (ویکی‌پدیا، بی‌بی‌سی، و یک سایت کاملاً ناشناخته) لینک بدهند، از نظر آماری در وب ارگانیک محال است. این الگوی متراکم (**Dense Bipartite Subgraph**) توسط الگوریتم‌های ضداسپم به عنوان یک «مزرعه لینک» (**Link Farm**) شناسایی شده و کل این زیرگراف پیش از ورود به ماتریس **HITS**، قرنطینه و حذف می‌گردد.

بخش 6: جمع‌بندی

در مدل‌سازی ریاضی وب، مرز بین یک «دایرکتوری ارزشمند» (مانند ساختار اولیه پورتال یاهو) و یک «مزرعه لینک اسپم» بسیار باریک است؛ زیرا از منظر توپولوژی خالص، هر دو ساختار، لینک‌های خروجی بسیار فراوان و متمرکزی دارند. این آسیب‌پذیری در الگوریتم پایه، به زیبایی نشان می‌دهد که چرا فرمول‌های بازگشتی خام در تحلیل شبکه‌های پیچیده به تنهایی در دنیای واقعی و خصمانه وب شکست می‌خورند؛ ریاضیات نیازمند «بافتار» (**Context**) است. پیوند دادن الگوریتم‌های گراف (تحلیل ساختار پیوندها) با الگوریتم‌های پیشرفته‌ی پردازش زبان طبیعی (**NLP** - تحلیل محتوا)، تنها راه مهندسی ایمن برای تشخیص نیت و قصد (**Malicious Intent**) در پشت ساختارهای به ظاهر قانونی است تا از دستکاری اعتبارات جلوگیری به عمل آید.

پاسخنامه مسئله 21.36: طراحی سیستم ترکیبی بازیابی اطلاعات (Hybrid Retrieval System)

بخش 1: مقدمه و هدف مسئله

یک موتور جستجوی مدرن نمی‌تواند تنها بر یک سیگنال متکی باشد. سیستم‌های پیشرو از یک «خط لوله چندمرحله‌ای» (**Multi-stage Pipeline**) استفاده می‌کنند تا تعادلی میان سرعت (برای میلیاردها صفحه) و دقت (برای نتایج نهایی) ایجاد کنند.

هدف این مسئله، فرمول‌بندی یک تابع امتیازدهی است که سیگنال‌های محتوایی (**TF-IDF**)، سیگنال‌های ساختاری جهانی (**PageRank**) و سیگنال‌های ساختاری محلی (**HITS**) را تلفیق می‌کند.

بخش 2: معماری خط لوله (Pipeline Architecture)

برای جلوگیری از فروپاشی پردازشی در مقیاس وب، امتیازدهی را در 3 فاز متوالی و فیلترشونده اجرا می‌کنیم:

1. فاز بازیابی سریع (Retrieval): استفاده از ایندکس معکوس برای یافتن سریع مقالات مرتبط بر اساس **TF-IDF** ترکیبی با **Anchor Text** از میان میلیاردها سند.

2. فاز فیلترینگ اولیه (Pruning): انتخاب 1000 نتیجه برتر بر اساس امتیاز مرحله قبل برای کاهش بار محاسباتی در گراف.

3. فاز رتبه‌بندی دقیق (Re-ranking): اجرای الگوریتم **HITS** منحصراً روی زیرگراف این 1000 صفحه و اعمال امتیاز نهایی.

بخش 3: تابع نهایی رتبه‌بندی (Scoring Function)

تابع امتیاز نهایی $S(q, d)$ برای پرس‌وجوی q و سند d به صورت ترکیبی خطی در فضای لگاریتمی تعریف می‌شود تا اثرات مختلف هم‌افزایی (**Synergy**) داشته باشند:

$$S(q, d) = \lambda_1 \cdot \text{Score}_{\text{text}} + \lambda_2 \cdot \log(\text{PR}_{\text{off}}) + \lambda_3 \cdot \text{Auth}_{\text{on}}$$

اجزای فرمول ریاضی:

$\text{Score}_{\text{text}}$: ترکیبی از **TF-IDF** متن خودِ سند و متون لینک‌های ورودی (**Anchor Texts**) به آن سند.

$\log(\text{PR}_{\text{off}})$: استفاده از لگاریتم بر روی توزیع نمایی **PageRank** جهانی؛ این کار برای جلوگیری از غلبه‌ی مخرب و شدید صفحات بسیار محبوب (نظیر ویکی‌پدیا) بر سایر نتایج مرتبط انجام می‌شود.

Auth_{on} : امتیاز اعتباری (**Authority**) خروجی از الگوریتم محلی **HITS** که در لحظه (**On-the-fly**) روی زیرگراف محاسبه شده است.

بخش 4: نمودار شماتیک خط لوله ترکیبی و کالیبراسیون

معماری خط لوله پردازشی و کالیبراسیون هوشمند

۱. فاز بازیابی سریع (Retrieval)

TF-IDF + Anchor Text

فیلتر اولیه از میان میلیاردها سند وب



۲. فاز رتبه‌بندی دقیق (Re-ranking)

HITS Algorithm



۳. خروجی و تولید امتیاز نهایی

 $S(q, d)$ Final Score

منطق کالبراسیون هوشمند (LTR)

پرسوجوی اطلاعاتی: افزایش ضریب متن (بزرگ شدن λ_1)پرسوجوی ناوبری: افزایش ضریب پیچرنک (بزرگ شدن λ_2)پرسوجوی مبهم: افزایش ضریب هیتس (بزرگ شدن λ_3)

بخش 5: کالبراسیون و تعیین پارامترها

پارامترهای $\lambda_1, \lambda_2, \lambda_3$ وزنهای اهمیتی هستند که باید با استفاده از دادههای آموزشی تنظیم شوند:

1. روش تنظیم (Learning to Rank - LTR): از الگوریتمهای یادگیری ماشین مانند RankNet یا LambdaMART استفاده

میکنیم تا با تغذیه دادههای آموزشی (شامل کلیکهای کاربران)، بهینهترین مقادیر برای متغیرهای λ_i استخراج گردد.

2. مکانیزم کالبراسیون هوشمند نسبت به نیت کاربر (Intent):

اگر پرسوجو اطلاعاتی باشد (مثل "تاریخچه تکامل پایتون"): وزن λ_1 (سیگنالهای متنی) توسط مدل افزایش مییابد.اگر پرسوجو ناوبری باشد (مثل "سایت ورود به دانشگاه شریف"): وزن λ_2 (اعتبار جهانی PageRank) افزایش مییابد.اگر پرسوجو مبهم باشد (مثل کلمه "عقاب"): وزن λ_3 (تحلیل معنایی محلی HITS) بیشترین ارزش را پیدا می کند.

بخش 6: تحلیل پیادهسازی سیستمی

برای پیادهسازی مهندسی این خط لوله، دو رویکرد ریاضی دیگر نیز الزامی است:

مدیریت ارزش متن لینک: وزن کلمات موجود در متن لینکها (Anchor Text) را پیش از محاسبه در تابع TF-IDF، ضربدر

یک ضریب تقویتی $k > 1$ میکنیم، زیرا متن لینکها توسط انسانها نوشته شده و معمولاً بسیار توصیفیتر و خلاصهتر از متن اصلی سند هستند.

نرمالسازی آماری (Z-Score Normalization): از آنجا که امتیازات (متنی، پیچرنک و هیتس) مقیاسها و دامنههای کاملاً

متفاوتی دارند، پیش از ترکیب خطی، امتیاز هر بخش را با استفاده از متدولوژی Z-Score به توزیعی با میانگین 0 و انحراف

معیار 1 منتقل میکنیم تا از غلبه یک سیگنال بر سیگنالهای دیگر جلوگیری نماییم.

بخش 7: جمع بندی

توازن میان سیگنال ایستا و پویا:

این معماری ترکیبی، نمایانگر یکی از بزرگترین دروس در طراحی موتورهای جستجو است: الگوریتم **PageRank** به ما نشان می‌دهد که ساختار وب در مقیاس کلان و مستقل از پرس‌وجو چگونه است (سیگنال ایستا)، در حالی که الگوریتم **HITS** به ما نشان می‌دهد که زیرگراف وب در لحظه و دقیقاً متناسب با پرس‌وجوی کاربر چگونه سازماندهی شده است (سیگنال پویا).

تلفیق ریاضیاتی این دو سیگنال با روش‌های یادگیری ماشین، سیستم را در برابر حملات سئوی کلاه سیاه (**Black-Hat SEO**) به شدت مقاوم می‌سازد؛ زیرا یک مهاجم اسپمر محال است بتواند به طور همزمان، هم پیچ‌رنک سراسری سایت‌های معتبر را فریب دهد و همزمان یک گراف محلی متراکم و معتبر را در لحظه‌ی جستجوی هر کاربر شبیه‌سازی کند.